

**Министерство науки и высшего образования Российской Федерации**  
**Федеральное государственное автономное образовательное учреждение**  
**высшего образования**  
**«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»**

**Институт перспективной инженерии**

**Департамент цифровых, робототехнических систем и электроники**

**Утверждена приказом проректора  
по образовательной деятельности  
от 29.12.2025 г. № 3251-О**

**Допущена к защите**

**«\_\_\_» июня 2026 г.**

И. о. директора департамента цифровых,  
робототехнических систем и электроники  
кандидат экономических наук, доцент  
Азаров И. В.

(подпись)

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА**  
**(КОМПЛЕКСНАЯ РАБОТА)**

*Проектирование, оптимизация и реализация программного комплекса*

*казуальной 2D-игры на основе жанра линейного автораннера*

Руководитель (координатор) выпускной квалификационной работы (комплексной работы):  
Братченко Наталья Юрьевна, канд. физико-математических наук, доцент департамента цифровых,  
робототехнических систем и электроники института перспективной инженерии

**Раздел:** Проектирование и оптимизация модульного ядра для 2D-автораннера

**Выполнил:**

*Магомедов Ильяс Аскерович*

**студент 4 курса, группы ПИЖ-б-о-22-1**

**направления подготовки 09.03.04 «Программная инженерия»**

**Направленность (профиль) «Разработка и сопровождение программного обеспечения»**

**очной формы обучения**

**Руководитель:**

*Братченко Наталья Юрьевна*

*канд. физико-математических наук, доцент  
департамента цифровых,  
робототехнических систем и электроники*

(подпись)

(подпись)

**Раздел:** Реализация и балансировка игрового процесса в линейном 2D-автораннере

**Выполнил:**

*Коровин Николай Александрович*

**студентка 4 курса, группы ПИЖ-б-о-22-1**

**направления подготовки 09.03.04 «Программная инженерия»**

**Направленность (профиль) «Разработка и сопровождение программного обеспечения»**

**очной формы обучения**

**Руководитель:**

*Братченко Наталья Юрьевна*

*канд. физико-математических наук, доцент  
департамента цифровых,  
робототехнических систем и электроники*

(подпись)

(подпись)

**Нормоконтролер:**

*Новикова Елена Николаевна, канд. физико-математических наук, доцент, доцент департамента цифровых, робототехнических систем и электроники*

(подпись)

**Дата защиты**

**«26» июня 2026 г.**

**Оценка** \_\_\_\_\_

Ставрополь, 2026

Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное учреждение  
высшего образования  
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии  
Департамент цифровых, робототехнических систем и электроники  
Направление подготовки 09.03.04 «Программная инженерия»  
Направленность (профиль) «Разработка и сопровождение программного обеспечения»

«УТВЕРЖДАЮ»

И. о. директора департамента цифровых,  
робототехнических систем и электроники  
кандидат экономических наук, доцент  
И. В. Азаров

(подпись)

«29» декабря 2025 г.

**ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ  
(КОМПЛЕКСНАЯ РАБОТА)**

**1. Тема выпускной квалификационной работы (комплексной работы)**

Проектирование, оптимизация и реализация программного комплекса казуальной 2D-игры на основе жанра линейного автораннера

**2. Тема раздела выпускной квалификационной работы (комплексной работы)**

Проектирование и оптимизация модульного ядра для 2D-автораннера

Утверждена приказом проректора по образовательной деятельности от 29.12.25 г. №3251-О

Студент Магомедов Ильяс Аскерович ПИЖ-б-о-22-1

Направление подготовки 09.03.04 «Программная инженерия»

Направленность (профиль) «Разработка и сопровождение программного обеспечения»

**2.1. Исходные данные для проектирования**

Материалы, полученные во время выполнения преддипломной практики.

**2.2. Структура и содержание раздела выпускной квалификационной работы (комплексной работы)**

2.1 Назначение модульного ядра в программном комплексе 2D-автораннера

2.2 Выбор и обоснование архитектурного стиля

2.3 Моделирование архитектуры с использованием подхода C4

2.3.1 Диаграмма контекста системы

2.3.2 Диаграмма контейнеров

2.3.3 Диаграмма компонентов игрового ядра

2.4 Проектирование пользовательского интерфейса

2.4.1 Разработка прототипов экранных форм

2.4.2 Описание навигации и пользовательских сценариев

2.5 Детальное проектирование ключевых классов ядра

2.5.1 Диаграммы классов ключевых компонентов

2.5.2 Диаграммы последовательности для основных сценариев

2.5.3 Применение шаблонов проектирования

2.6 Оптимизация модульного ядра и генерации объектов

2.7 Обоснование выбора стека технологий и инструментов разработки

2.8 Практическая проверка проектных решений

2.9 Выводы по разделу

Дата выдачи задания 29.12.2025 г.

Руководитель (координатор) работы \_\_\_\_\_ *Н.Ю. Братченко*  
подпись инициалы, фамилия

Руководитель раздела выпускной квалификационной работы (комплексной работы)  
\_\_\_\_\_ *Н.Ю. Братченко*  
подпись инициалы, фамилия

Задание к исполнению принял \_\_\_\_\_ *И.А. Магомедов*

### **3. Тема раздела выпускной квалификационной работы (комплексной работы)**

Реализация и балансировка игрового процесса в линейном 2D-автораннере

Утверждена приказом проректора по образовательной деятельности от 29.12. 25 г. № 3251-О

Студент Коровин Николай Александрович ПИЖ-б-о-22-1

Направление подготовки 09.03.04 «Программная инженерия»

Направленность (профиль) «Разработка и сопровождение программного обеспечения»

#### **3.1. Исходные данные для проектирования**

Материалы, полученные во время выполнения преддипломной практики.

#### **3.2. Структура и содержание раздела выпускной квалификационной работы (комплексной работы)**

##### 3.1 Организация процесса разработки

###### 3.1.1 Выбор и обоснование методологии разработки

###### 3.1.2 Организация репозитория и системы контроля версий

##### 3.2 Соответствие реализации спроектированной архитектуре

###### 3.2.1 Сопоставление компонентов архитектуры и модулей реализации

###### 3.2.2 Организация файловой структуры проекта

###### 3.2.3 Конфигурационные файлы и управление настройками Unity

##### 3.3 Детализация ключевых компонентов

###### 3.3.1 Реализация управления персонажем

###### 3.3.2 Реализация системы препятствий

###### 3.3.3 Реализация бонусов и временных эффектов

###### 3.3.4 Реализация системы очков, дистанции и рекорд

###### 3.3.5 Реализация пользовательского интерфейса игрового процесса

##### 3.4 Применение принципов качественного кода

###### 3.4.1 Принципы SOLID в реализации

###### 3.4.2 Принципы чистого кода

##### 3.5 Интеграция с библиотеками и пакетами Unity

##### 3.6 Методика игровой балансировки

###### 3.6.1 Балансировка скорости и кривой сложности

###### 3.6.2 Балансировка прыжка, дополнительного прыжка и подката

##### 3.7 Функциональное тестирование механик

##### 3.8 Выводы по разделу

Дата выдачи задания 29.12.2025 г.

Руководитель (координатор) работы \_\_\_\_\_ *Н.Ю. Братченко*  
подпись инициалы, фамилия

Руководитель раздела выпускной квалификационной работы (комплексной работы)

\_\_\_\_\_ *Н.Ю.Братченко*

подпись инициалы, фамилия

Задание к исполнению принял \_\_\_\_\_ *Н.А. Коровин*

4. Срок представления выпускной квалификационной работы (комплексной работы) к защите  
«26» июня 2026 г

Дата выдачи задания 29.12.2025 г.

Руководитель (координатор) выпускной квалификационной работы (комплексной работы)

\_\_\_\_\_ *Н.Ю. Братченко*

подпись инициалы, фамилия

Руководитель раздела выпускной квалификационной работы (комплексной работы)

\_\_\_\_\_ *Н.Ю. Братченко*

подпись инициалы, фамилия

Задание к исполнению принял \_\_\_\_\_ *И.А. Магомедов*

Задание к исполнению принял \_\_\_\_\_ *Н.А. Коровин*

**Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное  
учреждение высшего образования  
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»**

**Институт перспективной инженерии**

**Департамент цифровых, робототехнических систем и электроники**

**Направление подготовки 09.03.04 «Программная инженерия»**

**Направленность (профиль) «Разработка и сопровождение программного обеспечения»**

**КАЛЕНДАРНЫЙ ПЛАН**

Фамилия, имя, отчество (полностью) Магомедов Ильяс Аскерович

Тема раздела КВКР Проектирование и оптимизация модульного ядра для 2D-автораннера

Руководитель (координатор) канд. физ.-мат. наук, доцент Братченко Наталья Юрьевна

Руководитель раздела канд. физ.-мат. наук, доцент Братченко Наталья Юрьевна

| № | Наименование этапов выполнения выпускной квалификационной работы                                                                                                                                                                    | Срок выполнения работы | Примечание |
|---|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|------------|
| 1 | Анализ предметной области и формирование требований к программному обеспечению                                                                                                                                                      | 29.12.2025-28.01.2026  | Выполнено  |
| 2 | Проектирование и оптимизация модульного ядра для 2D-автораннера: назначение модульного ядра в программном комплексе 2D-автораннера, выбор и обоснование архитектурного стиля, моделирование архитектуры с использованием подхода C4 | 29.01.2026-11.02.2026  | Выполнено  |
| 3 | Проектирование и оптимизация модульного ядра для 2D-автораннера: проектирование пользовательского интерфейса, детальное проектирование ключевых классов ядра                                                                        | 12.02.2026-18.03.2026  | Выполнено  |
| 4 | Проектирование и оптимизация модульного ядра для 2D-автораннера: оптимизация модульного ядра и генерации объектов, практическая проверка проектных решений, выводы по разделу                                                       | 19.03.2026-15.04.2026  | Выполнено  |
| 5 | Расчет технико-экономической эффективности разработки и внедрения 2D-автораннера RusRunner                                                                                                                                          | 16.04.2026-13.05.2026  | Выполнено  |
| 6 | Безопасность и экологичность разработки                                                                                                                                                                                             | 14.05.2026-02.06.2026  | Выполнено  |
| 7 | Оформление КВКР и подготовка к защите                                                                                                                                                                                               | 03.06.2026-09.06.2026  | Выполнено  |
| 8 | Представление КВКР в ГЭК                                                                                                                                                                                                            | 10.06.2026             | Выполнено  |

Руководитель координатор Н.Ю. Братченко  
(подпись)

Руководитель раздела Н.Ю. Братченко  
(подпись)

И. о. директора департамента И. В. Азаров  
(подпись)

«26» июня 2026 г.

**Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное  
учреждение высшего образования  
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»**

**Институт перспективной инженерии**

**Департамент цифровых, робототехнических систем и электроники**

**Направление подготовки 09.03.04 «Программная инженерия»**

**Направленность (профиль) «Разработка и сопровождение программного обеспечения»**

**КАЛЕНДАРНЫЙ ПЛАН**

Фамилия, имя, отчество (полностью) Коровин Николай Александрович

Тема раздела КВКР Реализация и балансировка игрового процесса в линейном 2D-автораннере

Руководитель (координатор) канд. физ.-мат. наук, доцент Братченко Наталья Юрьевна

Руководитель раздела канд. физ.-мат. наук, доцент Братченко Наталья Юрьевна

| № | Наименование этапов выполнения выпускной квалификационной работы                                                                                                                               | Срок выполнения работы | Примечание |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|------------|
| 1 | Анализ предметной области и формирование требований к программному обеспечению                                                                                                                 | 29.12.2025-28.01.2026  | Выполнено  |
| 2 | Реализация и балансировка игрового процесса в линейном 2D-автораннере: организация процесса разработки, соответствие реализации спроектированной архитектуре, детализация ключевых компонентов | 29.01.2026-11.02.2026  | Выполнено  |
| 3 | Реализация и балансировка игрового процесса в линейном 2D-автораннере: применение принципов качественного кода, интеграция с библиотеками и пакетами Unity                                     | 12.02.2026-18.03.2026  | Выполнено  |
| 4 | Реализация и балансировка игрового процесса в линейном 2D-автораннере: методика игровой балансировки, функциональное тестирование механик, выводы по разделу                                   | 19.03.2026-15.04.2026  | Выполнено  |
| 5 | Расчет технико-экономической эффективности разработки и внедрения 2D-автораннера RusRunner                                                                                                     | 16.04.2026-13.05.2026  | Выполнено  |
| 6 | Безопасность и экологичность разработки                                                                                                                                                        | 14.05.2026-02.06.2026  | Выполнено  |
| 7 | Оформление КВКР и подготовка к защите                                                                                                                                                          | 03.06.2026-09.06.2026  | Выполнено  |
| 8 | Представление КВКР в ГЭК                                                                                                                                                                       | 10.06.2026             | Выполнено  |

Руководитель координатор Н.Ю. Братченко  
(подпись)

Руководитель раздела Н.Ю. Братченко  
(подпись)

И. о. директора департамента И. В. Азаров  
(подпись)

«26» июня 2026 г.

## СОДЕРЖАНИЕ

|                                                                                                       |    |
|-------------------------------------------------------------------------------------------------------|----|
| ВВЕДЕНИЕ .....                                                                                        | 11 |
| 1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ФОРМИРОВАНИЕ<br>ТРЕБОВАНИЙ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ .....             | 15 |
| 1.1 Характеристика предметной области.....                                                            | 15 |
| 1.1.1 Общая характеристика и основные понятия предметной области .....                                | 15 |
| 1.1.2 Анализ типовых сценариев деятельности целевой аудитории.....                                    | 17 |
| 1.1.3 Описание технологической и платформенной базы, регулирующей<br>разработку в данной области..... | 19 |
| 1.1.4 Выводы по результатам анализа предметной области.....                                           | 20 |
| 1.2 Исследование существующих решений.....                                                            | 20 |
| 1.2.1 Критерии сравнения программных продуктов .....                                                  | 20 |
| 1.2.2 Обзор и анализ аналога №1 Subway Surfers .....                                                  | 21 |
| 1.2.3 Обзор и анализ аналога №2 Jetpack Joyride.....                                                  | 21 |
| 1.2.4 Обзор и анализ аналога №3 Alto's Adventure.....                                                 | 22 |
| 1.2.5 Обзор и анализ аналога №4 Vector.....                                                           | 22 |
| 1.2.6 Обзор и анализ аналога №5 Geometry Dash .....                                                   | 23 |
| 1.2.7 Сравнительная характеристика и определение недостатков<br>существующих решений.....             | 23 |
| 1.2.8 Обоснование актуальности и конкурентных преимуществ собственной<br>разработки .....             | 24 |
| 1.3 Выявление проблем и постановка задачи.....                                                        | 24 |
| 1.3.1 Формулировка проблемной ситуации.....                                                           | 24 |
| 1.3.2 Определение цели и задач создания программного продукта.....                                    | 25 |
| 1.3.3 Описание концепции будущего решения .....                                                       | 26 |

|           |                |             |       |      |                                                                                                                                   |                                                                 |      |        |
|-----------|----------------|-------------|-------|------|-----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|------|--------|
|           |                |             |       |      | КВКР-СКФУ-09.03.04-220970-26                                                                                                      |                                                                 |      |        |
|           |                |             |       |      | КВКР-СКФУ-09.03.04-220967-26                                                                                                      |                                                                 |      |        |
| Изм.      | Лист           | № документа | Подп. | Дата |                                                                                                                                   |                                                                 |      |        |
| Разраб.   | Магомедов И.А. |             |       |      | Проектирование, оптимизация и<br>реализация программного комплекса<br>казуальной 2D-игры на основе жанра<br>линейного автораннера | Лит.                                                            | Лист | Листов |
| Разраб.   | Коровин Н.А.   |             |       |      |                                                                                                                                   |                                                                 | 7    | 127    |
| Проверил  | Братченко Н.Ю. |             |       |      |                                                                                                                                   | СКФУ. Институт ПИ,<br>Департамент ЦРСиЭ,<br>Группа ПИЖ-6-о-22-1 |      |        |
| Н. контр. | Новикова Е.Н.  |             |       |      |                                                                                                                                   |                                                                 |      |        |
| Утвердил  | Азаров И.В.    |             |       |      |                                                                                                                                   |                                                                 |      |        |

|       |                                                                                |    |
|-------|--------------------------------------------------------------------------------|----|
| 1.3.4 | Определение границ проекта и ограничений .....                                 | 26 |
| 1.4   | Спецификация требований.....                                                   | 27 |
| 1.4.1 | Детальное описание функциональных требований.....                              | 27 |
| 1.4.2 | Формулировка нефункциональных требований .....                                 | 28 |
| 1.5   | Выводы по разделу.....                                                         | 29 |
| 2     | ПРОЕКТИРОВАНИЕ И ОПТИМИЗАЦИЯ МОДУЛЬНОГО ЯДРА<br>ДЛЯ 2D-AВТОРАННЕРА.....        | 30 |
| 2.1   | Назначение модульного ядра в программном комплексе<br>2D-автораннера .....     | 30 |
| 2.2   | Выбор и обоснование архитектурного стиля .....                                 | 34 |
| 2.3   | Моделирование архитектуры с использованием подхода C4.....                     | 38 |
| 2.3.1 | Диаграмма контекста системы .....                                              | 39 |
| 2.3.2 | Диаграмма контейнеров .....                                                    | 40 |
| 2.3.3 | Диаграмма компонентов игрового ядра.....                                       | 42 |
| 2.4   | Проектирование пользовательского интерфейса.....                               | 44 |
| 2.4.1 | Разработка прототипов экранных форм .....                                      | 45 |
| 2.4.2 | Описание навигации и пользовательских сценариев .....                          | 47 |
| 2.5   | Детальное проектирование ключевых классов ядра.....                            | 49 |
| 2.5.1 | Диаграммы классов ключевых компонентов .....                                   | 50 |
| 2.5.2 | Диаграммы последовательности для основных сценариев .....                      | 53 |
| 2.5.3 | Применение шаблонов проектирования .....                                       | 57 |
| 2.6   | Оптимизация модульного ядра и генерации объектов.....                          | 59 |
| 2.7   | Обоснование выбора стека технологий и инструментов разработки .....            | 63 |
| 2.8   | Практическая проверка проектных решений .....                                  | 66 |
| 2.9   | Выводы по разделу.....                                                         | 68 |
| 3     | РЕАЛИЗАЦИЯ И БАЛАНСИРОВКА ИГРОВОГО ПРОЦЕССА В<br>ЛИНЕЙНОМ 2D-AВТОРАННЕРЕ ..... | 70 |
| 3.1   | Организация процесса разработки .....                                          | 70 |

|      |      |          |       |      |                                                              |      |
|------|------|----------|-------|------|--------------------------------------------------------------|------|
|      |      |          |       |      | КВКР-СКФУ-09.03.04-220970-26<br>КВКР-СКФУ-09.03.04-220967-26 | Лист |
| Изм. | Лист | № докум. | Подп. | Дата |                                                              | 8    |
|      |      |          |       |      |                                                              |      |

|       |                                                                                                     |    |
|-------|-----------------------------------------------------------------------------------------------------|----|
| 3.1.1 | Выбор и обоснование методологии разработки .....                                                    | 70 |
| 3.1.2 | Организация репозитория и системы контроля версий .....                                             | 71 |
| 3.2   | Соответствие реализации спроектированной архитектуре.....                                           | 72 |
| 3.2.1 | Сопоставление компонентов архитектуры и модулей реализации.....                                     | 72 |
| 3.2.2 | Организация файловой структуры проекта.....                                                         | 73 |
| 3.2.3 | Конфигурационные файлы и управление настройками Unity.....                                          | 75 |
| 3.3   | Детализация ключевых компонентов.....                                                               | 75 |
| 3.3.1 | Реализация управления персонажем.....                                                               | 75 |
| 3.3.2 | Реализация системы препятствий.....                                                                 | 78 |
| 3.3.3 | Реализация бонусов и временных эффектов .....                                                       | 80 |
| 3.3.4 | Реализация системы очков, дистанции и рекорд .....                                                  | 82 |
| 3.3.5 | Реализация пользовательского интерфейса игрового процесса.....                                      | 83 |
| 3.4   | Применение принципов качественного кода .....                                                       | 84 |
| 3.4.1 | Принципы SOLID в реализации .....                                                                   | 84 |
| 3.4.2 | Принципы чистого кода .....                                                                         | 85 |
| 3.5   | Интеграция с библиотеками и пакетами Unity.....                                                     | 86 |
| 3.6   | Методика игровой балансировки .....                                                                 | 87 |
| 3.6.1 | Балансировка скорости и кривой сложности .....                                                      | 87 |
| 3.6.2 | Балансировка прыжка, дополнительного прыжка и подката .....                                         | 89 |
| 3.7   | Функциональное тестирование механик .....                                                           | 91 |
| 3.8   | Выводы по разделу.....                                                                              | 92 |
| 4     | РАСЧЕТ ТЕХНИКО-ЭКОНОМИЧЕСКОЙ ЭФФЕКТИВНОСТИ<br>РАЗРАБОТКИ И ВНЕДРЕНИЯ 2D-АВТОРАННЕРА RUSRUNNER ..... | 94 |
| 4.1   | Обоснование необходимости расчета эффективности.....                                                | 94 |
| 4.2   | Описание объекта оценки.....                                                                        | 95 |
| 4.3   | Исходные данные для расчетов .....                                                                  | 96 |
| 4.4   | Методика расчета эффективности.....                                                                 | 97 |

|      |      |          |       |      |                                                              |      |
|------|------|----------|-------|------|--------------------------------------------------------------|------|
|      |      |          |       |      | КВКР-СКФУ-09.03.04-220970-26<br>КВКР-СКФУ-09.03.04-220967-26 | Лист |
|      |      |          |       |      |                                                              |      |
| Изм. | Лист | № докум. | Подп. | Дата |                                                              | 9    |

|       |                                                             |     |
|-------|-------------------------------------------------------------|-----|
| 4.5   | Затраты на разработку и внедрение .....                     | 98  |
| 4.6   | Расходы на эксплуатацию после внедрения .....               | 100 |
| 4.7   | Экономия ресурсов от внедрения .....                        | 101 |
| 4.8   | Расчет финансовых показателей.....                          | 102 |
| 4.9   | Анализ чувствительности, рисков и пути их минимизации ..... | 103 |
| 4.10  | Выводы по разделу .....                                     | 104 |
| 5     | БЕЗОПАСНОСТЬ И ЭКОЛОГИЧНОСТЬ РАЗРАБОТКИ .....               | 105 |
| 5.1   | Анализ основных вредных и опасных факторов.....             | 105 |
| 5.2   | Меры обеспечения безопасности.....                          | 106 |
| 5.2.1 | Организация рабочего места и режима труда .....             | 106 |
| 5.2.2 | Электробезопасность .....                                   | 107 |
| 5.2.3 | Пожарная безопасность .....                                 | 107 |
| 5.2.4 | Информационная безопасность игрового приложения.....        | 107 |
| 5.3   | Экологичность разработки.....                               | 108 |
| 5.4   | Выводы по разделу.....                                      | 109 |
|       | ЗАКЛЮЧЕНИЕ.....                                             | 110 |
|       | СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ .....                        | 114 |
|       | ПРИЛОЖЕНИЕ А.....                                           | 117 |
|       | ПРИЛОЖЕНИЕ Б .....                                          | 127 |

|      |      |          |       |      |                                                              |      |
|------|------|----------|-------|------|--------------------------------------------------------------|------|
|      |      |          |       |      | КВКР-СКФУ-09.03.04-220970-26<br>КВКР-СКФУ-09.03.04-220967-26 | Лист |
|      |      |          |       |      |                                                              |      |
| Изм. | Лист | № докум. | Подп. | Дата |                                                              | 10   |

## ВВЕДЕНИЕ

Разработка компьютерных игр является одним из направлений программной инженерии, в котором качество программного продукта определяется не только корректностью алгоритмов, но и согласованностью архитектуры, производительности, пользовательского управления и игрового баланса. Даже сравнительно небольшой казуальный проект объединяет обработку ввода, физику, столкновения, генерацию объектов, отображение интерфейса, хранение результатов и управление состояниями игровой сессии. Ошибки в любой из перечисленных подсистем непосредственно влияют на восприятие игры: нестабильная частота кадров ухудшает точность реакции, чрезмерно связанные компоненты затрудняют развитие механик, а неправильно выбранные параметры скорости и препятствий делают прохождение либо слишком простым, либо несправедливым.

Особенно наглядно взаимосвязь технических и игровых решений проявляется в жанре линейного 2D-автораннера. Базовые правила такого проекта легко воспринимаются пользователем: движение вперед выполняется автоматически, а основная задача заключается в своевременном выполнении прыжка, подката и других ограниченных действий. При этом кажущаяся простота жанра предъявляет повышенные требования к точности коллизий, времени реакции, последовательности появления препятствий, темпу роста сложности и скорости повторного запуска после поражения. Поэтому создание автораннера представляет собой подходящую практическую задачу для исследования способов построения модульного игрового ядра и управляемой балансировки игрового процесса.

Актуальность выпускной квалификационной работы обусловлена необходимостью разработки российских игровых программных комплексов, которые одновременно обеспечивают понятный пользовательский сценарий, устойчивую работу длительной сессии и возможность дальнейшего расширения без полного переписывания исходного кода. Для небольших Unity-проектов

характерна проблема быстрого усложнения структуры, где используется несколько крупных классов. Такой подход ускоряет создание первоначального прототипа, но затрудняет тестирование, оптимизацию и изменение балансовых параметров. Следовательно, практическое значение имеет разделение игры на самостоятельные компоненты, объединенные единым состоянием и координирующим слоем.

Объектом исследования является процесс проектирования и разработки программного комплекса казуальной 2D-игры жанра линейного автораннера в среде Unity.

Предмет исследования составляют архитектурные, алгоритмические и программные решения, обеспечивающие модульность игрового ядра, повторное использование объектов, реализацию базового игрового цикла, управление персонажем, генерацию препятствий и бонусов, рост сложности, подсчет результата и устойчивый перезапуск игровой сессии.

Целью выпускной квалификационной работы является проектирование, оптимизация и реализация программного комплекса казуальной 2D-игры на основе жанра линейного автораннера, обладающего модульной архитектурой, стабильным игровым циклом, настраиваемой кривой сложности и достаточной производительностью для повторяющихся сессий.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать предметную область казуальных 2D-игр и особенности жанра линейного автораннера;
- исследовать существующие игровые решения и определить применимые принципы организации управления, визуальной читаемости, прогрессии и рестарта;
- сформулировать функциональные и нефункциональные требования к программному комплексу;
- спроектировать компонентно-модульную архитектуру и определить состав ключевых компонентов игрового ядра;

- разработать единое состояние игровой сессии, системы прогрессии, счета, временных эффектов, ввода и пользовательского интерфейса;
- реализовать управление персонажем через прыжок, дополнительный прыжок и подкат;
- реализовать обработку столкновений, генерацию трассы, препятствий и бонусов;
- оптимизировать работу повторяющихся объектов игрового мира с применением объектных пулов и прокрутки мира относительно фиксированной позиции персонажа;
- выполнить расчетную и экспериментальную балансировку скорости, кривой сложности и временных окон основных манёвров;
- провести функциональное тестирование ключевых игровых сценариев;
- оценить технико-экономическую эффективность проекта;
- оценить безопасность и экологичность разработки.

Для решения поставленных задач применялись методы системного и сравнительного анализа, объектно-ориентированного проектирования, декомпозиции программного комплекса на компоненты, моделирования архитектуры с использованием C4 и UML, итерационно-инкрементной разработки, расчетного анализа параметров движения, функционального тестирования и технико-экономической оценки. При проектировании учитывались принципы SOLID и шаблоны Facade, Strategy, Object Pool и объект временного эффекта. Практическая реализация выполнена на языке C# в Unity с использованием 2D-физики.

В качестве исходного программного результата разработана игра RusRunner. В ней персонаж сохраняет ограниченное положение по горизонтальной оси, а иллюзия движения создается смещением сегментов трассы, препятствий, бонусов и фоновых слоев. Игрок выполняет наземный и дополнительный прыжки, использует подкат, преодолевает препятствия различных типов, собирает временные усиления, увеличивает дистанцию и счёт.

Система прогрессии постепенно повышает скорость и переключает профили генерации, а после поражения фиксируется лучший результат и обеспечивается быстрый запуск новой попытки.

Научно-техническая новизна работы в рамках учебного проекта заключается в комплексном рассмотрении архитектуры и балансировки как взаимозависимых частей одного программного продукта. Параметры игрового процесса не изолированы в отдельных несвязанных скриптах, а изменяют единый объект состояния и обрабатываются специализированными системами. Кривая сложности рассматривается не только как дизайнерское решение, но и как алгоритмическая зависимость, влияющая на скорость прокрутки, выбор конфигурации сегментов и доступное игроку время реакции.

Практическая значимость работы состоит в возможности использовать созданный программный комплекс как основу для дальнейшего развития 2D-автораннера и продвижения российских игр. Модульное разделение позволяет добавлять новые виды препятствий, бонусов, способов ввода и визуальных тем без перестройки всего игрового цикла.

Достоверность полученных результатов обеспечивается сопоставлением проектных моделей с фактической структурой Unity-проекта, анализом исходного кода ключевых компонентов, расчётом параметров прогрессии, а также функциональной проверкой основных сценариев.

Пояснительная записка включает пять глав, в которых последовательно рассмотрены анализ предметной области, проектирование ядра, реализация и балансировка игрового процесса, технико-экономическая оценка, безопасность и экологичность разработки.

# **1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ФОРМИРОВАНИЕ ТРЕБОВАНИЙ К ПРОГРАММНОМУ ОБЕСПЕЧЕНИЮ**

## **1.1 Характеристика предметной области**

### **1.1.1 Общая характеристика и основные понятия предметной области**

Настоящая выпускная квалификационная работа выполнена в области разработки компьютерных игр, а именно в сегменте казуальных 2D-проектов, ориентированных на короткие игровые сессии, быстрое освоение правил и повторяемость игрового цикла. В качестве общей темы комплексного диплома рассматривается проектирование, оптимизация и реализация программного комплекса казуальной 2D-игры на основе жанра линейного автораннера. Внутри общей темы выделяются два взаимосвязанных направления: проектирование и оптимизация модульного ядра для 2D-автораннера, а также реализация и балансировка игрового процесса в линейном 2D-автораннере.

Жанр автораннера представляет собой разновидность аркадной игры, в которой базовое движение персонажа вперёд выполняется автоматически, а основная активность игрока сосредоточена на своевременном выполнении ограниченного набора действий: прыжка, подката, использования усиления либо выбора момента для взаимодействия с элементами уровня. Подобная структура даёт низкий порог входа, но одновременно предъявляет высокие требования к точности столкновений, темпу нарастания сложности и качеству обратной связи. Ошибка игрока должна восприниматься как следствие его действия или бездействия, а не как результат неясной коллизии, неудачной камеры или перегруженного интерфейса.

Для разработки 2D-проектов в Unity доступны специализированные средства работы со спрайтами, Tilemap, анимацией и системой 2D-физики. Это делает движок подходящей средой для реализации линейного автораннера, где важны не только визуальные элементы, но и управляемость, воспроизводимость поведения объектов и удобство последующей настройки игровых параметров [18].

Казуальный автораннер как программный продукт находится на пересечении нескольких требований. С одной стороны, он должен оставаться простым и понятным: игроку необходима мгновенная реакция на выполнение команды, читаемое окружение, короткий цикл «запуск - попытка - поражение - рестарт». С другой стороны, для удержания интереса недостаточно одной механики бега. Нужны система очков, сбор коллекционных элементов, постепенное увеличение скорости, изменение плотности препятствий, вариативность ситуаций и ощущение прогресса даже при коротких игровых сессиях. Поэтому в подобном проекте архитектурные решения и баланс геймплея неразделимы: жёстко зашитая логика мешает балансировке, а слабо продуманный баланс нивелирует достоинства даже хорошо организованного кода.

В рамках данной работы под линейным 2D-автораннером понимается игра с боковой проекцией, в которой игрок наблюдает движение персонажа по горизонтально ориентированной трассе, реагирует на препятствия и собираемые объекты, а игровая сессия ограничивается моментом столкновения либо иным условием поражения. Под линейностью в данном случае понимается не полное отсутствие вариативности, а отсутствие сложной навигации по ветвящимся уровням: основное движение направлено вперёд, а внимание сосредоточено на ритме препятствий, точности действий и нарастании сложности.

Ключевыми понятиями предметной области являются:

- модульное ядро игры – совокупность базовых программных компонентов, обеспечивающих управление состояниями игры, персонажем, генерацией препятствий, подсчётом очков, сохранением данных и взаимодействием пользовательского интерфейса с игровой логикой;
- игровой цикл – повторяющаяся последовательность действий, включающая запуск забега, прохождение дистанции, сбор наград, столкновение или завершение попытки, анализ результата и последующий рестарт;
- балансировка – процесс подбора и корректировки параметров движения, частоты препятствий, расстояний между объектами, стоимости

ошибок и наград таким образом, чтобы игра оставалась понятной, но не теряла сложности и интереса;

- объектный пул – способ повторного использования ранее созданных экземпляров игровых объектов вместо постоянного вызова `Instantiate` и `Destroy` [24];

- параллакс – визуальный приём, при котором фоновые слои движутся с различной скоростью, создавая ощущение глубины сцены;

- хитбокс – область столкновения игрового объекта, влияющая на распознавание попаданий и взаимодействий;

- система сложности – набор правил, по которым изменяются скорость, плотность объектов, частота появления опасных ситуаций и иные параметры игрового процесса;

- сессия игрока – отдельная попытка прохождения, в пределах которой фиксируются очки, дистанция, собранные предметы и достигнутые игровые события.

Таким образом, предметная область настоящей работы охватывает не только визуальную реализацию 2D-игры, но и архитектурные, алгоритмические и проектные аспекты. Для качественного результата недостаточно собрать сцену из спрайтов и добавить прыжок. Необходимо обеспечить согласованность между физикой, управлением, спавном препятствий, ростом сложности, интерфейсом и сохранением результатов.

### **1.1.2 Анализ типовых сценариев деятельности целевой аудитории**

Целевой аудиторией разрабатываемого продукта являются пользователи, предпочитающие короткие игровые сессии, простое управление и быстрое включение в процесс. Для казуальных игр это принципиально: игрок не должен тратить значительное время на изучение правил, чтение длинных инструкций или освоение большого числа механик. Оптимальный сценарий входа в игру предполагает, что уже в первые секунды пользователь понимает цель текущей

попытки: двигаться вперёд, избегать препятствий, собирать награды и стремиться улучшить предыдущий результат.

Первый типовой сценарий – «быстрый запуск и короткая сессия». Игрок открывает приложение, запускает забег и почти сразу начинает взаимодействовать с игровым миром. Он выполняет несколько прыжков, преодолевает препятствия, собирает монеты либо очки и завершает сессию после ошибки. Важнейшей характеристикой такого сценария является скорость возврата к новой попытке. Если между поражением и повторным стартом возникает длинная задержка, перегруженное меню или избыточное количество подтверждений, игра теряет свой основной ритм.

Второй сценарий – «серия повторных попыток ради улучшения результата». Для автораннера это один из базовых паттернов поведения. Игрок стремится не просто завершить одну сессию, а улучшить личный рекорд, пройти чуть дальше, точнее выполнить прыжок, собрать больше предметов или протестировать другой стиль реакции на препятствия. Следовательно, система очков, отображение дистанции и фиксация рекордов должны быть предельно прозрачными и мгновенно считываемыми. Пользователь должен видеть, ради чего он продолжает играть.

Третий сценарий связан с постепенным освоением ритма сложности. В качественном автораннере первые секунды не должны перегружать игрока. Затем темп возрастает: увеличивается скорость, сокращаются безопасные интервалы, появляются комбинации препятствий. Если игра сразу слишком сложна, пользователь уходит; если же усложнение не чувствуется, интерес также быстро исчезает. Отсюда следует необходимость продуманной кривой сложности, в которой ранняя стадия выполняет обучающую функцию, а дальнейшая – проверяет устойчивость навыка.

Четвёртый сценарий относится уже не к игроку, а к разработчику и тестировщику. Для них важны возможность быстро менять параметры, проверять влияние скорости на длину прыжка, регулировать интервалы спавна, отключать или включать определённые типы препятствий, наблюдать поведение

системы при рестарте и фиксировать ошибки взаимодействия UI с игровыми состояниями. Если архитектура игры не позволяет изолированно настраивать отдельные элементы, балансировка превращается в трудоёмкий ручной процесс.

Таким образом, даже у компактной казуальной игры существует несколько разных сценариев использования: непосредственное прохождение, повторное достижение рекорда, адаптация к растущей сложности и внутренняя настройка игры на этапе разработки. Все они должны учитываться уже на этапе проектирования.

### **1.1.3 Описание технологической и платформенной базы, регулирующей разработку в данной области**

В отличие от отраслей с жёстко формализованной нормативной базой, разработка казуальных игр регулируется прежде всего технологическими и платформенными ограничениями. Для 2D-автораннера ключевыми являются требования к плавности отображения, отзывчивости управления, стабильности работы приложения, масштабируемости интерфейса под разные разрешения экрана и предсказуемости поведения игровой логики на устройствах с различной производительностью.

С платформенной точки зрения наиболее значимым является требование к плавности кадра. Для игр жанра автораннер поздний отклик интерфейса или выпадение кадров воспринимаются не как косметический недостаток, а как непосредственная причина ошибки игрока. Следовательно, требования к производительности должны входить в число базовых нефункциональных требований проекта наряду с требованиями к сохранности пользовательских данных и корректности восстановления состояния после перезапуска.

С практической точки зрения технологическая база проекта включает Unity как игровой движок, язык C# как средство реализации логики, встроенную систему физики 2D, анимационные компоненты, Canvas для интерфейса, а также локальные механизмы сохранения данных. При необходимости дальнейшего развития проекта такая основа позволяет расширять игру без полного

переписывания кода: добавлять новые препятствия, альтернативные сцены, таблицы параметров, визуальные темы и дополнительные режимы.

#### **1.1.4 Выводы по результатам анализа предметной области**

Предметная область линейных 2D-автораннеров характеризуется сочетанием кажущейся простоты и высокой технической чувствительности к деталям реализации. Пользователь видит компактную аркадную игру с одной базовой целью, однако за этой простотой находятся задачи синхронизации движения, физики, коллизий, генерации препятствий, роста сложности, подсчёта результатов и мгновенного перезапуска игровой сессии.

Для рассматриваемой темы особенно важно, что архитектура и баланс здесь не существуют по отдельности. Модульное ядро необходимо не только для «хорошего кода», но и для того, чтобы геймплей можно было быстро и безопасно настраивать. В свою очередь, балансировка влияет на требования к архитектуре: если параметры нельзя гибко менять, невозможно добиться устойчивого игрового ритма.

Проведённый анализ показывает, что Unity предоставляет подходящий инструментарий для реализации подобных проектов, а производительность, организация ресурсов и повторное использование объектов должны закладываться на уровне архитектурных решений, а не оставаться на финальную стадию разработки.

### **1.2 Исследование существующих решений**

#### **1.2.1 Критерии сравнения программных продуктов**

Для сопоставления существующих игр жанра runner и смежных аркадных решений были выбраны следующие критерии. Во-первых, тип базовой механики: боковой автораннер, lane-based runner, ритм-платформер или смешанная модель. Во-вторых, способ управления: один ввод, несколько жестов, переключение по полосам, физически обусловленное движение. В-третьих,

структура прогрессии: бесконечный забег, серия миссий, наличие улучшений и персонажей. В-четвёртых, характер роста сложности. В-пятых, визуальная читаемость сцены. В-шестых, предполагаемая пригодность отдельных решений для адаптации в учебный проект на Unity с упором на модульность и оптимизацию.

### **1.2.2 Обзор и анализ аналога №1 Subway Surfers**

Subway Surfers является одним из самых узнаваемых представителей жанра endless runner. Сильной стороной решения выступает мгновенная понятность правил и высокая визуальная читаемость угроз: игрок почти всегда видит, что именно стало причиной поражения. Кроме того, проект демонстрирует устойчивость жанровой формулы на длинной дистанции [17].

Однако для целей настоящей работы Subway Surfers выступает скорее концептуальным, чем прямым архитектурным аналогом. Игра построена по трёхполосной 3D-модели, где значительная часть напряжения создаётся за счёт смены дорожек и перспективы. Для линейного 2D-автораннера с боковой камерой эта схема не переносится напрямую. Тем не менее из данного аналога заимствуется важный принцип: игрок должен за доли секунды различать опасность, награду и безопасную траекторию.

### **1.2.3 Обзор и анализ аналога №2 Jetpack Joyride**

Jetpack Joyride представляет собой боковой endless runner, построенный на простой схеме управления: удержание приводит к набору высоты, отпускание - к снижению. Для темы настоящей работы этот аналог особенно важен, поскольку демонстрирует эффективность минимального набора действий при высокой вариативности игровых ситуаций [13].

К достоинствам Jetpack Joyride относятся лаконичное управление, высокий темп, короткие попытки и хорошая повторяемость игрового цикла. Недостатком с точки зрения учебного 2D-автораннера можно считать то, что значительная

часть выразительности строится не на прыжковой физике наземного персонажа, а на вертикальном управлении полётом. Следовательно, данный аналог полезен главным образом как пример компактного core loop и системы быстрых рестартов.

#### **1.2.4 Обзор и анализ аналога №3 Alto's Adventure**

Alto's Adventure демонстрирует более мягкую и атмосферную интерпретацию жанра: сложность растёт постепенно, а визуальная часть работает не только на информативность, но и на эмоциональное восприятие [16].

Сильными сторонами Alto's Adventure являются цельность художественного стиля, аккуратная кривая сложности и сочетание простого ввода с ощущением физической непрерывности движения. Для собственной разработки важен вывод о том, что даже простой автораннер не обязан быть исключительно агрессивным и перегруженным. Однако прямое копирование такого подхода нецелесообразно: в учебном проекте приоритетом остаются модульность, прозрачность архитектуры и управляемая балансировка, а не сложная атмосферная постановка.

#### **1.2.5 Обзор и анализ аналога №4 Vector**

Vector интересен как пример того, как анимационное качество, форма препятствий и кинематографичность движений могут усиливать ощущение скорости и риска [14].

Для настоящей работы Vector важен прежде всего как ориентир по выразительности движения. Если персонаж в 2D-автораннере перемещается технически корректно, но визуально «оторван» от поверхности, игра теряет убедительность. Из этого аналога следует практический вывод: ground check, высота прыжка, положение спрайта относительно земли и анимационная фаза приземления влияют на качество восприятия не меньше, чем общая логика забега.

### **1.2.6 Обзор и анализ аналога №5 Geometry Dash**

Geometry Dash, несмотря на принадлежность скорее к ритм-платформерам, близка теме исследования по нескольким причинам: автоматическое продвижение вперёд, жёсткая роль тайминга, быстрый рестарт после поражения и необходимость предельно честных коллизий [15].

Преимуществом Geometry Dash является образцовая ясность причинно-следственной связи: игрок почти всегда понимает, где именно допустил ошибку. Для собственной разработки это особенно ценно. Недостатком как прямого аналога является иная природа контента: игра построена на жёстко спроектированных последовательностях, а не на более свободном потоке препятствий с постепенным изменением параметров. Тем не менее логика мгновенного рестарта и строгой читаемости столкновений является значимым ориентиром.

### **1.2.7 Сравнительная характеристика и определение недостатков существующих решений**

Рассмотренные аналоги демонстрируют, что жанр runner обладает большим диапазоном интерпретаций. Subway Surfers показывает силу понятной аркадной схемы и быстрой визуальной читаемости. Jetpack Joyride доказывает, что один базовый способ управления способен удерживать интерес за счёт событийной насыщенности. Alto's Adventure демонстрирует ценность плавной кривой сложности и художественного стиля. Vector усиливает внимание к достоверности анимированных движений. Geometry Dash подчёркивает важность коллизий и мгновенного возврата к новой попытке.

Одновременно все эти решения имеют ограничения для целей дипломного проекта. Часть из них относится к иной перспективе или другой поджанровой модели, часть опирается на более сложную художественную постановку, не являющуюся центральной задачей учебной разработки, а часть не раскрывает архитектурные принципы, необходимые для проектирования собственного

модульного ядра. Следовательно, целесообразно не копировать конкретный коммерческий продукт, а выделить из каждого решения те принципы, которые полезны для собственной архитектуры и балансировки.

### **1.2.8 Обоснование актуальности и конкурентных преимуществ собственной разработки**

Актуальность собственной разработки определяется тем, что линейный 2D-автораннер является удобной и показательной моделью для исследования взаимосвязи архитектуры и геймплея. Этот жанр достаточно прост для локального проекта одного или двух разработчиков, но при этом содержит весь набор типичных для игровой индустрии задач: управление состояниями, движение персонажа, физику, коллизии, генерацию препятствий, оптимизацию рендеринга, пользовательский интерфейс, сохранение результатов и балансировку кривой сложности.

Конкурентным преимуществом разрабатываемого решения является именно сочетание двух направлений: модульного ядра и управляемой балансировки. В коммерческих проектах игрок видит уже готовый результат, но внутренняя архитектура обычно скрыта. В данном случае архитектурная прозрачность является самостоятельной ценностью. Система проектируется так, чтобы отдельные подсистемы – управление персонажем, спавн препятствий, подсчёт очков, сложность, UI и сохранения – могли развиваться независимо и тестироваться по отдельности.

## **1.3 Выявление проблем и постановка задачи**

### **1.3.1 Формулировка проблемной ситуации**

На основании анализа предметной области и существующих решений можно выделить следующие проблемы. Во-первых, в играх жанра автораннер даже незначительные ошибки в логике движения, позиционировании спрайта, обработке столкновений или обновлении интерфейса напрямую разрушают

ощущение «интересной» игры. Во-вторых, без модульного деления проекта становится затруднительно менять баланс: любое изменение скорости, дистанции между объектами или высоты прыжка начинает затрагивать сразу несколько фрагментов кода. В-третьих, визуально простые 2D-проекты нередко недооценивают вопросы оптимизации, хотя постоянный спавн препятствий, монет и эффектов быстро создаёт избыточную нагрузку на память и процессор.

### **1.3.2 Определение цели и задач создания программного продукта**

Целью выпускной квалификационной работы является проектирование, оптимизация и реализация программного комплекса казуальной 2D-игры в жанре линейного автораннера, обеспечивающего стабильную работу игрового цикла, удобство дальнейшей настройки и балансировки, а также достаточный уровень производительности на целевых устройствах.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и существующих решений жанра runner;
- сформировать требования к архитектуре и игровому процессу разрабатываемого продукта;
- спроектировать модульное ядро игры, включающее управление состояниями, персонажем, препятствиями, подсчётом результата и интерфейсом;
- реализовать базовый игровой цикл линейного 2D-автораннера;
- разработать механизмы роста сложности, систему очков, поражения и рестарта;
- выполнить оптимизацию ключевых подсистем, включая спавн повторяющихся объектов и организацию графических ресурсов;
- провести настройку и балансировку игрового процесса;

– выполнить тестирование и оценить качество реализованного решения.

### **1.3.3 Описание концепции будущего решения**

Разрабатываемая игра представляет собой одиночный казуальный 2D-проект с боковой камерой, в котором персонаж автоматически движется по уровню, а пользователь управляет действиями, необходимыми для преодоления препятствий и поддержания забега как можно дольше. Базовая логика строится вокруг непрерывного цикла: старт забега, движение, обработка препятствий и наград, фиксация результата, переход в состояние поражения и немедленная готовность к новой попытке.

С архитектурной точки зрения решение предполагает наличие следующих подсистем: менеджера игровых состояний; контроллера персонажа; детектора нахождения на земле; системы спавна препятствий и собираемых объектов; системы управления сложностью; подсистемы очков, дистанции и рекордов; пользовательского интерфейса; звукового сопровождения. При этом все перечисленные элементы должны взаимодействовать через чётко определённые точки обмена, а не через хаотические прямые зависимости.

Игровой процесс строится на последовательном усложнении. Начальная фаза даёт игроку возможность адаптироваться к темпу и механике. Далее возрастает скорость, появляются более плотные комбинации препятствий, увеличивается цена ошибки. Система должна поддерживать возможность быстрой перенастройки этих параметров: отдельно регулировать стартовую скорость, шаг роста сложности, интервалы спавна, вероятность сложных конфигураций и стоимость собираемых предметов.

### **1.3.4 Определение границ проекта и ограничений**

Настоящая ВКР является частью комплексного проекта. В рамках общей темы один блок работ относится к проектированию и оптимизации модульного

ядра 2D-автораннера, второй - к реализации и балансировке игрового процесса. Такое разделение не означает изоляцию задач друг от друга: балансировка невозможна без архитектурной управляемости, а архитектура теряет практическую ценность без реального игрового наполнения.

В границы проекта входят: реализация одиночной 2D-игры на Unity; создание базового управляемого персонажа; запуск линейного забега; система препятствий и наград; рост сложности; экран поражения; рестарт; фиксация рекордов; базовая оптимизация игровых объектов и интерфейса.

За пределами проекта остаются: сетевой режим, многопользовательское взаимодействие, внутриигровой магазин, серверная часть, система аккаунтов, облачные сохранения, аналитика продакшн-уровня и масштабная контентная экосистема. Художественные ресурсы в рамках диплома рассматриваются как функциональная часть прототипа, а не как самостоятельный объект глубокой художественной разработки.

## **1.4 Спецификация требований**

### **1.4.1 Детальное описание функциональных требований**

Система должна обеспечивать запуск игровой сцены и переход в состояние активного забега без длительной промежуточной подготовки. После старта персонаж должен автоматически двигаться вперёд либо создавать эквивалентное ощущение движения за счёт смещения мира относительно игрока.

Игрок должен иметь возможность выполнять минимум одно основное действие – прыжок. При наличии расширенной логики допускаются дополнительные действия, например двойной прыжок или подкат, однако их добавление не должно усложнять базовый цикл настолько, чтобы терялась казуальная направленность проекта.

Система обязана корректно определять нахождение персонажа на поверхности. Повторный прыжок в воздухе может быть разрешён только в том случае, если он предусмотрен дизайном как отдельная механика. Иначе ядро

игры должно предотвращать некорректное срабатывание прыжка вне допустимых условий.

Игра должна создавать препятствия и собираемые объекты по заранее заданным правилам. При этом логика появления объектов не должна приводить к заведомо неразрешимым для игрока ситуациям. Между препятствиями должны существовать интервалы, соответствующие возможностям персонажа с учётом текущей скорости.

Система должна рассчитывать и отображать очки, дистанцию, скорость и рекорд. После поражения игрок должен видеть понятное итоговое окно, содержащее как минимум итоговые показатели и кнопку повторного запуска.

После рестарта игра должна возвращаться в чистое рабочее состояние: счётчик очков сбрасывается, временные эффекты обнуляются, объекты возвращаются в исходное положение или корректно пересоздаются из пула, состояние интерфейса синхронизируется с новой сессией.

#### **1.4.2 Формулировка нефункциональных требований**

К производительности предъявляются следующие требования: игра должна обеспечивать плавную работу без заметных просадок в базовой сцене; повторяющиеся препятствия и собираемые объекты должны обрабатываться с использованием оптимизационных подходов, уменьшающих издержки на постоянное создание и уничтожение экземпляров; на целевом устройстве игра должна стремиться к стабильному отображению порядка 60 кадров в секунду.

К архитектуре предъявляется требование модульности: подсистемы не должны образовывать чрезмерное количество жёстких связей, а изменение балансовых параметров не должно требовать переписывания базовой логики.

К удобству использования предъявляются требования мгновенной понятности ввода, читаемости HUD, различимости текста на светлом и тёмном фоне, корректного масштабирования интерфейса под разные разрешения и минимального числа действий для запуска новой попытки.

К надёжности предъявляются требования корректного восстановления игры после поражения, отсутствия накопления ошибок при многократных рестартах, правильной работы сохранения рекорда и устойчивости игровых состояний.

## **1.5 Выводы по разделу**

В результате выполнения первого раздела была рассмотрена предметная область казуальных 2D-игр жанра линейного автораннера и определены ключевые требования к программному обеспечению разрабатываемого проекта.

Проведённый анализ показал, что автораннер как жанр сочетает простое пользовательское взаимодействие с высокой чувствительностью к качеству реализации. Для успешной разработки недостаточно обеспечить только движение персонажа и появление препятствий. Требуется согласованная работа систем управления, физики, коллизий, роста сложности, интерфейса и сохранения результатов. Отдельно установлено, что Unity предоставляет достаточный набор 2D-инструментов для подобных задач.

Исследование существующих решений показало, что разные представители жанра по-разному решают сходные задачи: Subway Surfers делает ставку на мгновенную читаемость и высокий темп, Jetpack Joyride – на простое управление и насыщенность событий, Alto's Adventure – на мягкую кривую сложности и атмосферу, Vector – на выразительность движения, Geometry Dash – на строгую честность тайминга и коллизий.

На основе анализа были сформулированы цель, задачи, функциональные и нефункциональные требования к программному комплексу. Полученные результаты являются достаточной основой для перехода к этапу проектирования архитектуры, описания модульного ядра и последующей реализации и балансировки игрового процесса.

## **2 ПРОЕКТИРОВАНИЕ И ОПТИМИЗАЦИЯ МОДУЛЬНОГО ЯДРА ДЛЯ 2D-АВТОРАННЕРА**

### **2.1 Назначение модульного ядра в программном комплексе 2D-автораннера**

Модульное ядро в программном комплексе 2D-автораннера выполняет роль центрального слоя, который удерживает вместе основные игровые правила: запуск забега, обработку ввода, движение персонажа, начисление очков, рост сложности, столкновения с препятствиями, подбор бонусов и завершение игровой сессии. На первый взгляд подобный проект может показаться небольшим набором Unity-скриптов, размещенных на объектах сцены. Однако в действительности даже простая игра быстро превращается в связанную систему, где любое неосторожное изменение скорости, коллайдера или генератора трассы начинает влиять на весь игровой цикл. Именно поэтому ядро проектируется не как один крупный скрипт, а как совокупность относительно самостоятельных компонентов.

В анализируемом проекте центральным координирующим классом является RunnerBootstrap. Он не реализует всю игровую логику напрямую, а организует взаимодействие между состоянием GameContext, контроллером персонажа RunnerController, генератором трассы TrackGenerator, системой прогрессии, системой счета и системой временных бонусов. Такой подход важен для сопровождения: когда правила подсчета очков меняются, не требуется переписывать физику прыжка; когда добавляется новый тип бонуса, не нужно изменять логику отображения меню. Разделение выглядит простым, но оно защищает проект от той самой ситуации, когда один файл на тысячу строк отвечает вообще за всё.

Назначение ядра можно описать через несколько устойчивых обязанностей. Первая обязанность связана с хранением и изменением состояния текущего забега. Для этого используется класс GameContext, в котором собраны значения текущей скорости, дистанции, счёта, состояние персонажа, флагов

неуязвимости и подката, множителей скорости и очков, а также дополнительных прыжков. Вторая обязанность заключается в управлении игровым временем: на каждом кадре ядро получает `Time.deltaTime`, передает его системам и тем самым обеспечивает плавное развитие процесса независимо от частоты кадров.

Третья обязанность ядра – согласование игрового мира и игрока. В классическом платформере персонаж перемещается по уровню вправо, но в данном автораннере выбран иной прием: персонаж фиксируется по оси X, а сгенерированные сегменты земли, препятствия и бонусы смещаются влево. Для игрока это выглядит как движение вперед. Для проектирования это, между прочим, удобнее: камера остается стабильной, персонаж не уезжает далеко от исходной точки, а генератору проще принимать решение о том, какие объекты уже ушли за экран и могут быть возвращены в пул.

Четвертая обязанность связана с расширяемостью. Интерфейсы `IAbility` и `IPowerUp`, а также конкретные реализации бонусов: `ShieldPowerUp`, `ScoreBoostPowerUp` и `SpeedBoostPowerUp`. Примечательно, что ядро не хранит знания о визуальных спрайтах бонусов; оно работает с их смыслом. Визуальная часть, напротив, находится в слоях `Presentation` и `UI`.

Пятая обязанность ядра – обеспечение предсказуемой работы игры после перезапуска и возврата в меню. Для этого в `RunnerBootstrap` используются методы `BeginRun`, `RestartRun` и `ReturnToMainMenu`, а также статический флаг `_startImmediatelyAfterReload`. Решение не самое торжественное, но вполне рабочее для учебного и прототипного проекта: после перезагрузки сцены можно сразу начать новый забег, не заставляя игрока снова нажимать кнопку запуска.

Таким образом, модульное ядро в данном проекте является не просто технической прослойкой между Unity и игроком. Через него проходит главный сценарий: пользователь запускает игру, персонаж начинает забег, мир прокручивается, сложность постепенно увеличивается, очки растут, бонусы временно изменяют состояние, а столкновение либо игнорируется за счет щита, либо завершает сессию. Разве можно держать такой сценарий в случайном

наборе обработчиков? Можно, но тогда проект очень быстро станет трудным для доработки.

Особенность выбранного решения состоит еще и в том, что большая часть логики остается обычным C#-кодом. Системы ScoreSystem, ProgressionSystem, PowerUpSystem и AbilitySystem не наследуются от MonoBehaviour, поэтому их проще понимать, тестировать и переносить. Они не завязаны напрямую на инспектор Unity, сцену, Canvas или конкретные префабы.

В результате ядро можно рассматривать как внутренний игровой движок конкретного проекта. Оно не заменяет Unity, конечно. Unity отвечает за сцену, кадры, физику, рендеринг, ресурсы и пользовательский ввод. Но поверх этих возможностей формируется собственный слой правил, который делает набор ассетов и компонентов именно автораннером. Без этого слоя на сцене были бы просто спрайты, коллайдеры и кнопки.

Основные элементы модульного и их назначение описаны в таблице 2.1. На рисунке 2.1 изображена схема взаимодействия модульного ядра с другими компонентами проекта.

Таблица 2.1 – Основные элементы модульного ядра проекта RusRunner

| Элемент ядра     | Файл проекта                       | Назначение                                                                                                           |
|------------------|------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| GameContext      | Assets/Scripts/Core/GameContext.cs | Хранение изменяемого состояния текущего забега: скорость, дистанция, очки, флаги эффектов и состояние персонажа.     |
| RunnerBootstrap  | Assets/RunnerBootstrap.cs          | Координация жизненного цикла забега, создание систем, обработка ввода, связь UI, генератора и контроллера персонажа. |
| RunnerController | Assets/RunnerController.cs         | Физика персонажа: прыжок, подкат, фиксация по оси X, работа с Rigidbody2D и CapsuleCollider2D.                       |
| TrackGenerator   | Assets/TrackGenerator.cs           | Процедурное создание сегментов трассы, препятствий и бонусов с применением пулов объектов.                           |

| Элемент ядра                     |   | Файл проекта                | Назначение                                                                  |
|----------------------------------|---|-----------------------------|-----------------------------------------------------------------------------|
| ScoreSystem<br>ProgressionSystem | и | Assets/Scripts/Core/Systems | Начисление очков, сохранение рекорда, рост скорости и сложности во времени. |
| PowerUpSystem<br>AbilitySystem   | и | Assets/Scripts/Core/Systems | Регистрация и выполнение временных эффектов и активируемых способностей.    |

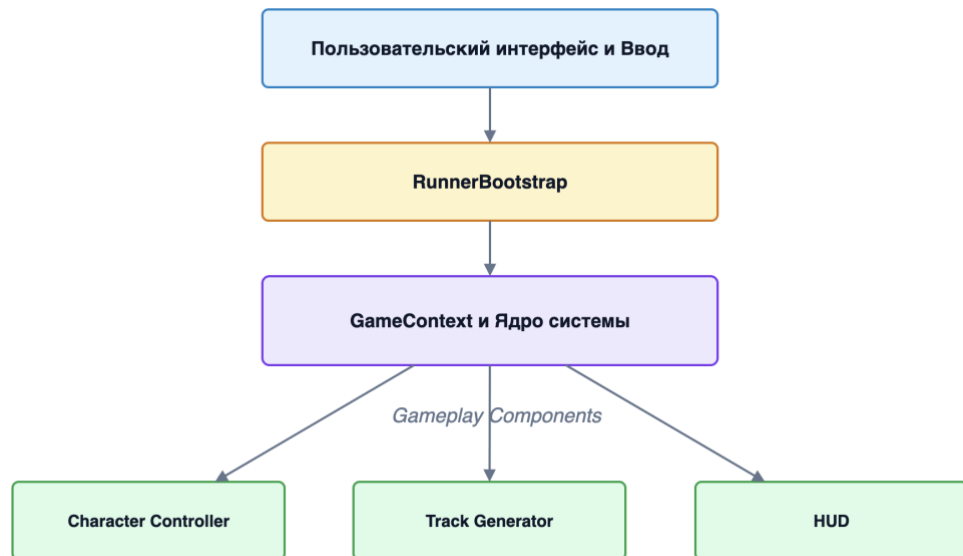


Рисунок 2.1 – Место модульного ядра в общей структуре программного комплекса

С точки зрения эксплуатации ядро должно обеспечивать устойчивость игрового цикла. При отсутствии ссылки на TrackGenerator или RunnerController проект не должен падать сразу при старте. Поэтому в RunnerBootstrap предусмотрен метод AutoFindReferences. Он пытается найти генератор трассы, объект игрока по тегу Runner, а затем получить RunnerController с найденного трансформатора. Это не отменяет необходимости аккуратной настройки сцены, но снижает хрупкость прототипа.

Отдельно нужно отметить связь ядра с интерфейсом. HudController не хранит собственную модель игры, а читает значения из RunnerBootstrap: CurrentScore, BestScore, CurrentDistance, CurrentSpeed, IsAlive, IsSliding, IsInvulnerable, HasScoreBoost, ExtraJumpsLeft. Получается, что интерфейс

отображает состояние, но не становится его владельцем. Это уменьшает вероятность рассинхронизации, когда на экране показано одно, а внутри игры происходит другое. Для автораннера, где игрок принимает решения быстро, такая согласованность особенно важна.

Назначение ядра также проявляется в обработке проигрыша. В методе `HandleObstacleHit` сначала проверяется, начат ли забег и жив ли персонаж. Затем учитывается неуязвимость: если активен щит, препятствие деактивируется, а сессия продолжается. Потом проверяется специальный случай препятствия для подката: если тип препятствия `Slide` и игрок действительно находится в подкате, столкновение не приводит к поражению. Только после этих проверок контекст переводится в состояние `IsAlive = false`, персонаж останавливается, генератор трассы получает нулевую скорость, а счет фиксируется как возможный рекорд.

На уровне пользовательского опыта это выглядит естественно: игрок бежит, прыгает, скользит, собирает бонусы и проигрывает при ошибке. На уровне проектирования за этим стоит цепочка маленьких проверок. Именно такие проверки и формируют игровое правило, а не только визуальную картинку.

## **2.2 Выбор и обоснование архитектурного стиля**

Для 2D-автораннера выбран компонентно-модульный архитектурный стиль, опирающийся на стандартную модель Unity: сцена состоит из `GameObject`, поведение объектов задается компонентами `MonoBehaviour`, а часть доменной логики выносится в обычные классы `C#`. Такой стиль хорошо подходит для игр малого и среднего размера, потому что позволяет использовать сильные стороны Unity и одновременно не превращать проект в набор неуправляемых скриптов. Важно не спорить с движком, а аккуратно надстроить над ним собственную архитектуру.

В рассматриваемом проекте это видно по разделению пространств имен. Базовая логика вынесена в `RusRunner.Core` и `RusRunner.Core.Systems`, игровые сущности находятся в `RusRunner.Game`, а визуальное оформление – в

RusRunner.Game.Presentation и RusRunner.Game.UI. Ввод вынесен в отдельный каталог Game/Input, где есть общий интерфейс IInputSource и две реализации: KeyboardInputSource и MobileInputSource. Такое разделение не выглядит чрезмерным. Напротив, оно позволяет читать проект по слоям: сначала состояние и системы, затем игровой процесс, затем представление.

Почему не была выбрана монолитная архитектура, где один PlayerController отвечает за движение, столкновения, счет, меню и генерацию? Ответ достаточно практичный: монолит быстрее написать на самом первом этапе, но он плохо переносит рост требований. В автораннере почти неизбежно появляются новые препятствия, новые бонусы, уровни сложности, настройки, рекорды, мобильное управление. Если все это находится в одном файле, разработчик начинает бояться изменять код. Любая мелкая правка может неожиданно сломать другой сценарий.

Компонентно-модульный стиль в данном случае помогает удерживать сложность. RunnerController знает про физику персонажа, но не знает, как сохраняется рекорд. ScoreSystem знает про очки и дистанцию, но не знает, какой спрайт используется для бонуса. TrackGenerator знает про активные сегменты, пулы и положение объектов, но не отвечает за текст на HUD. Это не идеальная изоляция, конечно, но для Unity-проекта она уже дает заметный выигрыш.

Можно также говорить о частичном применении идеи чистой архитектуры, но без строгого следования ей [5]. В проекте есть центральный контекст, есть доменные системы, есть внешние адаптеры в виде Unity-компонентов. При этом ядро не отделено в отдельную сборку, а сцена и MonoBehaviour все равно остаются важной частью решения.

С архитектурной точки зрения RunnerBootstrap выполняет роль фасада и координатора. Он скрывает от UI и сцены детали создания систем, обновления сложности, обработки бонусов и перезапуска. Однако он не должен разрастаться бесконечно. В текущем виде класс уже содержит несколько зон ответственности: запуск, ввод, столкновения, камера и создание runtime-систем. Поэтому при дальнейшем развитии проекта его можно разделить на отдельные сервисы:

RunStateController, CameraFollowController, InputDispatcher. Но для текущего объема игры выбранный компромисс оправдан.

Важной причиной выбора такого стиля стала необходимость оптимизации генерации объектов. TrackGenerator управляет отдельными списками активных сегментов, препятствий и бонусов, а также стеками пулов для разных типов объектов. Пул объектов является типичным решением для игр, где однотипные объекты создаются и удаляются часто. В Unity частые операции Instantiate и Destroy во время игры могут приводить к лишней нагрузке и сборке мусора, поэтому повторное использование GameObject становится не просто красивым приёмом, а практической мерой оптимизации [24].

Стоит отметить и то, что код уже отражает ориентацию на несколько платформ. Выбор источника ввода выполняется через условие Application.isMobilePlatform: для мобильных устройств используется MobileInputSource, для остальных – KeyboardInputSource. Если завтра потребуется добавить управление с геймпада, архитектура не заставит переписывать RunnerBootstrap целиком. Достаточно добавить новую реализацию InputSource и подставить ее по условию выбора платформы.

Компонентно-модульная архитектура также хорошо согласуется с C4-подходом к описанию системы. На верхнем уровне можно показать пользователя и программный комплекс, на уровне контейнеров – игровой клиент Unity и ресурсные подсистемы, а на уровне компонентов – RunnerBootstrap, TrackGenerator, RunnerController, системы ядра, UI и Presentation. За счет этого проект легче объяснить новичкам.

Итак, выбранный архитектурный стиль можно определить как гибрид Unity component-based architecture и модульного ядра с разделением игровой логики на системы. Это не академическая схема ради схемы. Она вырастает из реальных ограничений автораннера: игровой цикл должен обновляться каждый кадр, объекты должны генерироваться заранее, бонусы должны действовать ограниченное время, а интерфейс обязан быстро реагировать на изменение состояния.

В таблице 2.2 приведено сравнение архитектурных вариантов. На рисунке 2.2 изображены архитектурные слои проекта RusRunner.

Таблица 2.2 – Сравнение архитектурных вариантов для 2D-автораннера

| Вариант архитектуры               | Преимущества                                                                                 | Недостатки для проекта                                                                               | Итоговая оценка                                           |
|-----------------------------------|----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Монолитный контроллер             | Быстрый старт разработки, минимальное количество файлов.                                     | Слабая сопровождаемость, сильная связанность, трудность добавления новых бонусов и источников ввода. | Подходит только для самого раннего прототипа.             |
| Строгая чистая архитектура        | Высокая тестируемость и независимость ядра от Unity.                                         | Избыточна для небольшого игрового проекта; увеличивает объем шаблонного кода.                        | Может применяться частично, но не как жесткое требование. |
| ECS-подход                        | Высокий потенциал производительности, удобен для большого количества сущностей.              | Сложнее для реализации и объяснения; текущий проект не требует такого уровня оптимизации.            | Не выбран как основной стиль.                             |
| Компонентно-модульный стиль Unity | Соответствует движку, упрощает сцену, позволяет выделить игровые системы в отдельные классы. | Требует дисциплины, иначе MonoBehaviour снова начинают смешивать обязанности.                        | Выбран как основной архитектурный стиль.                  |

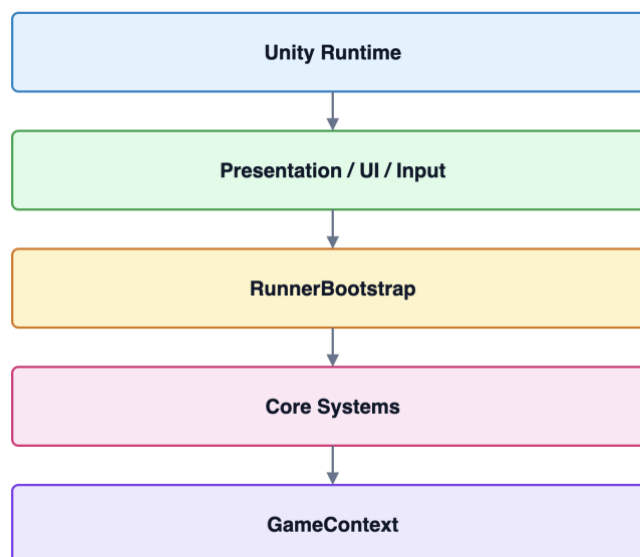


Рисунок 2.2 – Архитектурные слои проекта RusRunner

Практическая ценность выбранного стиля проявляется в том, что игровые системы можно рассматривать как небольшие независимые механизмы. Например, ProgressionSystem не хранит ссылку на сцену. Он принимает GameContext и deltaTime, затем увеличивает скорость и значение Difficulty01. ScoreSystem аналогично работает с контекстом и сохраняет лучший результат через PlayerPrefs только при завершении забега. PowerUpSystem хранит активные эффекты и уменьшает их таймеры. Все просто. Но именно эта простота и делает код удобным.

В проекте также используется подход к программированию через контракты. Интерфейсы IAbility и IPowerUp задают минимальный набор методов, который должен быть у способности или бонуса. Конкретная реализация может менять скорость, включать неуязвимость, добавлять множитель очков или давать дополнительное действие. Для ядра важно только то, что объект умеет применяться, обновляться и завершаться. Здесь хорошо просматривается принцип открытости/закрытости: новую разновидность эффекта желательно добавлять новым классом, а не изменением старого кода во многих местах.

HudController и MainMenuController строят адаптивный интерфейс программно, создавая Canvas, панели, кнопки, тексты и слайдеры. С одной стороны, такой подход уменьшает зависимость от заранее настроенной сцены и делает проект самодостаточным. С другой стороны, код интерфейса становится объёмным.

## **2.3 Моделирование архитектуры с использованием подхода C4**

Для описания архитектуры проекта использован подход C4, предложенный как способ создавать «карты» программной системы на разных уровнях детализации. Он удобен тем, что не заставляет сразу показывать все классы и связи. Сначала фиксируется контекст: кто взаимодействует с системой и зачем. Затем рассматриваются контейнеры, то есть крупные исполняемые или

хранимые части. После этого можно спуститься к компонентам и, если необходимо, к коду [2].

В отношении RusRunner C4-модель позволяет показать, что игра является не только сценой RunnerScene, но и программным комплексом из нескольких подсистем. Игрок взаимодействует с меню, игровым экраном и HUD. Unity Runtime обеспечивает выполнение скриптов, обработку физики 2D и отображение графики. Внутри клиентского приложения находится ядро, которое управляет состоянием забега, генерацией трассы, прогрессией, бонусами и реакцией на столкновения.

Важно не перегружать C4-диаграммы тем, что лучше оставить для UML [3]. Например, на контекстной диаграмме не нужно показывать CapsuleCollider2D, Rigidbody2D и отдельные методы TryGroundJump. Там достаточно указать игрока, программный комплекс, Unity как технологическую платформу и локальное хранилище PlayerPrefs, используемое для настроек и рекорда. На компонентной диаграмме, наоборот, такие детали, как RunnerController и TrackGenerator, уже уместны.

### **2.3.1 Диаграмма контекста системы**

Диаграмма контекста показывает RusRunner как самостоятельное приложение, с которым взаимодействует основной пользователь – игрок. Игрок запускает игру, выбирает начало забега, меняет настройки, управляет персонажем и видит результаты на HUD. Внешних серверов, сетевых сервисов и учетных записей в текущей версии проекта нет. Поэтому контекст получается компактным, что, кстати, не является недостатком. Для локального 2D-автораннера это естественная картина.

В качестве внешней технологической среды на диаграмме следует указать Unity Runtime. Формально Unity находится внутри приложения, но на уровне архитектурного описания удобно показать его как платформу, предоставляющую физический движок 2D, систему сцен, обработку кадров, рендеринг, ресурсы и PlayerPrefs. Именно через PlayerPrefs сохраняются

настройки громкости, целевая частота кадров и лучший результат. Пользователь напрямую с PlayerPrefs не взаимодействует, но приложение использует это хранилище для сохранения состояния между запусками.

Сценарии пользователя на контекстной диаграмме можно обозначить кратко: начать забег, управлять персонажем, собрать бонус, столкнуться с препятствием, увидеть результат, перезапустить игру, изменить настройки. Эти действия образуют границу системы: все, что связано с обработкой этих действий, относится к программному комплексу, а не к внешней среде. Важно не рисовать на этой диаграмме внутренние классы, иначе уровень контекста потеряет смысл.

На рисунке 2.3 нарисована диаграмма контекста системы RusRunner.

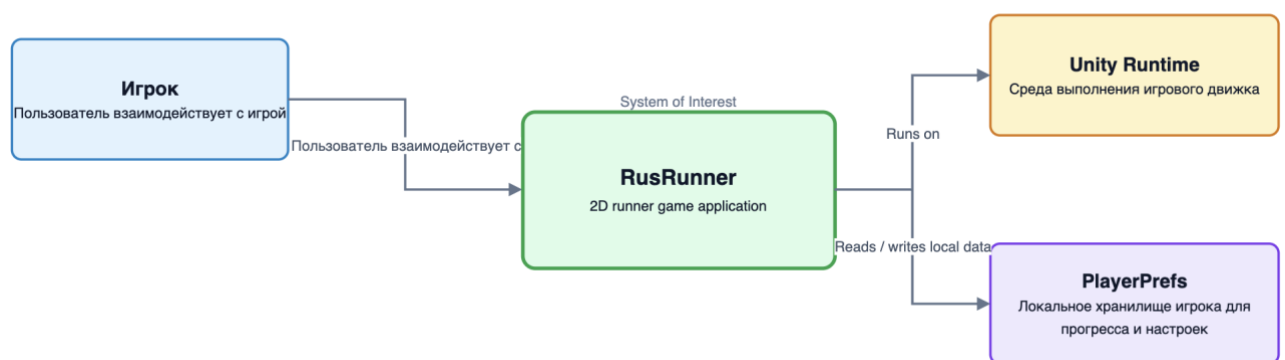


Рисунок 2.3 – Диаграмма контекста системы RusRunner

### 2.3.2 Диаграмма контейнеров

На уровне контейнеров система может быть представлена как Unity-клиент, включающий сцену RunnerScene, набор C#-скриптов, ресурсы Resources/Art, настройки проекта и локальное хранилище PlayerPrefs. В веб-приложениях контейнерами часто становятся фронтенд, сервер и база данных. В игровом Unity-проекте контейнерная модель выглядит иначе: основная исполняемая часть – клиентская сборка игры, а ресурсы и локальные данные являются сопровождающими контейнерами.

Главным контейнером является игровой клиент Unity. Он содержит сцены, компоненты MonoBehaviour и обычные C#-классы. Сцена RunnerScene служит

точкой сборки: в ней находятся объекты персонажа, земли, камеры, Canvas и скрипты, обеспечивающие игровой процесс. При этом часть элементов интерфейса и представления создается программно, поэтому клиент не полностью зависит от ручной настройки сцены.

Второй контейнер – ресурсный каталог Assets/Resources/Art. В нем находятся изображения фона, земли, персонажа, препятствий и бонусов. Класс ArtSpriteLibrary загружает спрайты из Resources по относительному пути, например Background/ground или Runner/run\_01. Благодаря этому ScenePresentationBootstrap и TrackGenerator могут назначать визуальное оформление объектам во время выполнения. Это чуть упрощает сборку прототипа: достаточно иметь корректную структуру ресурсов.

Третий контейнер – локальные настройки и сохранения. В коде GameSettings используются ключи RusRunner.MusicVolume, RusRunner.EffectsVolume и RusRunner.TargetFps, а в ScoreSystem – ключ rusrunner\_best\_score. Эти значения сохраняются через PlayerPrefs. Для текущего проекта такого хранилища достаточно, потому что игра не требует профилей, синхронизации между устройствами и сложной статистики. Если система будет расширяться, контейнер локального хранения можно заменить или дополнить JSON-файлом, SQLite или облачным сервисом.

В таблице 2.3 указаны контейнеры, их содержимое и назначение. На рисунке 2.4 нарисована диаграмма компонентов проекта RusRunner.

Таблица 2.3 – Контейнеры программного комплекса RusRunner

| Контейнер                  | Содержимое                                                                       | Назначение                                                                   |
|----------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| Игровой клиент Unity       | RunnerScene, C#-скрипты, компоненты MonoBehaviour, настройки физики и рендеринга | Выполнение игрового процесса и отображение результата пользователю.          |
| Ресурсный каталог Art      | Спрайты фона, земли, персонажа, препятствий, бонусов                             | Предоставление визуальных материалов для генератора и презентационного слоя. |
| PlayerPrefs                | Рекорд, громкость музыки и эффектов, целевой FPS                                 | Сохранение небольших пользовательских настроек и результата между сессиями.  |
| ProjectSettings и Packages | Unity 6000.0.43f1, URP 17.0.4, Input System 1.13.1, uGUI 2.0.0                   | Фиксация технологической среды и подключенных пакетов.                       |

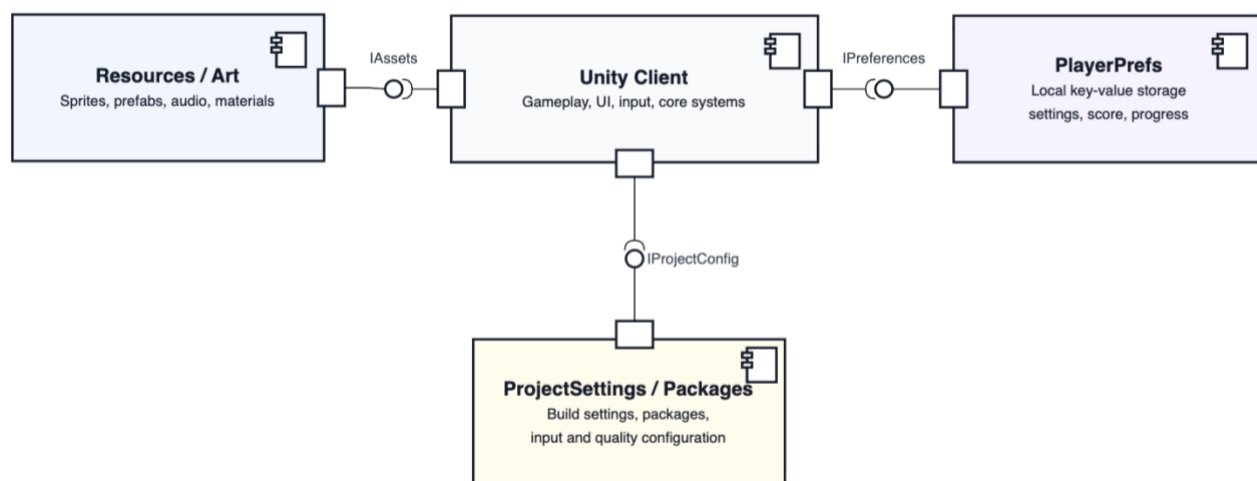


Рисунок 2.4 – Диаграмма компонентов проекта RusRunner

### 2.3.3 Диаграмма компонентов игрового ядра

Компонентная диаграмма должна раскрывать внутреннее устройство игрового клиента. В центре находится **RunnerBootstrap**, потому что именно он запускает и обновляет игровой цикл. С ним связаны **GameContext**, **ProgressionSystem**, **ScoreSystem**, **PowerUpSystem**, **RunnerController**, **TrackGenerator**, **HudController** и **MainMenuController**. При этом **RunnerBootstrap** не обязан знать, как именно **HudController** создает кнопки или как **TrackGenerator** строит каждый сегмент, но он задает общий порядок работы.

**GameContext** на компонентной диаграмме удобно показать как общий объект состояния. Его читают и изменяют системы ядра. **RunnerController** обновляет признаки **IsGrounded** и **IsSliding**, **ProgressionSystem** меняет **CurrentSpeed** и **Difficulty01**, **ScoreSystem** увеличивает **DistanceMeters** и **Score**, **PowerUpSystem** временно изменяет множители и признаки эффектов. Такая модель похожа на легковесный игровой state object. Она проще, чем полноценная ECS-архитектура, но уже отделяет состояние от представления.

**TrackGenerator** следует показать как компонент процедурной генерации мира. Он получает сложность через **SetDifficulty** и скорость прокрутки через **SetScrollSpeed**. На каждом кадре генератор сдвигает активные объекты влево, порождает новые сегменты впереди игрока и возвращает ушедшие объекты в

пулы. TrackGenerator взаимодействует не только с землей, но и с препятствиями, бонусами и презентационным слоем через ArtSpriteLibrary.

RunnerController следует отнести к компонентам управления персонажем. Он работает с Rigidbody2D, CapsuleCollider2D и Physics2D.Raycast, но наружу предоставляет более высокоуровневые операции: TryGroundJump, TryAirJump, Slide, Halt, KeepFixedOnGround и RestoreAfterMenuOrRestart. Это делает связь с ядром достаточно чистой: RunnerBootstrap не рассчитывает силу прыжка и не меняет коллайдер напрямую.

UI-компоненты на диаграмме лучше показать отдельно. MainMenuController отвечает за главное меню и настройки, HudController – за отображение статуса забега и панели проигрыша. Они обращаются к RunnerBootstrap как к публичному фасаду состояния. Такой вариант проще для сопровождения, чем доступ UI напрямую к GameContext, потому что внешний интерфейс RunnerBootstrap уже отфильтрован и содержит только те данные, которые действительно нужны экрану (рис. 2.5).

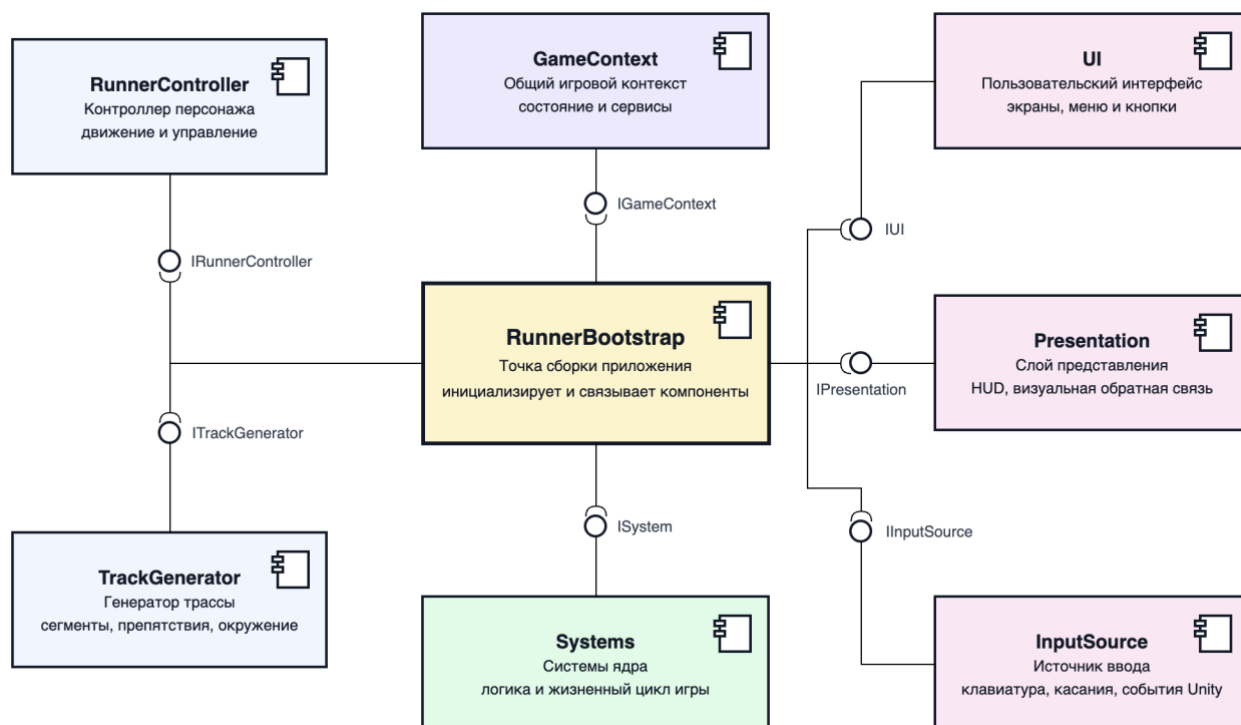


Рисунок 2.5 – Диаграмма компонентов игрового ядра RusRunner

Отдельно стоит сказать про входы в ядро: команды ввода `JumpPressed` и `SlidePressed`, события столкновения `OnTriggerEnter2D` и `OnTriggerEnter2D` у `PowerUpPickup`, команды UI `StartGame`, `Restart` и `ReturnToMainMenu`. Все они приводят к изменению `GameContext` или состояния сцены. Поэтому ядро не является пассивной библиотекой: оно реагирует на поток событий, но сохраняет контроль над правилами.

## 2.4 Проектирование пользовательского интерфейса

Пользовательский интерфейс в 2D-автораннере выполняет не только декоративную функцию. Он задает вход в игровой процесс, показывает текущую информацию во время забега, позволяет изменить базовые настройки и обеспечивает понятное завершение игровой сессии. В проекте эта зона реализована классами `MainMenuController` и `HudController`, причем оба контроллера строят адаптивный интерфейс программно на базе Unity UI: `Canvas`, `CanvasScaler`, `Text`, `Button`, `Slider`, `Image` и `EventSystem`.

Решение создавать UI программно имеет свои плюсы и минусы. Плюс в том, что интерфейс может быть восстановлен даже при изменении сцены или отсутствии заранее подготовленного `Canvas`. `MainMenuController` содержит метод `EnsureUi`, который создает `Canvas`, корневой объект, панель главного меню, панель настроек, слайдеры громкости и кнопки выбора FPS. `HudController` аналогично вызывает `EnsureAdaptiveUi`, создает верхнюю панель статуса и `overlay` для поражения. Минус очевиден: код становится длиннее, а визуальное редактирование в инспекторе Unity используется меньше.

Для дипломного проекта такой подход оправдан, потому что он демонстрирует программное проектирование интерфейса, а не только ручную расстановку элементов. Кроме того, UI получает адаптивность через `CanvasScaler` с режимом `ScaleWithScreenSize` и базовым разрешением 1920×1080. Это позволяет интерфейсу сохранять пропорции на разных экранах. В

автораннере, где часть аудитории может запускать игру на ноутбуках с разным разрешением, такая настройка не лишняя [28].

Интерфейс можно разделить на три группы экранных форм. Первая группа – главное меню, где пользователь начинает игру, открывает настройки или выходит. Вторая группа – экран настроек, где изменяются громкость музыки, громкость эффектов и целевой FPS. Третья группа – игровой HUD, который отображает очки, рекорд, дистанцию, скорость и состояние персонажа. При поражении поверх игрового экрана показывается панель с итоговой статистикой и кнопками перезапуска или возврата в меню.

#### **2.4.1 Разработка прототипов экранных форм**

Прототип главного меню должен быть простым, потому что пользователь приходит в игру не читать интерфейс, а быстро начать забег. На форме целесообразно разместить название игры, кнопку «Играть», кнопку «Настройки» и кнопку выхода (рис. 2.6).

Прототип настроек включает два слайдера и набор кнопок частоты кадров (рис. 2.7). Значения сохраняются через `GameSettings.Save`, после чего сразу применяется `Application.targetFrameRate` и общий уровень громкости `AudioListener.volume`. Здесь видно, что интерфейс настроек связан не с игровым забегом напрямую, а с пользовательскими предпочтениями. Поэтому его логично держать отдельно от HUD.

Прототип игрового HUD должен быть максимально ненавязчивым. В верхней части экрана размещаются значения очков, рекорда, дистанции и скорости. Рядом или ниже выводится строка состояния: бег, подкат, активный щит, множитель очков, оставшиеся дополнительные прыжки. Такая информация помогает игроку понимать последствия действий. Например, если собран щит, статус показывает «щит», а игрок может рискнуть пройти через препятствие.

Панель проигрыша появляется только при `IsAlive = false`. Она должна содержать итоговые очки, рекорд, дистанцию и два основных действия: начать заново или вернуться в меню. В коде `HudController` дополнительно указывает,

что можно нажать R. Это мелкая деталь, но она делает управление удобнее для пользователя клавиатуры (рис. 2.8).

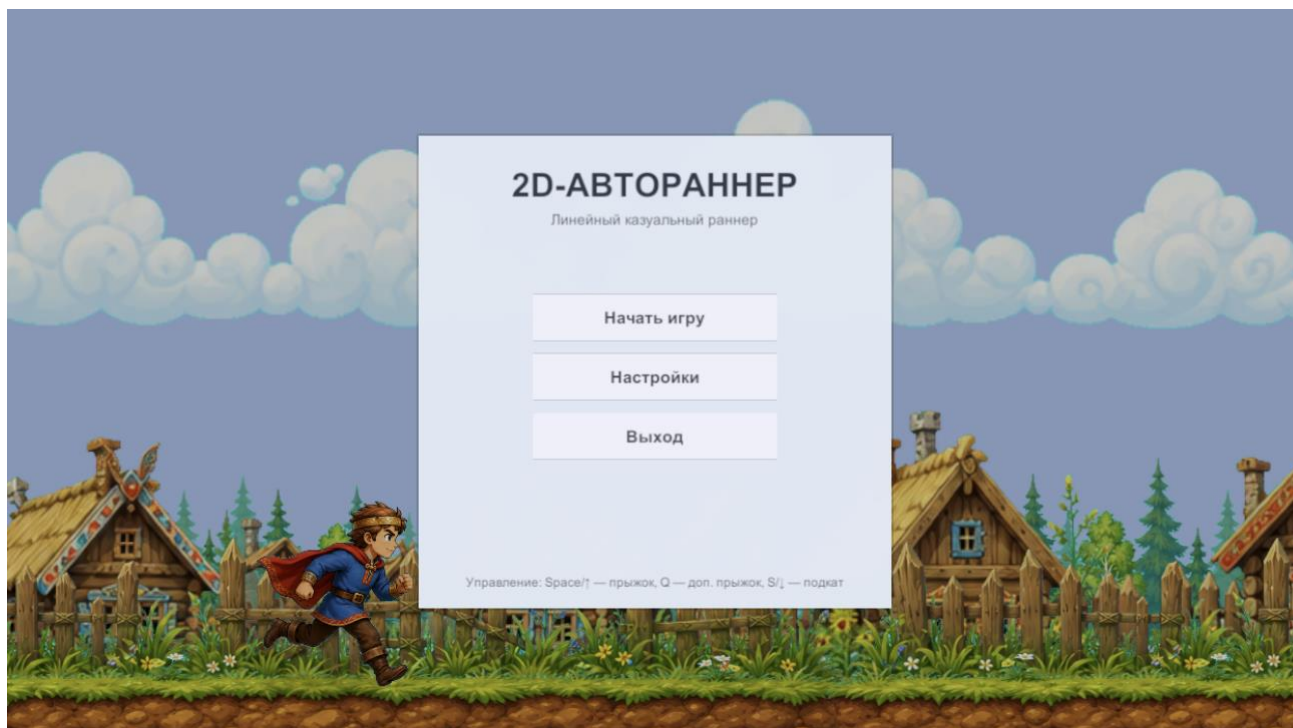


Рисунок 2.6 – Прототип главного меню

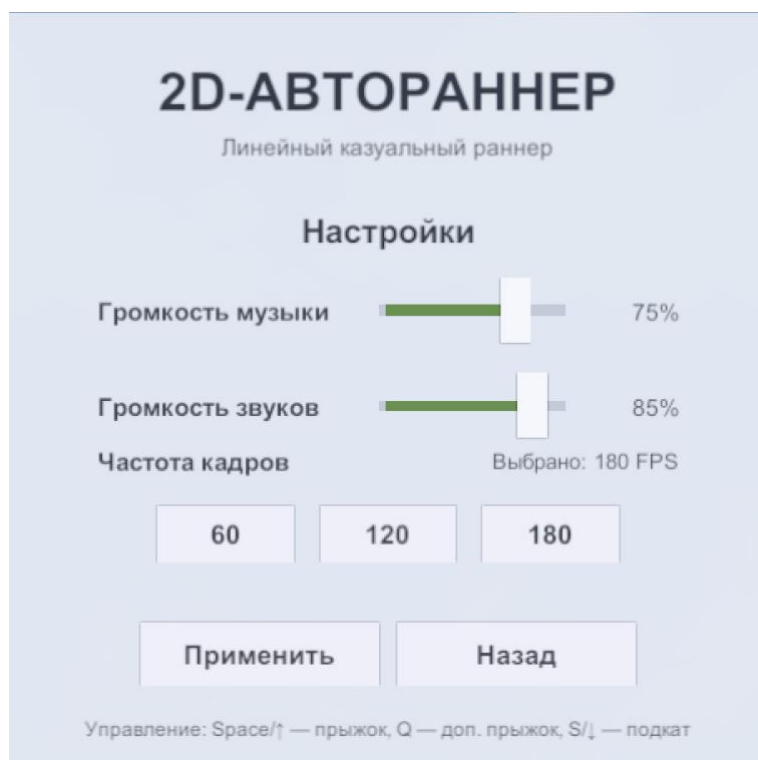


Рисунок 2.7 – Прототип экрана настроек

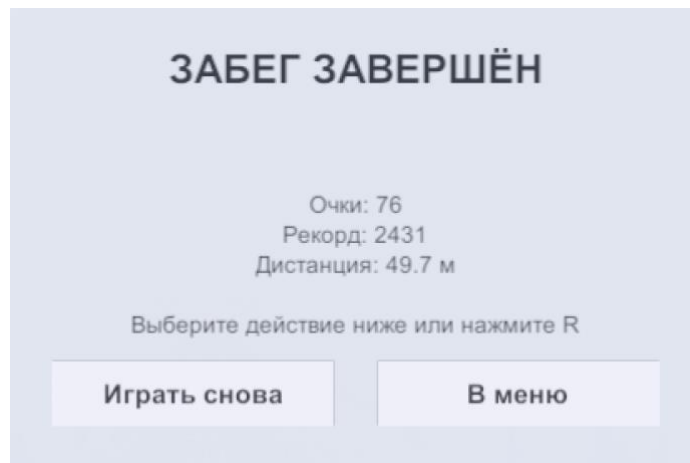


Рисунок 2.8 – Прототип панели поражения

В таблице 2.4 описаны экранные формы пользовательского интерфейса, их основные элементы и связанные классы.

Таблица 2.4 – Экранные формы пользовательского интерфейса

| Экранная форма   | Основные элементы                           | Связанные классы                             |
|------------------|---------------------------------------------|----------------------------------------------|
| Главное меню     | Название, кнопки запуска, настроек и выхода | MainMenuController, RunnerBootstrap          |
| Настройки        | Слайдеры громкости, выбор FPS, сохранение   | MainMenuController, GameSettings             |
| Игровой HUD      | Очки, рекорд, дистанция, скорость, статус   | HudController, RunnerBootstrap               |
| Панель поражения | Итоги забега, перезапуск, возврат в меню    | HudController, RunnerBootstrap, SceneManager |

### 2.4.2 Описание навигации и пользовательских сценариев

Навигация в проекте строится не через несколько сцен, а преимущественно через переключение состояний внутри одной игровой сцены. Пользователь попадает в главное меню, нажимает кнопку запуска, после чего MainMenuController скрывает меню и вызывает BeginRun. Игровой процесс начинается без загрузки отдельной сцены. Это уменьшает задержки и упрощает структуру проекта.

Сценарий изменения настроек начинается с главного меню. Пользователь нажимает «Настройки», контроллер скрывает основную панель и показывает панель настроек. После изменения значений вызывается SaveSettings, который

передает параметры в `GameSettings.Save`. Затем настройки применяются, а пользователь возвращается в меню. Такое поведение понятно: настройки не запускают забег сами по себе, они только меняют параметры выполнения.

Сценарий игрового забега начинается после `BeginRun`. `RunnerBootstrap` сбрасывает `GameContext`, создает runtime-системы, восстанавливает `RunnerController` и задает генератору трассы начальную сложность. Далее каждый кадр выполняется цепочка `Update`: контроллер персонажа обновляет состояние, ввод преобразуется в прыжок или подкат, системы прогрессии и счета изменяют значения контекста, генератор прокручивает мир. HUD каждый кадр читает публичные свойства `RunnerBootstrap` и отображает их на экране.

Сценарий поражения запускается при столкновении с препятствием. Если нет щита и нет корректного подката под препятствием `Slide`, `RunnerBootstrap` переводит контекст в состояние `IsAlive = false`, останавливает персонажа, выключает прокрутку мира и фиксирует возможный рекорд. HUD показывает панель поражения.

Пользователь может нажать кнопку перезапуска или клавишу `R`, после чего сцена загружается заново с флагом немедленного старта. Можно вернуться и в меню – тогда флаг сбрасывается.

Получается достаточно цельная навигационная схема. Пользователь не видит технических перезагрузок, а разработчик получает простой способ сбросить сцену в чистое состояние.

С точки зрения идеальной архитектуры можно было бы реализовать полный `reset` объектов без `SceneManager.LoadScene`. Но для автораннера с процедурной трассой перезагрузка сцены – надежное решение, которое уменьшает риск забыть очистить какой-либо активный объект или пул.

На рисунке 2.9 нарисована диаграмма деятельности пользователя с игрой `RusRunner`.

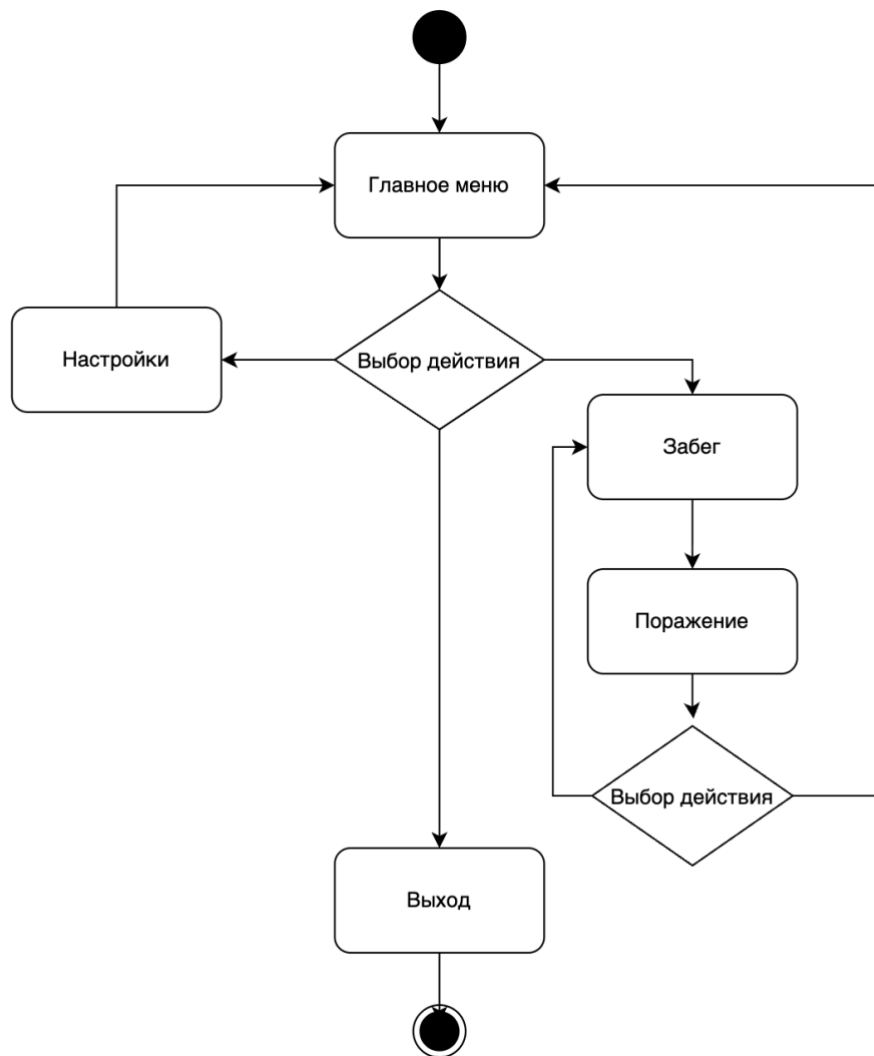


Рисунок 2.9 – Диаграмма деятельности пользователя

## 2.5 Детальное проектирование ключевых классов ядра

Детальное проектирование ключевых классов позволяет перейти от общей архитектуры к конкретным программным решениям. В проекте RusRunner ключевыми являются классы, которые либо управляют игровым циклом, либо изменяют состояние забега, либо обеспечивают постоянное порождение объектов игрового мира. Это RunnerBootstrap, GameContext, RunnerController, TrackGenerator, ScoreSystem, ProgressionSystem, PowerUpSystem, AbilitySystem, а также интерфейсы IInputSource, IAbility и IPowerUp.

Наличие методов TryGroundJump и Slide в RunnerController объясняется тем, что это физические действия персонажа, завязанные на Rigidbody2D и

Collider2D. А вот начисление очков в этот класс помещать нельзя: персонаж не должен сам решать, сколько баллов получает игрок.

В целом структура кода отражает переход от Unity-компонентов к более чистым системам. MonoBehaviour-классы отвечают за работу со сценой, объектами, вводом, UI и физикой. Обычные C#-классы отвечают за логику счета, прогрессии и временных эффектов. Этот переход не абсолютный, но он уже делает проект более понятным.

### **2.5.1 Диаграммы классов ключевых компонентов**

На диаграмме классов игрового ядра центральным элементом должен быть GameContext. Он не содержит сложного поведения, но его поля используются почти всеми системами. В нем хранятся CurrentSpeed, DistanceMeters, Score, IsGrounded, IsAlive, IsInvulnerable, IsSliding, SpeedMultiplier, ScoreMultiplier и AirJumpsLeft. Метод Reset возвращает контекст в исходное состояние. Благодаря этому при начале нового забега RunnerBootstrap не создает разрозненный набор переменных, а сбрасывает единую модель состояния.

RunnerBootstrap на диаграмме следует связать с RunnerController, TrackGenerator, ProgressionSystem, ScoreSystem, PowerUpSystem и InputSource. Он содержит публичные свойства для UI, но внутренние поля контекста наружу напрямую не отдает. Это важно, потому что UI получает только представление данных, а не возможность произвольно менять состояние игры.

RunnerController зависит от Unity-физики. На диаграмме можно показать его связь с Rigidbody2D, CapsuleCollider2D, Transform groundCheck и LayerMask groundMask. Методы TryGroundJump, TryAirJump и Slide должны быть выделены как основные операции управления. Метод Tick отвечает за обновление заземления, таймера подката, coyote time и фиксации позиции по X. Внутренняя логика коллайдера скрыта в ApplySlideCollider и RestoreCollider.

TrackGenerator является наиболее насыщенным классом по внутренним структурам. Он содержит списки активных GroundSegment, SpawnedObstacle и SpawnedPickup, а также отдельные Stack<GameObject> для разных пулов.

Активные объекты хранятся отдельно от свободных объектов в пуле. Это и есть основа оптимизации генерации.

Системы ScoreSystem и ProgressionSystem являются независимыми сервисами. ScoreSystem имеет метод Tick для начисления дистанции и очков, а также FinalizeRun для сохранения лучшего результата. ProgressionSystem увеличивает CurrentSpeed и Difficulty01. PowerUpSystem содержит список активных IPowerUp и вызывает Apply, Tick и Expire по таймеру. AbilitySystem похож по структуре, но работает с IAbility и регистрацией по строковому идентификатору.

В таблице 2.5 указаны зоны ответственности ключевых классов и интерфейсов игрового ядра. На рисунке 2.10 нарисована диаграмма классов ядра.

Таблица 2.5 – Ключевые классы и интерфейсы игрового ядра

| Класс или интерфейс | Тип              | Ключевая ответственность                                                                  |
|---------------------|------------------|-------------------------------------------------------------------------------------------|
| RunnerBootstrap     | MonoBehaviour    | Координация игрового цикла, запуск и перезапуск, обработка ввода, столкновений и бонусов. |
| GameContext         | Обычный C#-класс | Единое изменяемое состояние текущего забега.                                              |
| RunnerController    | MonoBehaviour    | Физическое поведение персонажа: прыжок, подкат, остановка, коллайдеры.                    |
| TrackGenerator      | MonoBehaviour    | Генерация, прокрутка и повторное использование объектов трассы.                           |
| ScoreSystem         | Обычный C#-класс | Начисление очков и сохранение лучшего результата.                                         |
| ProgressionSystem   | Обычный C#-класс | Плавный рост скорости и сложности.                                                        |
| PowerUpSystem       | Обычный C#-класс | Управление активными временными бонусами.                                                 |
| InputSource         | Интерфейс        | Абстракция источника пользовательского ввода.                                             |
| IPowerUp / IAbility | Интерфейсы       | Контракты для расширяемых бонусов и способностей.                                         |

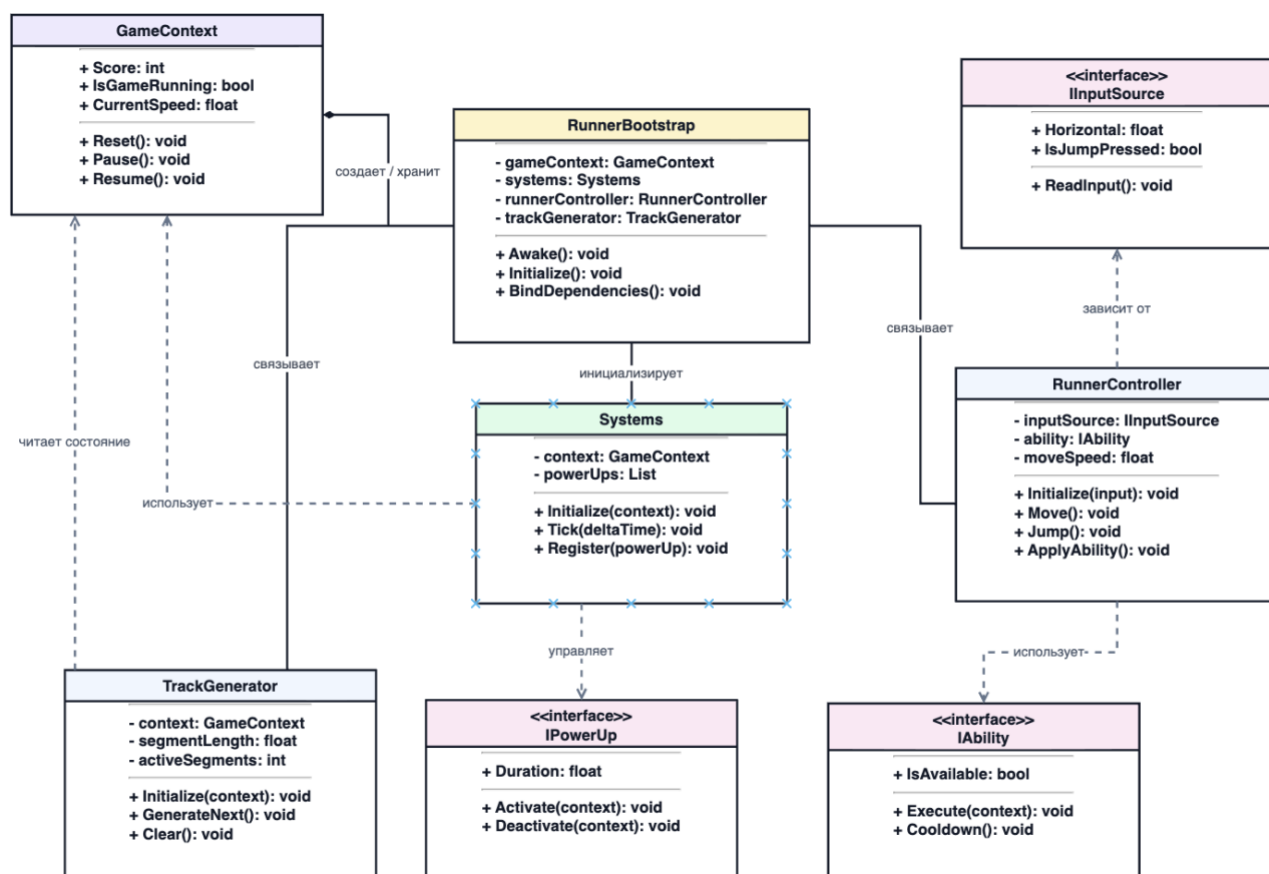


Рисунок 2.10 – UML-диаграмма классов игрового ядра

Фрагмент показывает, что состояние забега хранится в одном объекте. Это не просто удобство. Если разные подсистемы будут хранить собственные копии скорости, счета или признака жизни, рано или поздно возникнет рассинхронизация. GameContext в таком случае выступает как общий источник правды для игрового цикла.

Отдельная диаграмма классов может быть подготовлена для бонусов. На ней IPowerUp является интерфейсом, а ShieldPowerUp, ScoreBoostPowerUp и SpeedBoostPowerUp – конкретными реализациями. ShieldPowerUp включает IsInvulnerable, ScoreBoostPowerUp временно меняет ScoreMultiplier, SpeedBoostPowerUp изменяет SpeedMultiplier. Важно, что все эти эффекты имеют DurationSeconds, поэтому PowerUpSystem может управлять ими одинаково (рис. 2.11).

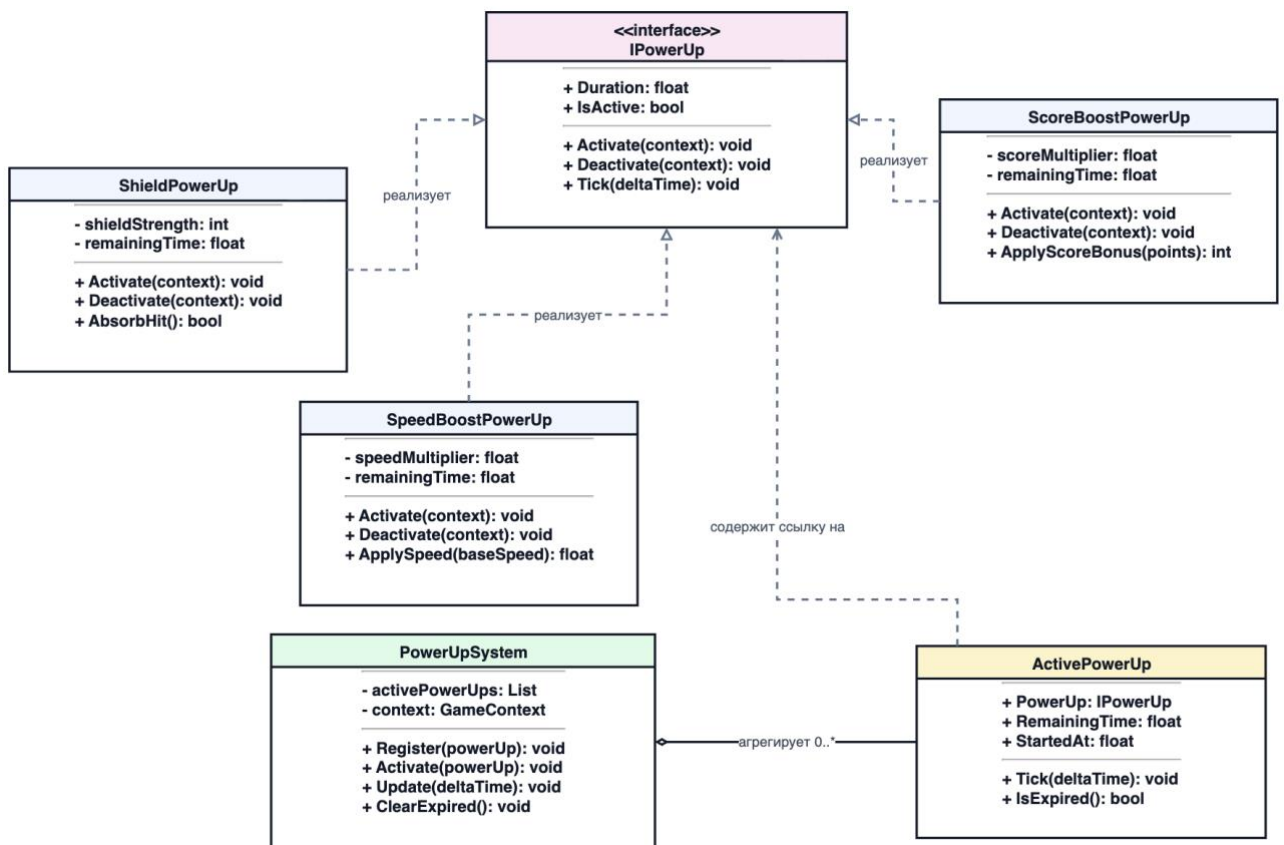


Рисунок 2.11 – UML-диаграмма классов подсистемы бонусов

## 2.5.2 Диаграммы последовательности для основных сценариев

Диаграммы последовательности позволяют показать динамику работы ядра. Для проекта RusRunner целесообразно подготовить как минимум четыре сценария: запуск забега, обработка прыжка, подбор бонуса и столкновение с препятствием. Эти сценарии покрывают основные типы взаимодействий: UI-команда, пользовательский ввод, триггер игрового объекта и событие поражения.

Сценарий запуска начинается в MainMenuController. Пользователь нажимает кнопку запуска, обработчик StartGame скрывает меню и вызывает RunnerBootstrap.BeginRun. Затем RunnerBootstrap применяет настройки, создает ProgressionSystem и ScoreSystem, сбрасывает GameContext, устанавливает базовую скорость и переводит `_isRunStarted` в `true`. RunnerController восстанавливает физическое состояние персонажа, а TrackGenerator получает

нулевую сложность и текущую скорость прокрутки. После этого в Update начинается обычный игровой цикл (рис. 2.12).

Сценарий прыжка начинается с `InputSource`. Для клавиатуры `KeyboardInputSource` возвращает `true` при нажатии соответствующей клавиши, для мобильного управления `MobileInputSource` анализирует касание и свайп. `RunnerBootstrap.ProcessInput` вызывает `runnerController.TryGroundJump`. Внутри `RunnerController` проверяется заземление или остаток `coyote time`, сбрасывается подкат, задается `AirJumpsLeft` и вызывается `JumpImpulse`. Контекст получает `IsGrounded = false` (рис. 2.13).

Сценарий подбора бонуса начинается с `PowerUpPickup.OnTriggerEnter2D`. Объект определяет столкновение с персонажем, передает тип бонуса в `RunnerBootstrap`. Метод `CollectPickup` выбирает конкретный объект бонуса: `ShieldPowerUp`, `ScoreBoostPowerUp` или `SpeedBoostPowerUp`, после чего добавляет его в `PowerUpSystem`. Система вызывает `Apply` и кладет эффект в список активных. На последующих кадрах `Tick` уменьшает таймер, а после завершения вызывает `Expire` (рис. 2.14).

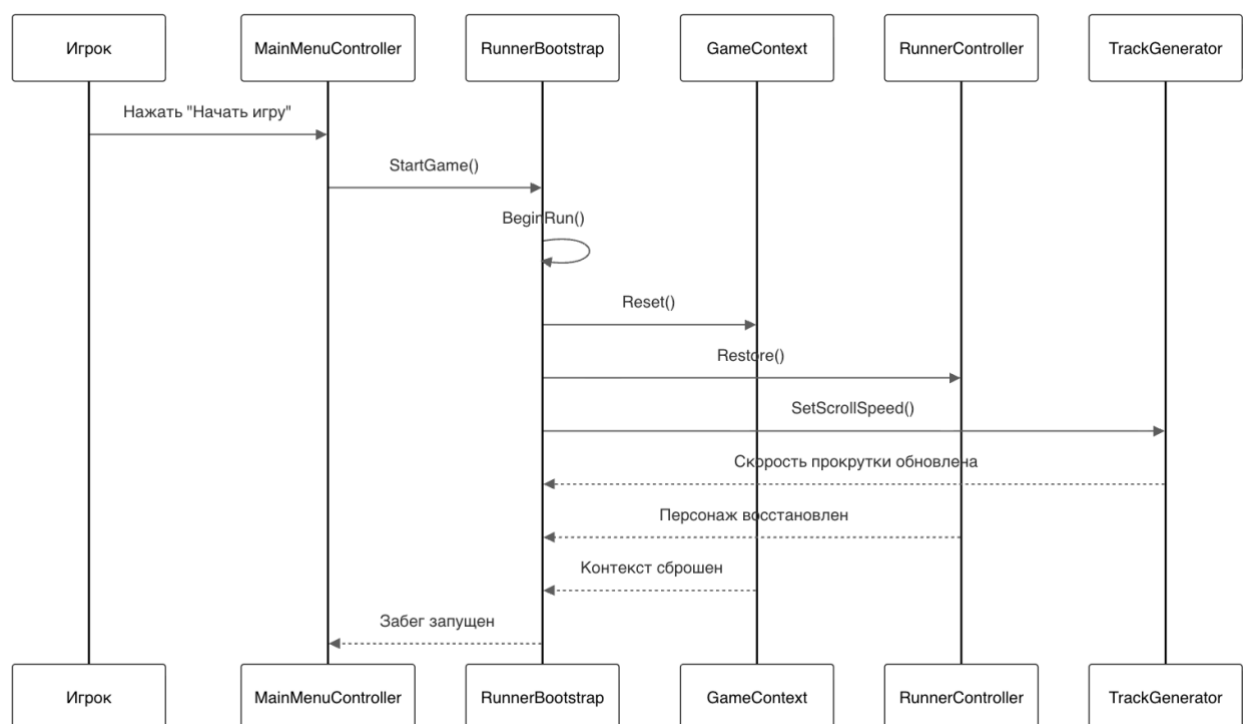


Рисунок 2.12 – Диаграмма последовательности запуска забега

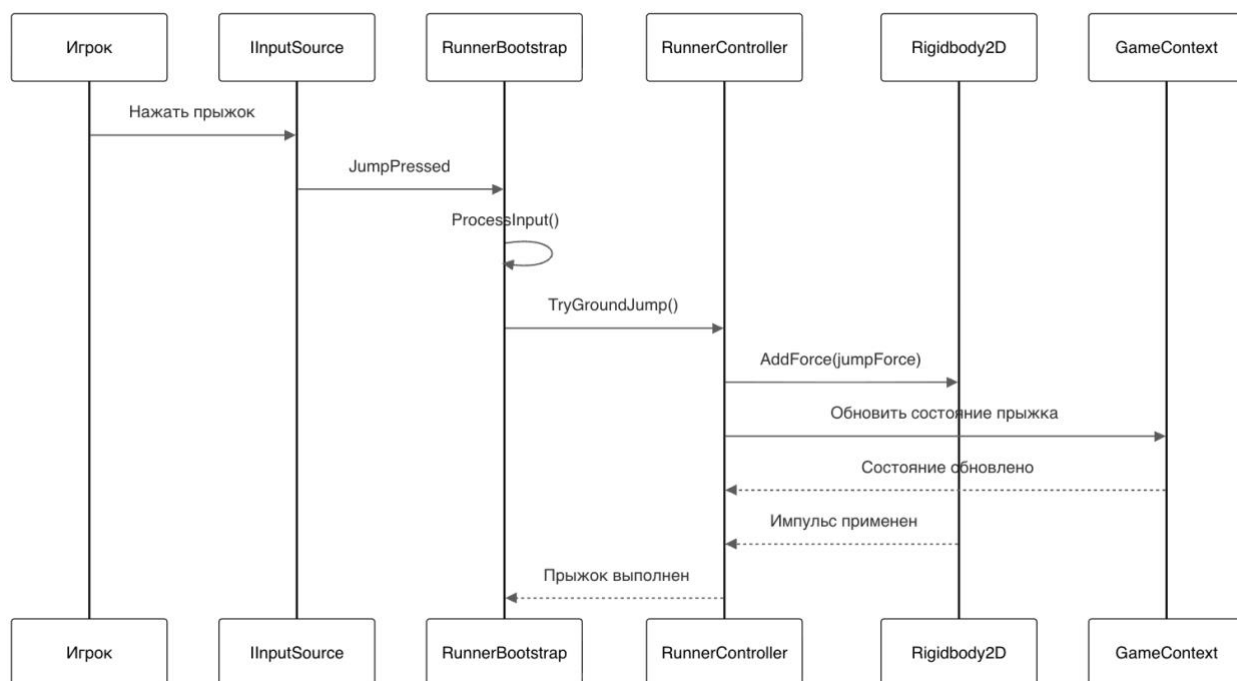


Рисунок 2.13 – Диаграмма последовательности обработки прыжка

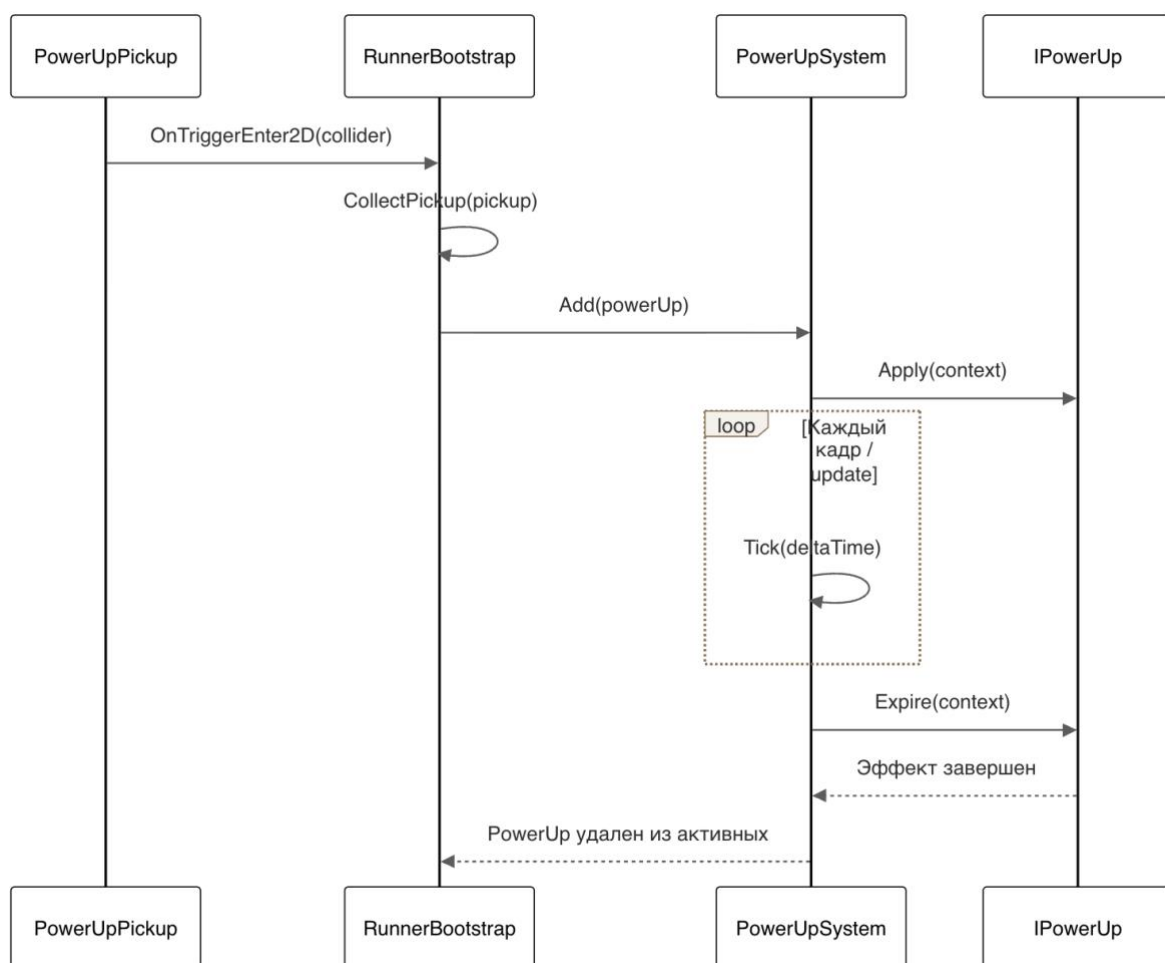


Рисунок 2.14 – Диаграмма последовательности подбора бонуса

Сценарий столкновения с препятствием начинается в RunnerCollisionRelay или Obstacle. Событие передается в RunnerBootstrap.HandleObstacleHit. Здесь проверяется несколько условий: начат ли забег, жив ли персонаж, активна ли неуязвимость, является ли препятствие типом Slide и находится ли игрок в подкате. Если защита не срабатывает, контекст переводится в состояние поражения, персонаж останавливается, генератор трассы перестает прокручивать мир, а ScoreSystem сохраняет рекорд (рис. 2.15).

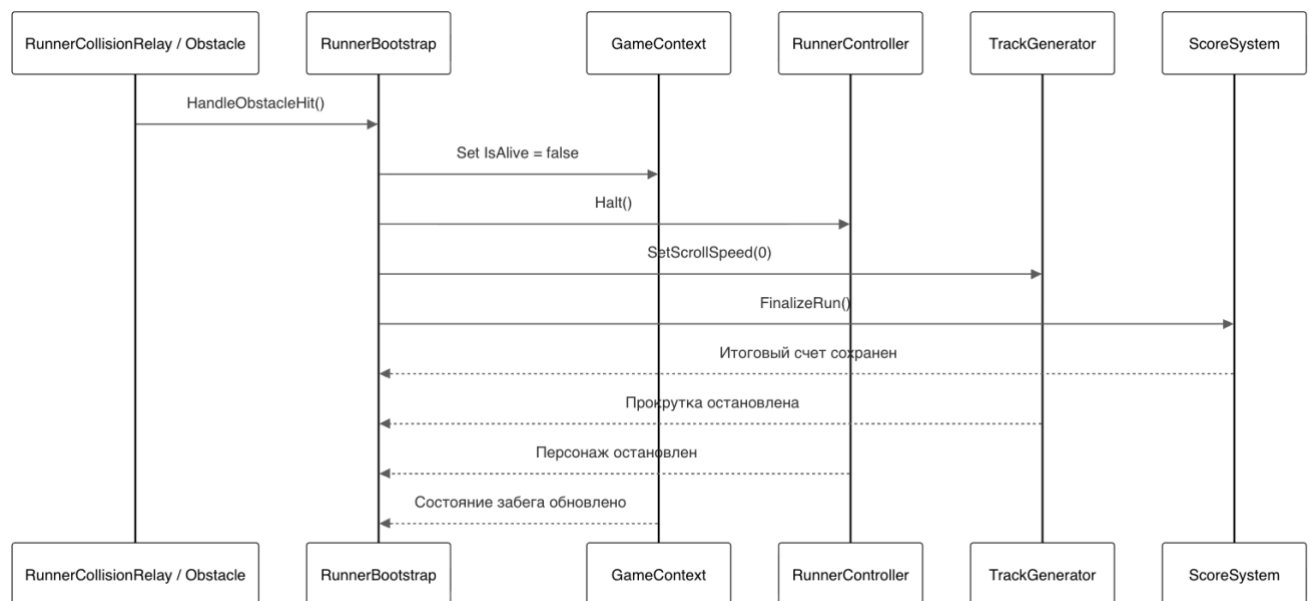


Рисунок 2.15 – Диаграмма последовательности столкновения с препятствием

Для сценария игрового цикла можно дополнительно сделать укрупнённую диаграмму Update. Она должна показать порядок: проверка состояния забега, Tick контроллера персонажа, ProcessInput, Tick прогрессии, Tick бонусов, Tick счета, MoveForward, SetDifficulty и SetScrollSpeed (рис. 2.16).

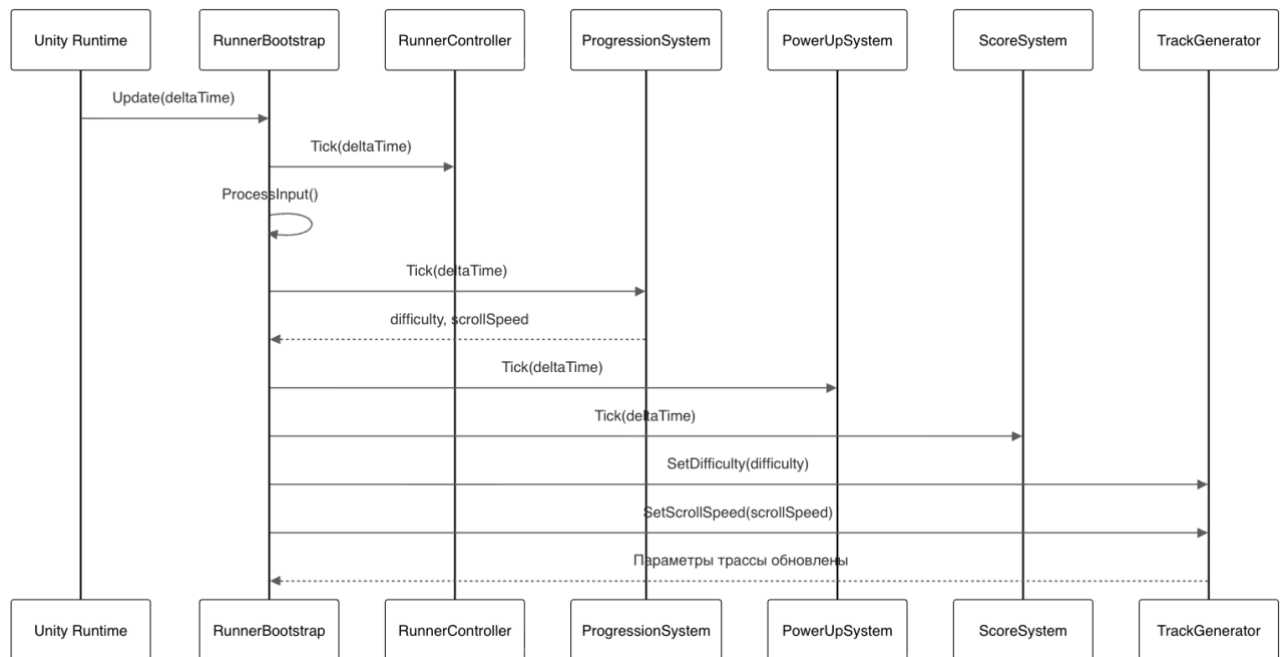


Рисунок 2.16 – Диаграмма последовательности одного кадра игрового цикла

### 2.5.3 Применение шаблонов проектирования

В проекте применяются несколько шаблонов проектирования, хотя не все они названы прямо в коде [4]. Наиболее заметный шаблон – Facade. Его роль выполняет RunnerBootstrap, который предоставляет внешним компонентам упрощенный доступ к игровому процессу. UI не знает всех деталей ScoreSystem, ProgressionSystem и PowerUpSystem, а обращается к методам BeginRun, RestartRun, ReturnToMainMenu, CollectPickup и публичным свойствам состояния.

Второй важный шаблон – Object Pool. Он реализован в TrackGenerator через набор стеков GameObject для земли, препятствий и бонусов. Когда объект уходит за пределы экрана, он не уничтожается, а выключается и помещается в соответствующий пул. Когда нужен новый объект, генератор пытается взять его из пула, активировать и перенастроить. Это уменьшает количество операций создания и удаления объектов во время игры.

Третий шаблон – Strategy. Он проявляется в InputSource. RunnerBootstrap работает с абстракцией источника ввода, а конкретная стратегия выбирается в зависимости от платформы. KeyboardInputSource реализует ввод с клавиатуры,

MobileInputSource – ввод через касания и свайпы. Благодаря этому ядро не превращается в набор условий внутри каждого обработчика.

Четвертый шаблон можно определить как Command/Effect object для бонусов. Каждый бонус является объектом, который знает, как применить эффект к GameContext и как отменить его после истечения времени. ShieldPowerUp, ScoreBoostPowerUp и SpeedBoostPowerUp меняют разные поля контекста, но для PowerUpSystem они выглядят одинаково. Это аккуратный способ управлять временными эффектами.

Также в проекте просматривается Template-like организация игрового цикла. RunnerBootstrap.Update задает общий порядок операций на кадре, а конкретные детали делегируются отдельным объектам. Сам RunnerBootstrap не вычисляет новый счет, не проверяет raucast земли и не пересобирает списки объектов трассы. Он вызывает системы в правильной последовательности.

В таблице 2.6 указаны использованные шаблоны.

Таблица 2.6 – Использование шаблонов проектирования в проекте

| Шаблон        | Проявление в проекте                                             | Практический эффект                                      |
|---------------|------------------------------------------------------------------|----------------------------------------------------------|
| Facade        | RunnerBootstrap скрывает детали работы подсистем ядра            | UI и внешние события обращаются к одному координатору.   |
| Object Pool   | Стеки объектов в TrackGenerator для земли, препятствий и бонусов | Снижение количества Instantiate/Destroy во время забега. |
| Strategy      | InputSource, KeyboardInputSource, MobileInputSource              | Замена способа ввода без переписывания игрового цикла.   |
| Effect Object | IPowerUp и конкретные бонусы                                     | Единая обработка разных временных эффектов.              |
| State Object  | GameContext как общий объект состояния                           | Согласованность данных между подсистемами.               |

Применение шаблонов в проекте нельзя назвать формальным. Они введены там, где решают конкретные задачи: уменьшить связанность, повысить расширяемость, снизить нагрузку при генерации. Это удачный вариант для практической разработки. Шаблон ради шаблона редко улучшает код, а иногда делает его тяжелее. Здесь же большинство решений поддержано реальной игровой необходимостью.

## 2.6 Оптимизация модульного ядра и генерации объектов

Оптимизация модульного ядра автораннера связана прежде всего с особенностью жанра. Игра постоянно создает иллюзию движения вперед: земля, препятствия, бонусы и фон должны непрерывно появляться перед игроком и исчезать позади него. Если каждый такой объект создавать и уничтожать через `Instantiate` и `Destroy`, нагрузка на рантайм будет расти, а на слабых устройствах могут появиться заметные микрофризы. Для игры, где реакция игрока зависит от плавности движения, это критично.

В проекте выбран подход повторного использования объектов. `TrackGenerator` хранит активные объекты в списках, а свободные – в стеках пулов. Для земли используется `_groundPool`, для препятствий – `_jumpObstaclePool`, `_slideObstaclePool`, `_airObstaclePool`, для бонусов – `_shieldPickupPool`, `_scorePickupPool` и `_speedPickupPool`. Разделение пулов по типам избавляет от лишней перенастройки объекта и уменьшает вероятность ошибки, когда объект одного вида внезапно используется как другой.

Алгоритм работы генератора можно описать так. При инициализации создаются корневые объекты и шаблоны, очищаются устаревшие `placeholder`-элементы сцены, затем строится начальный набор сегментов. На каждом кадре генератор получает скорость прокрутки, сдвигает активные объекты влево, проверяет расстояние до точки генерации и, пока впереди недостаточно трассы, создает следующий сегмент. После этого объекты, ушедшие за `despawnDistance`, выключаются и возвращаются в соответствующий пул.

Параметры `lookAheadDistance` и `despawnDistance` являются важными для производительности и качества геймплея. Первый задаёт, насколько далеко вперед должна быть подготовлена трасса. Если значение слишком маленькое, игрок может увидеть появление объектов прямо перед собой. Если слишком большое, в сцене будет слишком много активных объектов. Второй параметр отвечает за удаление позади игрока. Слишком маленькое значение может привести к преждевременному исчезновению объектов у края экрана, слишком большое – к накоплению лишних объектов.

Оптимизация также касается самой модели движения. В проекте персонаж фиксируется по оси X, а мир смещается влево. Это уменьшает накопление больших координат и облегчает работу камеры: в LateUpdate камера стремится к фиксированной позиции `_cameraFixedPosition`. Из-за этого сцена не начинает дергаться от постоянного следования за быстро движущимся объектом. Решение довольно простое, но для автораннера оно практично.

Еще один аспект оптимизации – разделение визуального и логического обновления. `RunnerPresentation` отвечает за внешний вид персонажа: спрайты бега, подката, оттенки при эффектах и смену кадров. Он не начисляет очки и не принимает решения о поражении. `ParallaxLoopLayer` отдельно прокручивает фоновые слои и перераспределяет тайлы. За счет этого визуальные эффекты не смешиваются с ядром.

В системах ядра используются простые операции и компактные структуры данных. `ProgressionSystem` увеличивает скорость и сложность арифметически. `ScoreSystem` обновляет дистанцию и счет на основе `effectiveSpeed`. `PowerUpSystem` проходит список активных эффектов с конца к началу, что позволяет безопасно удалять истекшие эффекты. В `TrackGenerator` также используется обратный проход по спискам при переработке ушедших объектов. Это стандартный, но надёжный приём.

Оптимизация не должна ломать читаемость. В проекте не используется чрезмерно сложная микропроизводительность, например ручное управление массивами вместо списков во всех местах. Для текущего количества объектов `List` и `Stack` достаточно. Важно, что оптимизация выбрана точно: там, где постоянно появляются и исчезают объекты, используется пул. Там, где обновляется пара чисел, применяется обычная арифметика. И это нормальный баланс.

Для дальнейшего развития можно предложить несколько направлений. Во-первых, вынести конфигурации сегментов трассы в реальные `ScriptableObject`-ассеты, а не только создавать runtime-конфиги при отсутствии списка. Во-вторых, расширить систему пулов универсальным классом `ObjectPool<T>` или

использовать готовую реализацию `UnityEngine.Pool.ObjectPool<T>`, если версия проекта и стиль разработки это допускают. В-третьих, добавить профилирование через `Unity Profiler` и измерить фактическое количество активных объектов, вызовов GC и длительность кадра [26; 27].

Также можно оптимизировать ввод и UI. `HudController` обновляет текст каждый кадр. Для небольшого проекта это не проблема, но при расширении интерфейса полезно обновлять текст только при изменении значения или с небольшой частотой, например несколько раз в секунду для числовых показателей. Однако здесь важно не переусердствовать: очки и дистанция в автораннере воспринимаются как динамический показатель, поэтому слишком редкое обновление будет выглядеть грубо.

В таблице 2.7 указаны направления для оптимизации модульного ядра. На рисунке 2.17 показан оптимизационный код проекта возврата препятствий в пул.

Таблица 2.7 – Направления оптимизации модульного ядра

| Проблема                            | Реализованное решение                                        | Ожидаемый эффект                                      |
|-------------------------------------|--------------------------------------------------------------|-------------------------------------------------------|
| Частое создание объектов трассы     | Пулы <code>GameObject</code> в <code>TrackGenerator</code>   | Снижение затрат на создание и уничтожение объектов.   |
| Накопление объектов за экраном      | <code>RecycleOffscreen</code> и <code>despawnDistance</code> | Освобождение сцены от неактуальных объектов.          |
| Рывки камеры при движении персонажа | Фиксация <code>Runner</code> по X и прокрутка мира           | Более стабильная камера и упрощение координат.        |
| Рост сложности во времени           | <code>ProgressionSystem</code> с <code>Difficulty01</code>   | Плавное усложнение без ручной смены уровней.          |
| Временные бонусы                    | <code>PowerUpSystem</code> с таймерами активных эффектов     | Единое управление применением и завершением эффектов. |

Оптимизация генерации также связана с управлением сложностью. `TrackGenerator` получает значение `Difficulty01` от `ProgressionSystem` и выбирает конфигурации сегментов с учетом `DifficultyWeight`.

Чем выше сложность, тем плотнее могут размещаться препятствия и бонусы, а длина сегментов может немного отличаться. Это позволяет избежать

полностью статичной трассы. Игрок ощущает, что забег постепенно становится напряженнее, хотя структура уровней не задается вручную.

```
for (var i = _activeObstacles.Count - 1; i >= 0; i--)
{
    var obstacle = _activeObstacles[i];
    if (obstacle.GameObject == null)
    {
        _activeObstacles.RemoveAt(i);
        continue;
    }

    if (obstacle.GameObject.transform.position.x >= _runnerX - despawnDistance)
    {
        continue;
    }

    obstacle.GameObject.SetActive(false);
    GetObstaclePool(obstacle.Type).Push(obstacle.GameObject);
    _activeObstacles.RemoveAt(i);
}
```

Рисунок 2.17 – Принцип возврата препятствий в пул

Особый интерес представляет сочетание бонусов с прогрессией. SpeedBoostPowerUp увеличивает SpeedMultiplier, а значит влияет и на ощущаемую скорость, и на начисление дистанции. ScoreBoostPowerUp повышает множитель очков. ShieldPowerUp временно меняет правила столкновения. Все эти эффекты проходят через один GameContext, поэтому системы остаются согласованными. Если бы каждый бонус сам менял отдельные объекты сцены без общей модели, поведение стало бы труднее предсказать.

С точки зрения памяти проект использует Resources для загрузки спрайтов. Это удобно для прототипа, но в будущем стоит учитывать, что Resources может затруднять тонкое управление загрузкой и выгрузкой ассетов. Для более крупного проекта логично перейти к Addressables или явным ссылкам через инспектор.

Но для текущего объема игры Resources даёт простую и понятную схему: файл лежит в Resources/Art, библиотека загружает его по имени, презентационный слой назначает спрайт.

Подводя промежуточный итог, оптимизация ядра в проекте строится не вокруг одной большой технологии, а вокруг набора небольших решений: прокрутка мира вместо перемещения персонажа, повторное использование объектов, простые системы состояния, разделение визуального и логического кода, адаптивная подготовка трассы впереди игрока. Именно совокупность этих приёмов делает автораннер устойчивым.

## **2.7 Обоснование выбора стека технологий и инструментов разработки**

В качестве основной платформы разработки выбран Unity. Используется версия редактора Unity 6000.0.43f1, а в manifest.json подключены пакеты com.unity.feature.2d, Universal Render Pipeline 17.0.4, Input System 1.13.1 и uGUI 2.0.0. Такой стек соответствует задаче разработки 2D-автораннера: Unity предоставляет сцену, физику 2D, работу со спрайтами, систему UI и удобный цикл обновления MonoBehaviour [19; 22; 28].

Выбор Unity обоснован несколькими причинами. Во-первых, движок хорошо подходит для 2D-игр и содержит встроенные инструменты для спрайтов, 2D-физику и сцен. Во-вторых, Unity использует C#, что упрощает структурирование проекта через классы, интерфейсы и пространства имен. В-третьих, платформа позволяет быстро получить исполняемый прототип, настроить камеру, коллайдеры, Canvas и ресурсные ассеты без написания собственного низкоуровневого движка.

Язык C# выбран не только потому, что он является основным языком скриптов Unity. Он позволяет естественно выразить модульность проекта: интерфейсы InputSource, IPowerUp, IAbility; классы систем ScoreSystem и ProgressionSystem; статический класс GameSettings; строго типизированные

enum PickupType и ObstacleType. Код следует распространённым соглашениям именования: типы и публичные члены записаны в PascalCase, приватные поля часто имеют префикс подчёркивания. Подобные соглашения согласуются с практикой написания C#-скриптов для Unity и помогают сохранять единообразие игровой логики [10].

Universal Render Pipeline выбран как современный рендеринговый пайплайн Unity. Для данного проекта он не используется как сложная графическая система с множеством эффектов, однако обеспечивает нормальную основу для 2D-визуализации и дальнейшего расширения.

Пакет uGUI используется для интерфейса: Canvas, Button, Text, Slider, Image, EventSystem. Несмотря на появление UI Toolkit, uGUI остается удобным для быстрого создания игровых HUD и меню. В проекте это особенно видно: MainMenuController и HudController программно создают элементы интерфейса, настраивают CanvasScaler, подписываются на onClick и обновляют текстовые поля. Для небольшого 2D-автораннера такое решение вполне рационально [28].

Input System уже присутствует, но абстракция ввода пока реализована простыми средствами. В дальнейшей версии можно перевести InputSource на Input Actions, не меняя остальную архитектуру [22].

Для хранения простых данных используется PlayerPrefs. Через него сохраняются громкость, целевой FPS и лучший счет. Это оправдано, потому что данные малы и не требуют реляционной структуры. Использовать базу данных ради трех настроек было бы избыточно. При этом GameSettings изолирует работу с ключами PlayerPrefs, поэтому остальной код не рассыпает строковые ключи по всему проекту [25].

Редакторский скрипт ArtSpritePostprocessor автоматизирует импорт изображений из Assets/Resources/Art. Он устанавливает тип TextureImporterType.Sprite, одиночный режим спрайта, прозрачность alpha, отключает mipmap, задает Clamp, Bilinear и Uncompressed. Это полезная мелочь: художник или разработчик может добавить новый PNG в ресурсную папку, а проект автоматически применит нужные настройки. Вручную это легко забыть.

Инструментально проект можно сопровождать в Visual Studio или Rider. В manifest.json подключены com.unity.ide.visualstudio и com.unity.ide.rider, поэтому оба варианта поддерживаются. Для дипломной работы достаточно указать, что разработка ведется в среде Unity Editor с использованием C# IDE, а контроль результата осуществляется через запуск сцены RunnerScene и проверку основных сценариев.

Таким образом, стек технологий соответствует назначению проекта: Unity отвечает за игровую платформу, C# – за модульную архитектуру, URP – за современную 2D-визуализацию, uGUI – за меню и HUD, PlayerPrefs – за локальные настройки, а собственные классы ядра связывают эти возможности в законченную игровую систему. Стек не перегружен. И это хорошо.

В таблице 2.8 указаны инструменты и технологии проекта.

Таблица 2.8 – Стек технологий проекта

| Технология или инструмент | Использование в проекте                        | Обоснование выбора                                                   |
|---------------------------|------------------------------------------------|----------------------------------------------------------------------|
| Unity 6000.0.43f1         | Основная среда разработки и выполнения игры    | Поддержка 2D-сцен, физики, спрайтов, UI и кроссплатформенной сборки. |
| C#                        | Реализация игровой логики и модульного ядра    | Строгая типизация, классы, интерфейсы, удобная интеграция с Unity.   |
| URP 17.0.4                | Рендеринговая основа проекта                   | Актуальный пайплайн Unity и поддержка 2D Renderer.                   |
| uGUI 2.0.0                | Главное меню, настройки, HUD, панель поражения | Быстрое создание игровых интерфейсов и обработка кнопок/слайдеров.   |
| PlayerPrefs               | Настройки и лучший результат                   | Простое локальное хранение небольшого объема данных.                 |
| Resources/Art             | Загрузка спрайтов представления                | Удобная структура ресурсов для прототипа и учебного проекта.         |
| Visual Studio / Rider     | Редактирование C#-кода                         | Интеграция с Unity и поддержка навигации по проекту.                 |

При выборе инструментов важно учитывать не только возможности, но и учебную воспроизводимость. Unity и C# хорошо документированы, а структура проекта легко открывается проверяющим. Если бы в проекте использовался собственный движок, объём низкоуровневого кода резко вырос бы, но ценность

для задачи автораннера не обязательно стала бы выше. Если бы применялся сложный ECS-стек, пришлось бы отдельно объяснять технологию, которая для текущего масштаба не обязательна.

## 2.8 Практическая проверка проектных решений

После проектирования архитектуры важно проверить, насколько предложенная структура соответствует реальному игровому сценарию. В случае RusRunner базовая проверка может выполняться не только через запуск сцены, но и через последовательный анализ состояний. До начала забега `_isRunStarted` должен быть равен `false`, генератор трассы получает нулевую скорость, а `RunnerController` удерживает персонажа на земле. После вызова `BeginRun` контекст сбрасывается, скорость получает базовое значение, меню скрывается, а генератор начинает прокручивать мир. Такая проверка хорошо показывает, что состояние игры не «размазано» по интерфейсу, а контролируется ядром.

Для проверки подсистемы движения следует отдельно протестировать прыжок, подкат и дополнительный прыжок. Обычный прыжок должен выполняться только при заземлении или в пределах короткого `coyote time`. Подкат должен быть доступен только на земле и временно изменять `CapsuleCollider2D`. Дополнительный прыжок уменьшает `AirJumpsLeft` и не должен срабатывать бесконечно. Здесь важно проследить, что визуальная анимация не подменяет физическое правило: даже если спрайт выглядит как подкат, решающим остается состояние `GameContext` и параметры коллайдера.

Подсистема столкновений проверяется тремя сценариями. В первом игрок сталкивается с обычным препятствием без активного щита, и забег завершается. Во втором активен `ShieldPowerUp`, поэтому препятствие деактивируется, а `IsAlive` остается `true`. В третьем игрок проходит препятствие типа `Slide` в состоянии подката, и поражение также не наступает. Эти сценарии полезно вынести в таблицу тестирования, потому что они прямо подтверждают корректность правил, описанных в архитектуре.

На рисунке 2.18 изображена схема проверки жизненного цикла забега.

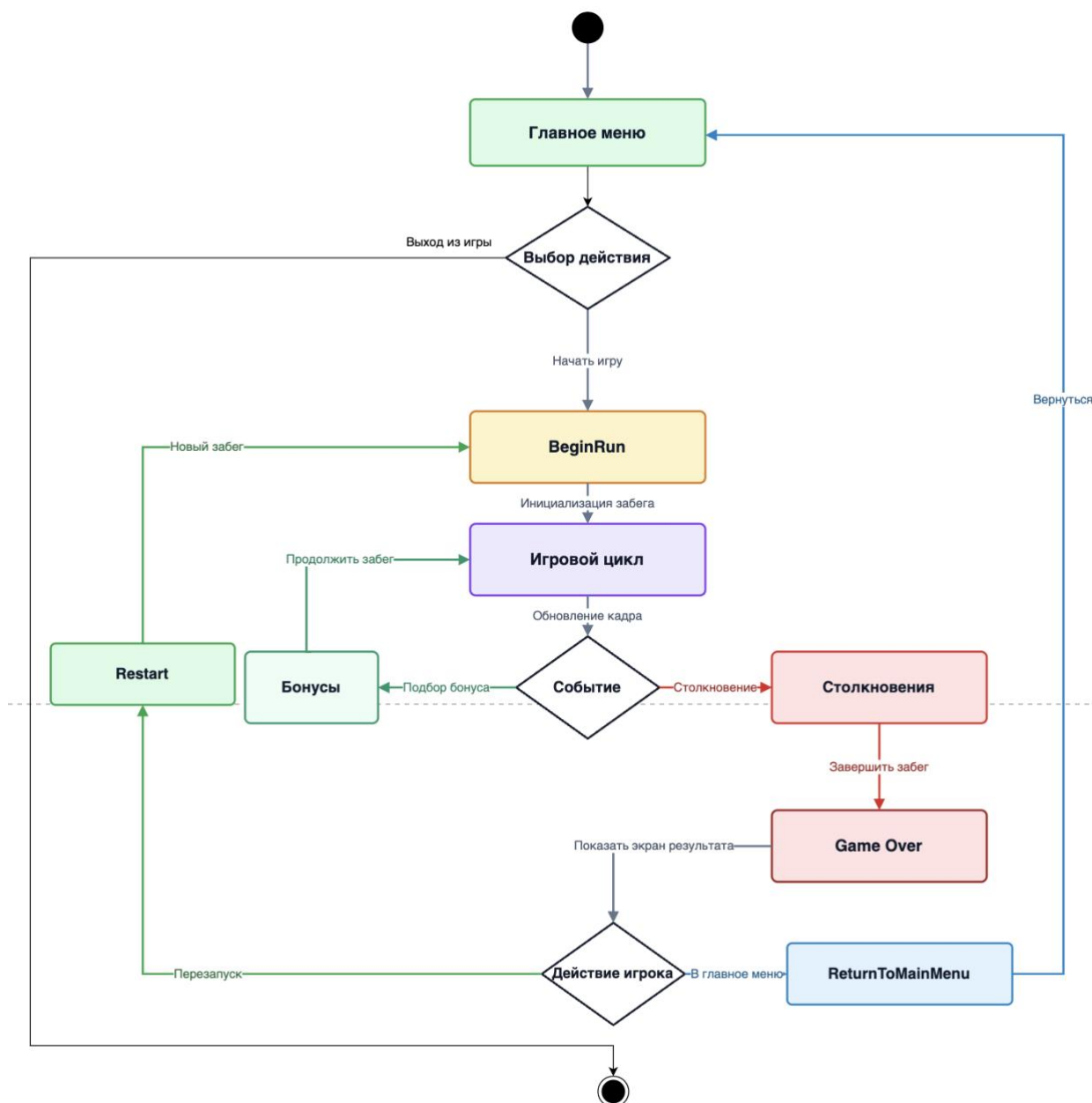


Рисунок 2.18 – Схема проверки жизненного цикла забега

Проверка генерации трассы должна включать наблюдение за количеством активных объектов во время длительного забега. Оно не должно расти бесконечно. Сегменты земли, препятствия и бонусы после ухода за экран должны исчезать из активных списков и возвращаться в пулы. Если при длительном запуске количество объектов в иерархии Unity постоянно увеличивается, значит, механизм переработки объектов работает неверно.

Пользовательский интерфейс также нуждается в проверке. Главное меню должно отображаться при первом запуске сцены, кнопка запуска должна переводить игру в состояние забега, экран настроек обязан сохранять значения через GameSettings, а HUD должен появляться только во время активной сессии. После проигрыша панель Game Over должна показать очки, рекорд и дистанцию.

## **2.9 Выводы по разделу**

Во второй главе было выполнено проектирование модульного ядра 2D-автораннера RusRunner на основе анализа исходного кода Unity-проекта. Установлено, что центральным элементом архитектуры является RunnerBootstrap, который координирует игровой цикл, запуск и перезапуск забега, обработку ввода, реакцию на столкновения, применение бонусов и связь с генератором трассы. При этом состояние текущей игровой сессии вынесено в GameContext, а отдельные аспекты логики реализованы самостоятельными системами: ScoreSystem, ProgressionSystem, PowerUpSystem и AbilitySystem.

В главе обоснован выбор компонентно-модульного архитектурного стиля. Он соответствует модели Unity, но не ограничивается механическим размещением скриптов на объектах сцены. Логика проекта разделена на ядро, игровой слой, ввод, представление и пользовательский интерфейс. Такое разделение повышает сопровождаемость и создает основу для дальнейшего расширения: добавления новых бонусов, способов ввода, типов препятствий и вариантов визуального оформления.

Архитектура была описана с использованием подхода C4. Для проекта предложены диаграмма контекста, диаграмма контейнеров и диаграмма компонентов игрового ядра. Эти схемы позволяют показать систему на разных уровнях детализации: от взаимодействия игрока с приложением до связей между RunnerBootstrap, TrackGenerator, RunnerController, GameContext, системами ядра и UI.

Отдельно рассмотрено проектирование пользовательского интерфейса. Выделены основные экранные формы: главное меню, экран настроек, игровой HUD и панель поражения. Показано, что интерфейс не хранит собственную игровую модель, а отображает состояние, предоставляемое RunnerBootstrap. Такой подход снижает риск рассинхронизации данных и делает UI зависимым от публичного фасада ядра, а не от внутренних деталей реализации.

В разделе детального проектирования описаны ключевые классы и сценарии взаимодействия. Для дальнейшего оформления работы предложены UML-диаграммы классов, диаграммы последовательности запуска забега, обработки прыжка, подбора бонуса, столкновения с препятствием и одного кадра игрового цикла. Рассмотрено применение шаблонов Facade, Object Pool, Strategy, Effect Object и State Object.

Особое внимание уделено оптимизации генерации объектов. В TrackGenerator используется повторное применение GameObject через пулы для сегментов земли, препятствий и бонусов. Это решение снижает количество операций создания и уничтожения объектов во время забега и лучше соответствует динамике автораннера. Кроме того, проект использует прокрутку мира при фиксированной позиции персонажа, что упрощает работу камеры и переработку объектов за пределами экрана.

В завершение обоснован выбранный стек технологий: Unity 6000.0.43f1, C#, Universal Render Pipeline, uGUI, PlayerPrefs, Resources/Art и IDE Visual Studio/Rider. Выбранные средства соответствуют задаче разработки 2D-игры, позволяют быстро получить работающий прототип и при этом поддерживают модульную структуру кода. Подводя итог, можно сказать, что ядро проекта RusRunner спроектировано как практичная, расширяемая и достаточно оптимизированная основа для 2D-автораннера.

### **3 РЕАЛИЗАЦИЯ И БАЛАНСИРОВКА ИГРОВОГО ПРОЦЕССА В ЛИНЕЙНОМ 2D-АВТОРАННЕРЕ**

#### **3.1 Организация процесса разработки**

##### **3.1.1 Выбор и обоснование методологии разработки**

Для проекта выбрана итерационно-инкрементная методология с короткими циклами проверки. Жёсткая каскадная схема здесь была бы неудобна: ощущение управления нельзя полностью определить на бумаге, поскольку оно зависит от сочетания силы прыжка, гравитации, размеров коллайдера, скорости прокрутки мира и визуальной анимации. Числовые параметры уточнялись после запуска сцены, то есть реализация и проверка неизбежно чередовались [1].

Полный Scrum для небольшой команды оказался бы избыточным. В проекте нет необходимости вводить отдельные роли владельца продукта и Scrum-мастера. При этом из гибких подходов были сохранены полезные элементы: ограниченный набор задач на итерацию, проверяемый результат, приоритизация дефектов и фиксация критерия готовности.

Инкрементом считалось не количество написанных строк, а завершённый фрагмент поведения. Например, задача «реализовать подкат» включала переход в состояние `IsSliding`, изменение `CapsuleCollider2D`, ограничение действия по времени, корректное прохождение под высоким препятствием и восстановление коллайдера. Пока один из этих пунктов не выполнялся, механика не считалась готовой.

Работа была разбита на четыре этапа. На первом стабилизировались запуск и завершение игровой сессии. На втором отрабатывались прыжок, подкат и дополнительный прыжок. Третий этап был связан с препятствиями, бонусами и счетом. На четвертом уточнялись интерфейс, параметры сложности и поведение при многократном рестарте. Последовательность не являлась абсолютно линейной: обнаруженный дефект мог вернуть работу к предыдущему блоку, и это нормально для игрового прототипа.

Критерий готовности задачи формулировался наблюдаемым способом. Механика считалась завершенной, когда:

- она воспроизводимо работала в начальной фазе забега, после роста скорости и после перезапуска сцены;
- не создавала неразрешимых комбинаций;
- не оставляла персонажа или интерфейс в промежуточном состоянии.

### **3.1.2 Организация репозитория и системы контроля версий**

Система контроля версий необходима для сохранения истории изменений C#-кода, сцен, настроек проекта и метаданных Unity. Для RusRunner целесообразно применять Git с удаленным репозиторием GitHub. Ветвление позволяет изолировать работу над механикой до ее проверки, а pull request обеспечивает точку просмотра различий и фиксации способа тестирования. Базовые операции с ветками в Git создают независимые указатели на цепочки коммитов и поэтому подходят для параллельной разработки [11].

Основной веткой является main, содержащая проверяемое состояние проекта. Для отдельных задач создаются короткоживущие ветки. После проверки изменения объединяются через pull request. Долгоживущая ветка develop для проекта такого масштаба не обязательна: она увеличивает число точек интеграции, но не дает заметного выигрыша небольшой команде.

В репозиторий Unity обычно включаются Assets, Packages и ProjectSettings. Каталоги Library, Temp, Logs, Obj, Build и пользовательские настройки IDE исключаются через .gitignore, поскольку они пересоздаются и занимают значительный объем. Для проекта особенно важно хранить meta-файлы вместе с ассетами: Unity использует GUID из этих файлов для ссылок между сценами, скриптами и ресурсами. Потеря meta может визуально оставить файл на месте, но разорвать ссылку в сцене [20].

Особое внимание необходимо уделять конфликтам сцен. Два разработчика, одновременно изменяющие RunnerScene.unity, могут получить

сложное текстовое слияние YAML. Поэтому задачи желательно распределять так, чтобы один участник менял сцену, а другой работал с самостоятельными C#-файлами или ресурсами.

Для файлов сцен и настроек полезно включить Force Text и Visible Meta Files. Текстовая сериализация не делает слияние сцены полностью безопасным, но позволяет увидеть изменения в diff. Дополнительно стоит договориться, что один участник не перестраивает RunnerScene одновременно с другим без предварительного разделения объектов на префабы. Это простое организационное правило экономит время [21].

## **3.2 Соответствие реализации спроектированной архитектуре**

### **3.2.1 Сопоставление компонентов архитектуры и модулей реализации**

Центральная роль RunnerBootstrap проявляется в последовательности Update. До запуска он останавливает генератор и удерживает персонажа на исходной позиции. После поражения прекращает прокрутку и обрабатывает только команду рестарта. В активном состоянии сначала обновляется контроллер персонажа, затем ввод, прогрессия, бонусы и счет, после чего новое значение сложности и скорости передается генератору. Такая последовательность соответствует роли координатора, описанной в архитектуре.

GameContext используется как единый источник изменяемого состояния. ProgressionSystem записывает CurrentSpeed, ScoreSystem обновляет DistanceMeters и Score, PowerUpSystem меняет множители и признак неуязвимости, RunnerController устанавливает IsGrounded и IsSliding. UI не изменяет эти поля напрямую, а читает свойства RunnerBootstrap. В результате риск рассинхронизации снижается.

TrackGenerator не принимает решений о поражении и не начисляет очки. Он получает только текущую сложность и скорость прокрутки, создает объекты, перемещает их и возвращает в пулы. Объект Obstacle передает событие

столкновения через RunnerCollisionRelay в координатор. Подобное разделение позволяет изменять генератор без вмешательства в систему счета.

С точки зрения архитектуры наиболее заметное отклонение связано с вводом и способностями. Интерфейс InputSource является корректной стратегией, но RunnerBootstrap отдельно проверяет клавишу Q и R через UnityEngine.Input. В результате часть команд проходит через абстракцию, а часть остается жестко связанной с клавиатурой.

Связь реализованных модулей отображено на рисунке 3.1.

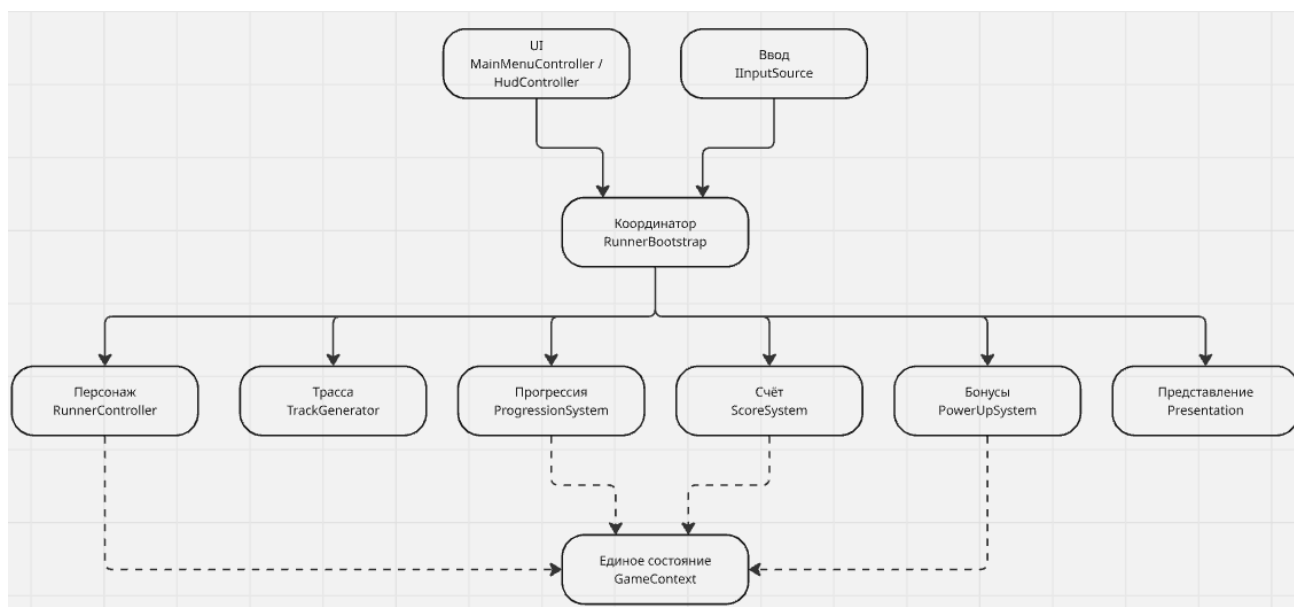


Рисунок 3.1 – Связь реализованных модулей с координатором и единым состоянием

### 3.2.2 Организация файловой структуры проекта

Файловая структура сочетает скрипты в корне Assets и модульное разбиение в Assets/Scripts. RunnerBootstrap, RunnerController, TrackGenerator, HudController и MainMenuController расположены непосредственно в Assets. Ядро, контракты, ввод, бонусы и представление собраны в иерархии Scripts.

В Assets/Scripts/Core размещены GameContext и GameSettings, а в подпапке Systems – ProgressionSystem, ScoreSystem, PowerUpSystem и AbilitySystem. Каталог Game содержит игровые сущности и контракты, Input – реализации источников ввода, PowerUps и Abilities – расширяемые эффекты, Presentation –

визуальные адаптеры. Пространства имен RusRunner.Core, RusRunner.Core.Systems, RusRunner.Game, RusRunner.Game.Input и RusRunner.Game.Presentation в основном повторяют эту структуру.

Сцены расположены в Assets/Scenes. Главная рабочая сцена называется RunnerScene.unity. В Assets/Resources/Art находятся спрайты окружения, персонажа, препятствий и бонусов; доступ к ним выполняется через ArtSpriteLibrary и стандартный механизм Resources. Каталог Editor содержит ArtSpritePostprocessor, который применяется только в редакторе и настраивает импорт графики.

Файловая структура игры RusRunner отображено на рисунке 3.2.

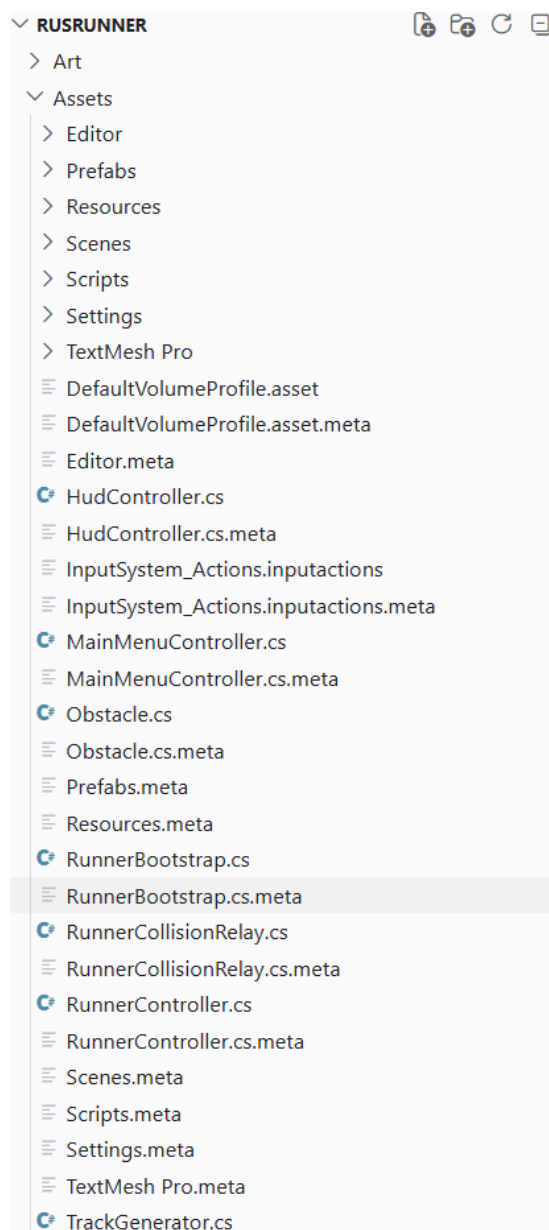


Рисунок 3.2 – Файловая структура Unity-проекта RusRunner

### **3.2.3 Конфигурационные файлы и управление настройками Unity**

Версия редактора зафиксирована в ProjectVersion.txt: Unity 6000.0.43f1. Это важно для воспроизводимости, поскольку сериализация сцен, набор встроенных пакетов и поведение импортера могут отличаться между версиями. В manifest.json указаны Universal Render Pipeline 17.0.4, Input System 1.13.1, uGUI 2.0.0 и Test Framework 1.4.6, а также стандартные 2D-модули.

В PlayerSettings установлено исходное разрешение 1920×1080. CanvasScaler в программно создаваемом интерфейсе использует это разрешение как referenceResolution и режим ScaleWithScreenSize.

GameSettings инкапсулирует пользовательские параметры. Громкость музыки, громкость эффектов и целевая частота кадров сохраняются через PlayerPrefs. Допустимые значения FPS нормализуются до 60, 120 или 180.

## **3.3 Детализация ключевых компонентов**

### **3.3.1 Реализация управления персонажем**

Персонаж реализован объектом Runner с тегом Player. На нем находятся Rigidbody2D, CapsuleCollider2D, RunnerController и RunnerCollisionRelay. Rigidbody2D имеет массу 1, интерполяцию и непрерывный режим обнаружения столкновений. Последние два параметра дополнительно задаются в Awake, что снижает риск пропуска контакта при росте скорости мира [23].

В отличие от классического платформера Runner почти не перемещается по горизонтали. Метод MoveForward принимает скорость, но фактически обнуляет горизонтальную составляющую velocity. Сам персонаж удерживается около фиксированной координаты X, а земля, препятствия и бонусы движутся влево. Благодаря этому камера остается стабильной, а расчет удаления объектов опирается на ограниченный диапазон координат.

Обычный прыжок выполняется методом TryGroundJump. Действие разрешено, когда персонаж находится на земле либо не истек coyote time. Coyote time смягчает границу платформы: игрок может нажать прыжок через небольшой

промежуток после того, как проверка земли перестала срабатывать. Для автораннера это повышает ощущение честности. Интервал невелик и почти не замечается визуально, зато компенсирует один-два поздних кадра реакции.

После наземного прыжка `GameContext.AirJumpsLeft` устанавливается в 1. Дополнительный прыжок вызывается клавишей Q через `TryAirJump`, доступен только в воздухе и уменьшает счетчик. После приземления дополнительный прыжок возвращается к 1 при следующем наземном прыжке. Так устраняется вариант с неограниченными дополнительными прыжками.

Листинг кода ограничения дополнительного прыжка приведён в приложении Б.1.

При подкате на время `slideDuration` `GameContext.IsSliding` становится истинным, а `CapsuleCollider2D` меняется с обычного размера на уменьшенный. По завершении таймера исходный размер и `offset` восстанавливаются. Подкат запускается только на земле.

Подкат читается через `GetKey`, то есть удержание кнопки S или стрелки вниз способно повторно продлевать действие после окончания короткого таймера, что позволяет удерживать персонажа в таком положении столько, сколько необходимо игроку для преодоления препятствия.

На рисунках 3.3–3.5 отображены различные состояния персонажа.

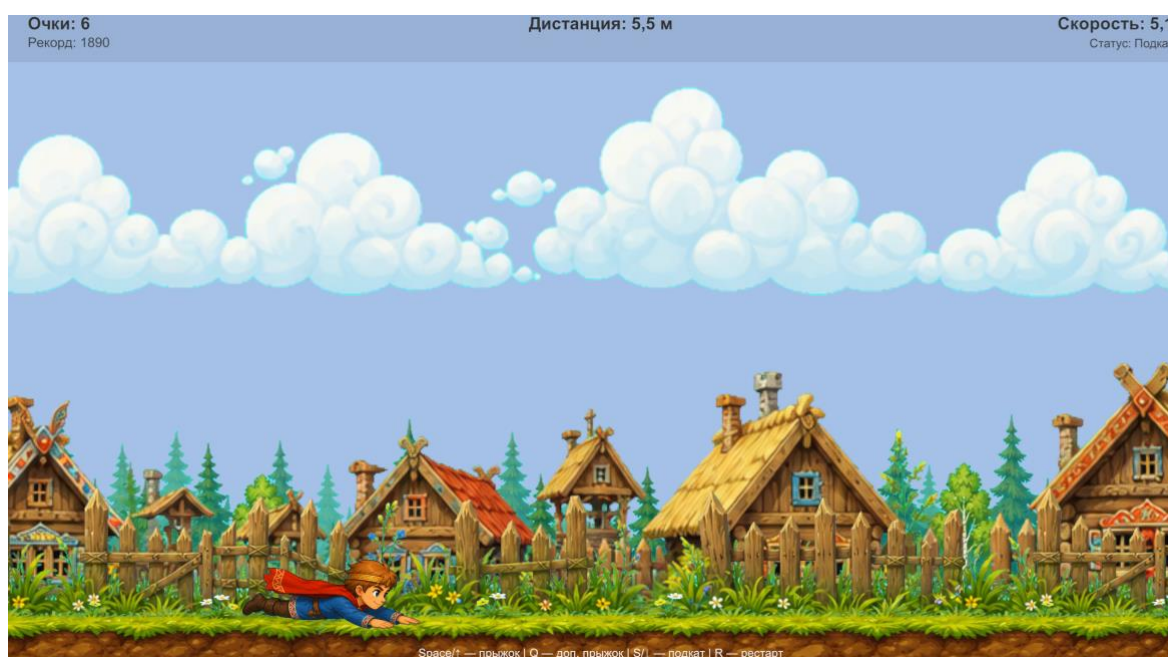


Рисунок 3.3 – Подкат

Метод `Halt` останавливает `Rigidbody2D` после поражения, а `RestoreAfterMenuOrRestart` возвращает динамическое состояние и коллайдер. Эти методы важны для повторной попытки. Без явного восстановления персонаж мог бы начать новый забег с уменьшенной капсулой или остаточной вертикальной скоростью.

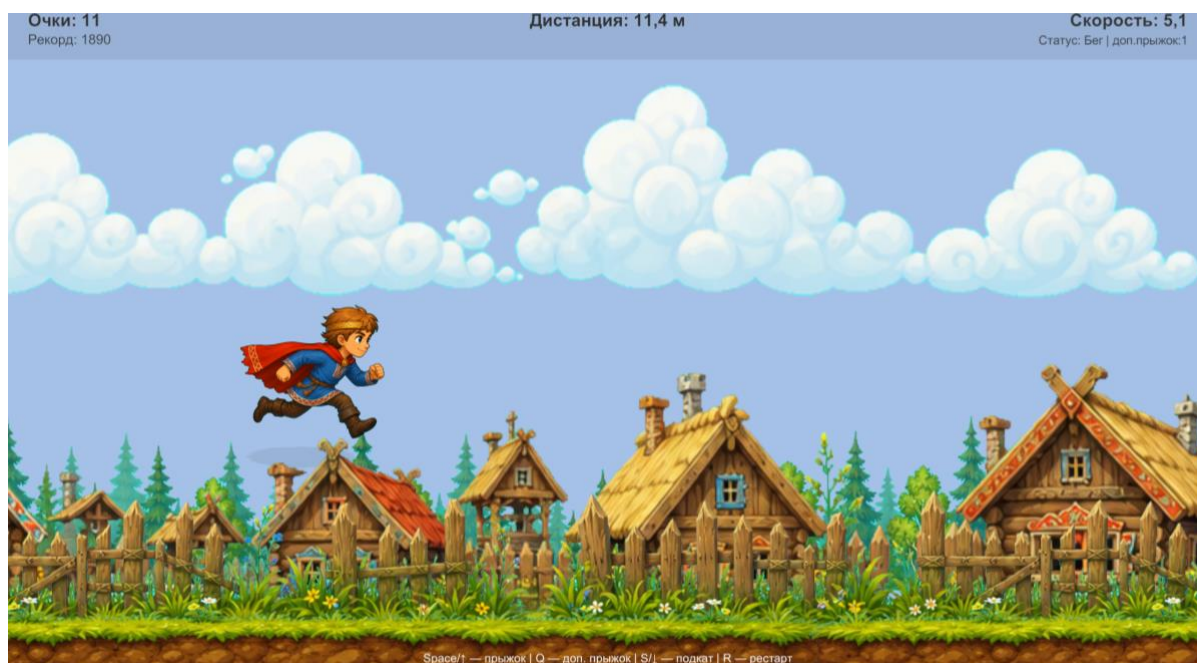


Рисунок 3.4 – Прыжок

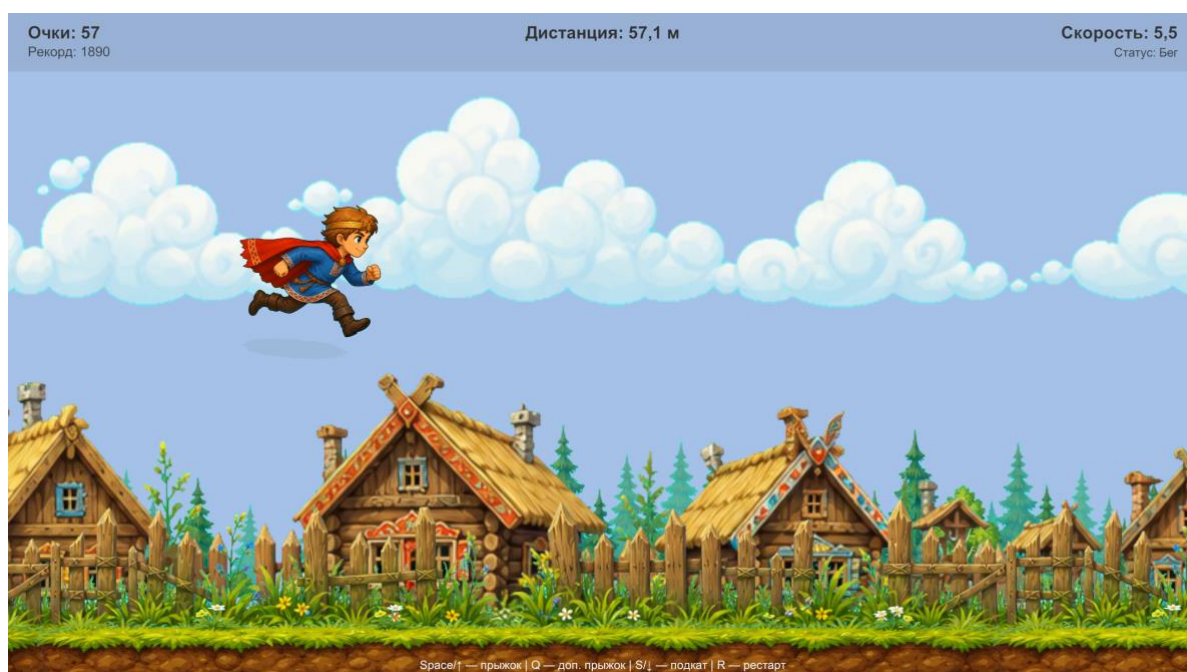


Рисунок 3.5 – Дополнительный прыжок

### 3.3.2 Реализация системы препятствий

Система препятствий построена вокруг TrackGenerator. Генератор создаёт последовательность сегментов земли, поддерживает достаточный запас трассы впереди камеры и перемещает активные объекты влево с текущей скоростью. Когда элемент уходит за предел despawnDistance, он деактивируется и возвращается в свой пул. В результате игровой мир выглядит бесконечным, хотя в памяти повторно используется ограниченное число экземпляров.

В проекте предусмотрены три типа препятствий: Jump, Slide и Air. Jump размещается около поверхности и требует прыжка. Slide представляет собой вытянутую горизонтальную преграду на высоте, под которой нужно пройти с помощью подката. Air находится в воздухе и формирует ситуацию, в которой лишний высокий прыжок может быть нежелателен; в зависимости от расположения игрок проходит под ним либо корректирует траекторию с помощью дополнительного прыжка.

Габариты препятствий задаются при создании объектов.

Вероятность типа препятствия выбирается случайно с преобладанием наземных препятствий, так как обычный прыжок является базовым действием. Подкат встречается реже, а воздушная преграда добавляет вариативность и не должна доминировать.

Каждый сегмент имеет длину и плотность препятствий. При отсутствии подготовленных ScriptableObject-конфигураций TrackGenerator формирует четыре runtime-профиля: VillageEasy, ForestMid, OutpostHard и Rush. Профиль выбирается по минимальному расстоянию между текущим Difficulty01 и весом.

Само столкновение передаётся через RunnerCollisionRelay или Obstacle в RunnerBootstrap.HandleObstacleHit. Сначала проверяется, начат ли забег и жив ли персонаж. Затем учитывается щит. После этого для типа препятствия Slide проверяется IsSliding. Только если защитные условия не сработали, GameContext.IsAlive становится false, RunnerController останавливается, скорость генератора обнуляется, а ScoreSystem фиксирует возможный рекорд.

На рисунках 3.6–3.8 отображены типы препятствий в игре.

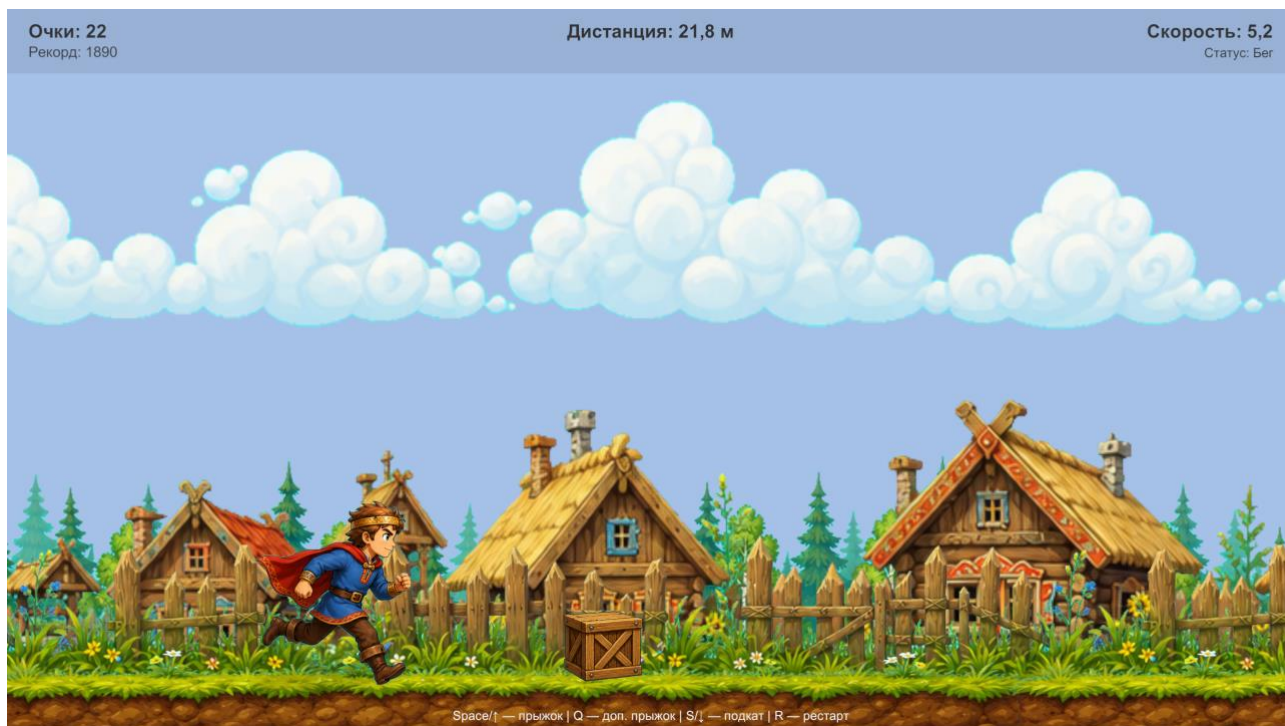


Рисунок 3.6 – Наземное препятствие

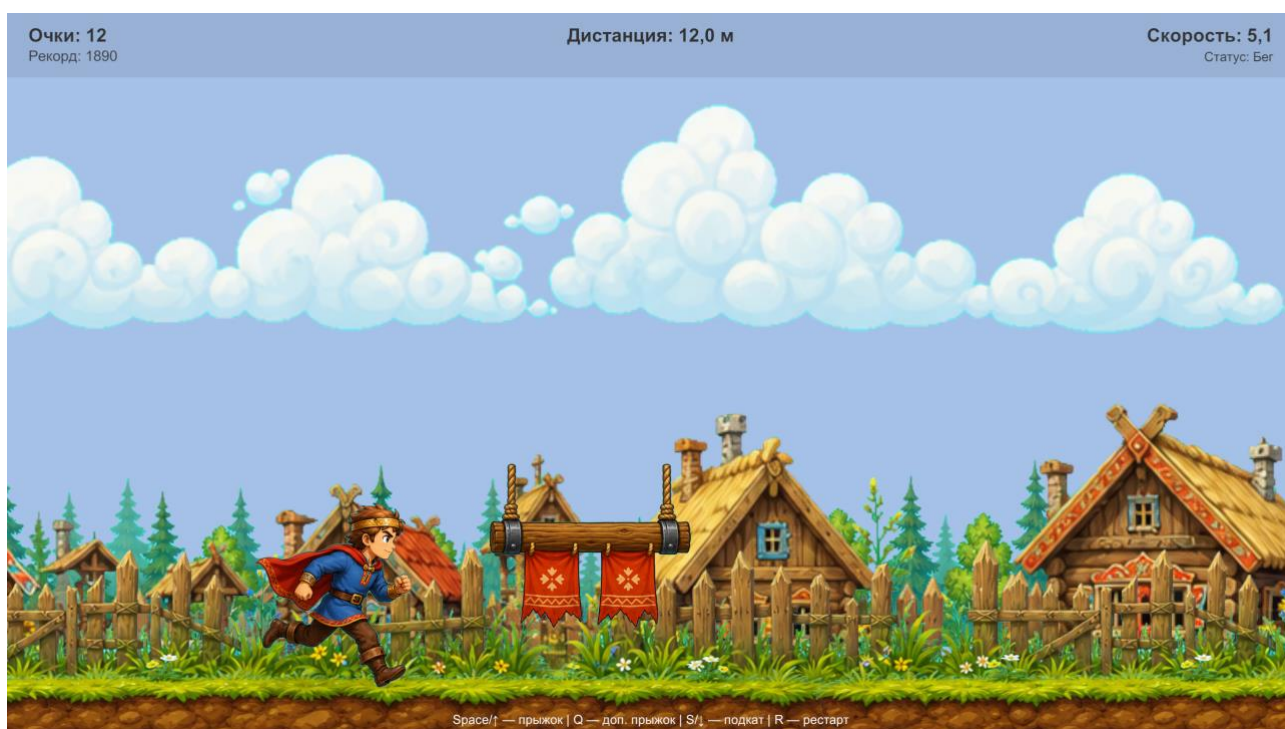


Рисунок 3.7 – Препятствие для подката

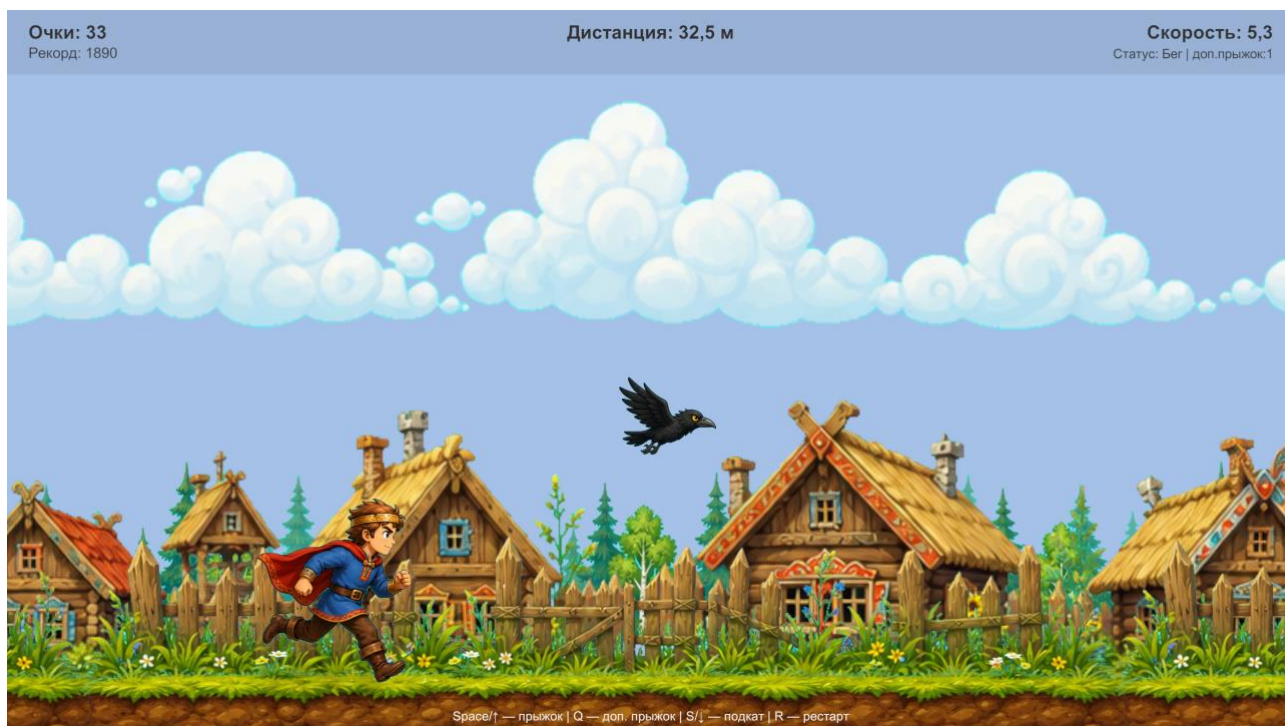


Рисунок 3.8 – Воздушное препятствие

### 3.3.3 Реализация бонусов и временных эффектов

Бонусы представлены объектами `PowerUpPickup` и классами, реализующими `IPowerUp`. При входе персонажа в триггер `pickup` передает тип в `RunnerBootstrap`. Координатор создает нужный эффект и добавляет его в `PowerUpSystem`, после чего объект подбора выключается. Система вызывает `Apply`, хранит оставшееся время, выполняет `Tick` и завершает эффект через `Expire`.

Бонус «Щит» устанавливает `IsInvulnerable`. При контакте с препятствием оно деактивируется, а забег продолжается. Ускорение увеличивает `SpeedMultiplier`.

Бонус «Удвоение очков» увеличивает `ScoreMultiplier`, превращая стандартный множитель 1 в значение 2.

Бонус «Ускорение» увеличивает `SpeedMultiplier` на 0,25, поэтому эффективная скорость становится равной 125% базовой скорости.

Устойчивое решение для бонусов – повторный подбор бонуса обновляет таймер. Листинг кода обновления таймеров активных бонусов представлен в

приложении Б.2. Обратный проход по списку позволяет безопасно удалять завершённые элементы. Если двигаться от начала к концу, удаление сдвигает индексы и следующий эффект может быть пропущен.

Типы бонусов в игре отображены на рисунках 3.9–3.11.

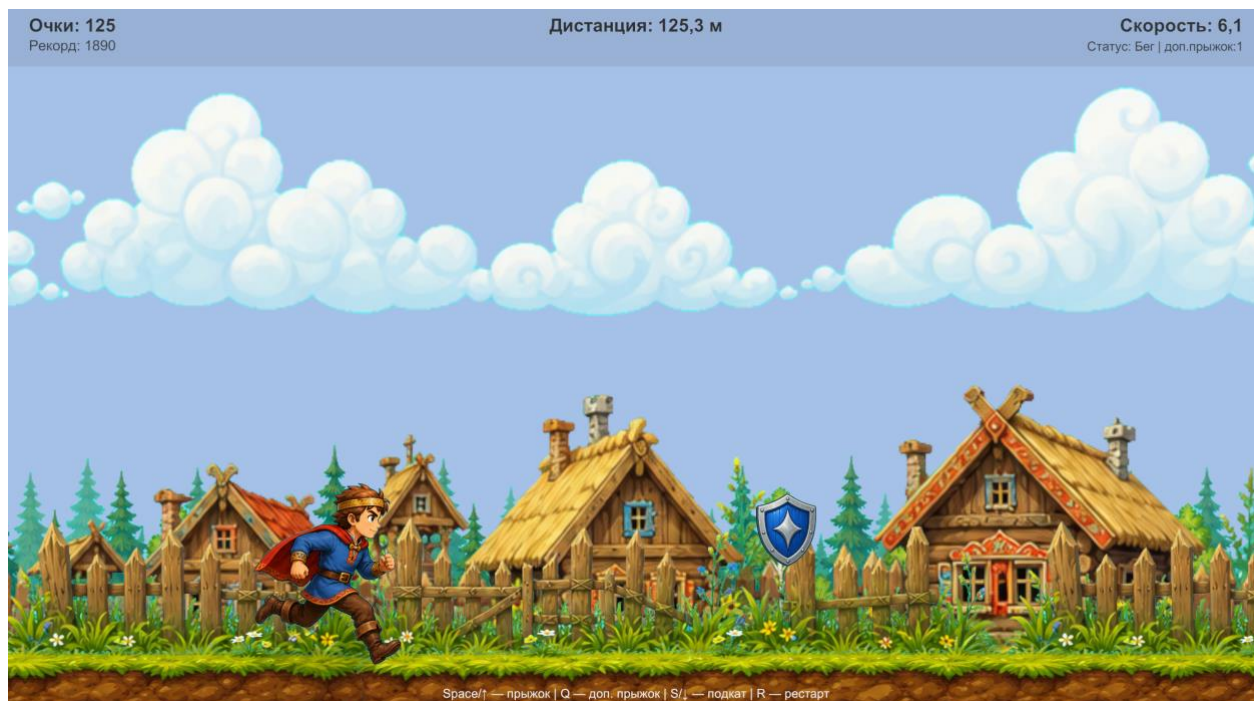


Рисунок 3.9 – Бонус «Щит»

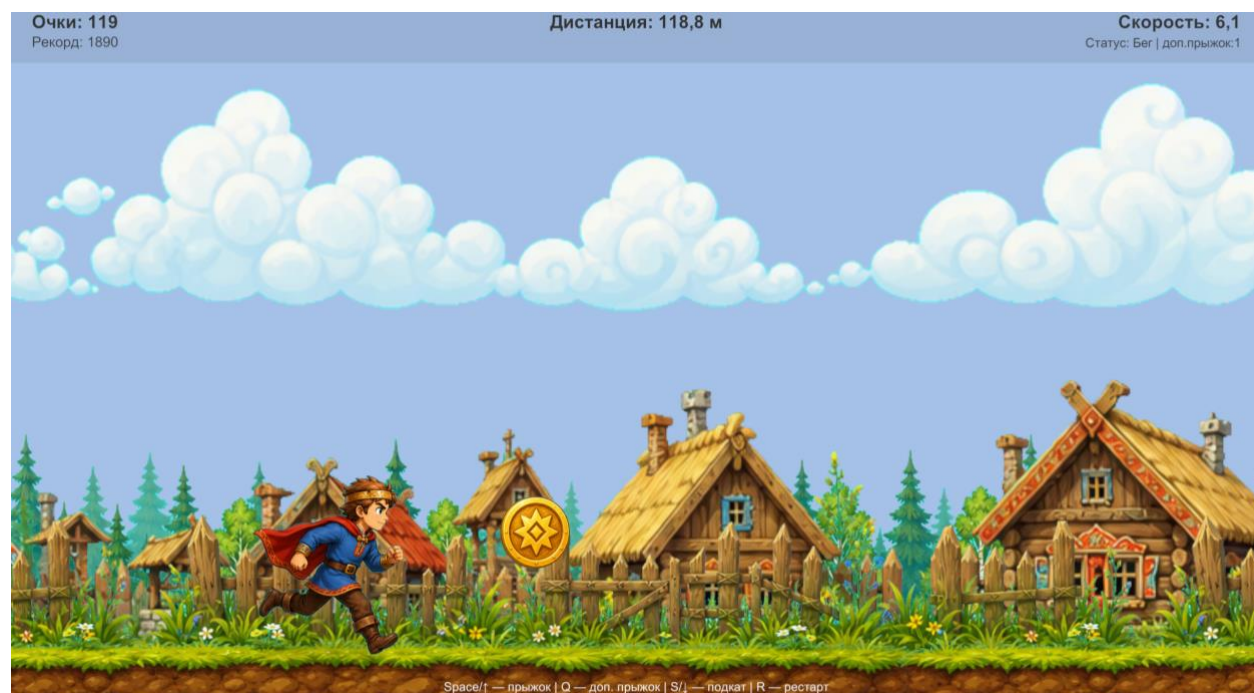


Рисунок 3.10 – Бонус «Удвоение очков»



Рисунок 3.11 – Бонус «Ускорение»

### 3.3.4 Реализация системы очков, дистанции и рекорд

Расчет результата вынесен в ScoreSystem. На каждом активном кадре система получает GameContext и deltaTime, определяет effectiveSpeed как произведение CurrentSpeed и SpeedMultiplier, увеличивает пройденную дистанцию, затем начисляет очки с учетом scorePerMeter и ScoreMultiplier. При стандартном множителе одно очко соответствует одной условной единице дистанции.

Формула связывает результат с реальным темпом мира. Если активирован SpeedBoost, игрок быстрее проходит условную дистанцию и одновременно получает больше очков за секунду. ScoreBoost влияет только на очки. Такое различие читаемо: ускорение повышает риск и темп, множитель счета награждает успешное прохождение без изменения сложности.

Лучший результат загружается из PlayerPrefs по ключу rusrunner\_best\_score. Сохранение выполняется в FinalizeRun только тогда, когда итоговый счет превысил прежний рекорд. Это уменьшает число записей и делает

момент фиксации понятным: рекорд обновляется при завершении попытки, а не на каждом кадре.

После начала новой сессии текущие значения сбрасываются через `GameContext.Reset`, тогда как `BestScore` остается в `ScoreSystem`. HUD поэтому может одновременно показывать текущий счет и ранее достигнутый рекорд. При поражении итоговая панель использует те же публичные свойства `RunnerBootstrap`.

`PlayerPrefs` подходит для одного числового рекорда. Однако этот механизм не предназначен для защищенного хранения и легко изменяется пользователем. Если появятся таблицы лидеров, профили или проверяемые достижения, потребуется иной слой хранения.

### **3.3.5 Реализация пользовательского интерфейса игрового процесса**

Пользовательский интерфейс строится программно классами `MainMenuController` и `HudController`. При отсутствии нужных объектов они создают `Canvas`, `CanvasScaler`, панели, кнопки, текстовые поля, слайдеры и `EventSystem`.

Главное меню содержит запуск, настройки и выход. Экран настроек предоставляет два слайдера громкости и кнопки выбора 60, 120 или 180 FPS. После сохранения `GameSettings` записывает значения в `PlayerPrefs` и применяет целевую частоту кадров. Меню скрывается при начале забега, не требуя перехода в отдельную сцену.

HUD отображает текущие очки, рекорд, дистанцию, скорость и строку состояния. Строка сообщает о подкате, щите, множителе счета и оставшемся дополнительном прыжке. Такая обратная связь нужна для временных эффектов: без неё игрок не понимает, закончился ли щит и почему препятствия снова стали опасными.

После поражения появляется окно поражения с итоговым счётом, рекордом, дистанцией и действиями запустить снова игру или выйти в главное меню. Дополнительно поддерживается клавиша `R`. `RestartRun` устанавливает

статический флаг немедленного старта и перезагружает активную сцену. `ReturnToMainMenu` очищает флаг и также перезагружает сцену. Перезагрузка даёт надёжный полный сброс, хотя обходится дороже ручного восстановления состояния.

Реализованный HUD отображён на рисунках 3.12–3.13.

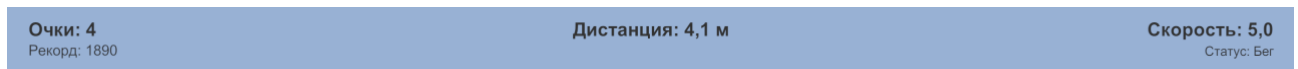


Рисунок 3.12 – HUD статистики и состояний



Рисунок 3.13 – HUD подсказок по управлению персонажем

## 3.4 Применение принципов качественного кода

### 3.4.1 Принципы SOLID в реализации

Принцип единственной ответственности наиболее явно соблюдается в системах ядра. `ScoreSystem` отвечает за начисление и сохранение результата, `ProgressionSystem` – за рост скорости и нормализованной сложности, `PowerUpSystem` – за жизненный цикл временных эффектов. `RunnerController` сосредоточен на физике персонажа. Такое разделение позволяет менять формулу счета, не затрагивая `raycast` земли.

Принцип открытости и закрытости поддерживается интерфейсом `IPowerUp`. Новый временный эффект добавляется отдельным классом с методами `Apply`, `Tick` и `Expire`. `PowerUpSystem` при этом не меняется.

Принцип подстановки Лисков соблюдается для реализаций `IPowerUp` и `InputSource` в той мере, в какой координатор использует их через общий контракт. Любой источник ввода обязан предоставить `JumpPressed`, `SlidePressed` и `DashPressed`, а эффект – корректно примениться и отмениться.

Принцип разделения интерфейсов выражен компактными контрактами. `IPowerUp` не содержит методов UI или загрузки ресурсов, `InputSource` не знает

о Rigidbody2D. IAbility также отделен от временного бонуса: способность активируется по команде, а power-up приходит из мира и действует ограниченное время.

В таблице 3.1 кратко указаны реализованные в проекте принципы.

Таблица 3.1 – Применение SOLID

| Принцип | Реализация в проекте                                                                       |
|---------|--------------------------------------------------------------------------------------------|
| SRP     | ScoreSystem, ProgressionSystem, PowerUpSystem и RunnerController разделены по обязанностям |
| OCP     | IPowerUp допускает новые эффекты. Новые IPowerUp обрабатываются общей системой таймеров    |
| LSP     | KeyboardInputSource реализует IInputSource                                                 |
| ISP     | Интерфейсы ввода, способности и бонуса компактны                                           |
| DIP     | Основной цикл читает ввод через абстракцию                                                 |

### 3.4.2 Принципы чистого кода

Имена большинства классов и методов передают назначение без дополнительного комментария: TryGroundJump, TryAirJump, RecycleOffscreen, FinalizeRun, SetScrollSpeed. Префикс Try указывает, что действие может не выполниться. Методы Apply и Expire образуют понятную пару. Такая лексика снижает время чтения.

Методы систем небольшие и в основном находятся на одном уровне абстракции. ProgressionSystem.Tick не обращается к UI, ScoreSystem.FinalizeRun не двигает персонажа. В TrackGenerator код сложнее из-за построения объектов и пулов, однако обязанности разделены между вспомогательными методами. Это лучше большого Update, содержащего генерацию, выбор спрайта, перемещение и очистку одновременно.

Значения, влияющие на баланс, преимущественно вынесены в сериализованные поля: jumpForce, slideDuration, baseSpeed, speedGainPerSecond, density и lookAheadDistance. Разработчик может менять их в Inspector, не редактируя алгоритм.

GameSettings скрывает строковые ключи PlayerPrefs и нормализацию FPS. Это предотвращает расхождение имён ключей в разных контроллерах. ScoreSystem также хранит ключ рекорда внутри своего модуля. Константы не

распространяются по проекту, что уменьшает вероятность опечатки. В коде есть защитные проверки ссылок и `AutoFindReferences`.

### 3.5 Интеграция с библиотеками и пакетами Unity

Universal Render Pipeline версии 17.0.4 обеспечивает рендеринговую основу. Для автораннера не применяются сложные постэффекты, однако URP дает современную конфигурацию проекта и поддержку 2D Renderer. Визуальные слои дополнительно организованы программно: фон и облака движутся с разными коэффициентами в `ParallaxLoopLayer` [19].

Пакет `uGUI` 2.0.0 используется напрямую. `MainMenuController` и `HudController` создают `Canvas`, `Image`, `Button`, `Slider` и `Text`, подписываются на события `onClick` и `onValueChanged`. Интеграция не требует внешней разметки, но тесно связывает код контроллера с типами `UnityEngine.UI` [28].

`PlayerPrefs` применяется как встроенный механизм локального хранения. `GameSettings` сохраняет три параметра, `ScoreSystem` – рекорд. После изменения настроек вызывается `PlayerPrefs.Save`. Для небольшого объема данных интеграция проста и достаточна, но она не обеспечивает шифрование или сложную структуру профиля [25].

`Input System 1.13.1` подключен, однако актуальная логика опирается на старый `UnityEngine.Input`, так как на данный момент реализовано управление только через клавиатуру и мышь. Такое состояние допустимо на переходном этапе, поскольку `Active Input Handling` установлен в `Both`. В дальнейшем при разработке управления через мобильные устройства или геймпад ввод можно перевести на `Input Actions` [22].

`Resources` используется для загрузки спрайтов через `ArtSpriteLibrary`. `ArtSpritePostprocessor` интегрируется с редакторским конвейером импорта. Для файлов в `Resources/Art` он устанавливает тип `Sprite`, одиночный режим, прозрачность, `Clamp`, `Bilinear` и отключает `mirrmap`. Автоматизация устраняет ручную настройку каждого PNG и делает добавление ресурсов более

предсказуемым. В таблице 3.2 указаны пакеты и встроенные библиотеки, которые используются в проекте.

Таблица 3.2 – Пакеты и встроенные библиотеки проекта

| Компонент    | Версия/механизм | Использование                               |
|--------------|-----------------|---------------------------------------------|
| Unity Editor | 6000.0.43f1     | Сцены, жизненный цикл MonoBehaviour, сборка |
| URP          | 17.0.4          | Рендеринг и 2D Renderer                     |
| uGUI         | 2.0.0           | Меню, настройки, HUD, overlay               |
| Input System | 1.13.1          | Будущие Input Actions                       |
| PlayerPrefs  | Встроенный API  | Настройки и рекорд                          |
| Resources    | Встроенный API  | Загрузка спрайтов ArtSpriteLibrary          |

### 3.6 Методика игровой балансировки

#### 3.6.1 Балансировка скорости и кривой сложности

Базовая скорость установлена равной 5. ProgressionSystem каждую секунду добавляет 0,05, пока значение сложности Difficulty01 увеличивается со скоростью 0,015 и ограничивается диапазоном от 0 до 1. Линейная модель проста для восприятия и настройки: разработчик может заранее вычислить состояние через заданное время, а игрок ощущает постепенное, а не скачкообразное ускорение.

Нормализованная сложность достигает 1 примерно через 66,7 секунды. После этого профиль генерации уже не меняется в сторону более сложного, однако скорость продолжает расти. Получаются два слоя прогрессии: сначала одновременно усложняются состав трассы и темп, затем геометрическая насыщенность стабилизируется, а давление создается скоростью. Такой подход предотвращает бесконечный рост плотности, который рано или поздно породил бы невозможные для прохождения ситуации. Формулы скорости и сложности указаны в формулах 3.1–3.2.

$$CurrentSpeed(t) = 5 + 0,05t \quad (3.1)$$

$$Difficulty01(t) = \min(1; 0,015t) \quad (3.2)$$

SpeedBoost умножает скорость на 1,25, но ProgressionSystem продолжает хранить базовое CurrentSpeed. Это удобно: после истечения эффекта игра возвращается к естественной кривой, а не к измененной постоянной скорости. Однако одновременно растут дистанция и счет за секунду. Ускорение является и риском, и наградой.

Профиль выбирается по ближайшему DifficultyWeight. Граница между VillageEasy с весом 0,18 и ForestMid с весом 0,45 находится около 0,315. При скорости роста Difficulty01 0,015 это соответствует 21 секунде. Следующие границы 0,60 и 0,875 достигаются на 40-й и 58,3-й секундах. Значит, первая минута забега естественно делится на четыре фазы. В таблице 3.3 отображены установленные параметры для профилей сложности. В таблице 3.4 указаны фазы кривой сложности после настройки.

Таблица 3.3 – Профили генерации трассы

| Профиль     | Длина сегмента | Плотность препятствий | Вероятность бонуса | DifficultyWeight |
|-------------|----------------|-----------------------|--------------------|------------------|
| VillageEasy | 12,0           | 0,48                  | 0,14               | 0,18             |
| ForestMid   | 12,5           | 0,62                  | 0,20               | 0,45             |
| OutpostHard | 11,5           | 0,74                  | 0,22               | 0,75             |
| Rush        | 10,5           | 0,82                  | 0,28               | 1,00             |

Таблица 3.4 – Фазы кривой сложности

| Интервал времени | Профиль     | Скорость в начале–конце | Характер игрового процесса                                                        |
|------------------|-------------|-------------------------|-----------------------------------------------------------------------------------|
| 0–21 с           | VillageEasy | 5,00–6,05               | Знакомство с управлением, относительно редкие препятствия и широкий запас реакции |
| 21–40 с          | ForestMid   | 6,05–7,00               | Устойчивое чередование прыжка и подката, повышенная частота бонусов               |
| 40–58,3 с        | OutpostHard | 7,00–7,92               | Сокращение временных интервалов, более плотные комбинации                         |
| После 58,3 с     | Rush        | От 7,92 и выше          | Максимальная плотность препятствий, дальнейший рост давления за счет скорости     |

Вероятность бонуса также растет от 0,14 до 0,28. Это компенсирует усложнение: на высокой скорости игрок чаще получает шанс на щит или усиление, но подобрать объект становится труднее. Баланс строится не только через наказание. Награда должна появляться чаще именно тогда, когда риск ее

получения возрастает [12]. Скорость следует оценивать вместе с lookAheadDistance = 28. При скорости 5 подготовленный участок соответствует примерно 5,6 секунды движения, при скорости 10 – 2,8 секунды, при скорости 14 – только 2 секундам. Генератор поддерживает трассу впереди постоянно, но визуальная дальность камеры может быть меньше. Для поздней игры важно, чтобы объект не появлялся заметно в кадре и у игрока оставалось время для распознавания ситуации.

В таблице 3.5 указаны проведённые замеры прогрессии до 120 секунд.

Таблица 3.5 – Замеры прогрессии

| Время, секунды | CurrentSpeed | Difficulty01 | Активный профиль | LookAhead, секунды |
|----------------|--------------|--------------|------------------|--------------------|
| 15 с           | 5,75         | 0,225        | VillageEasy      | 4,87               |
| 30 с           | 6,50         | 0,450        | ForestMid        | 4,31               |
| 45 с           | 7,25         | 0,675        | OutpostHard      | 3,86               |
| 60 с           | 8,00         | 0,900        | Rush             | 3,50               |
| 90 с           | 9,50         | 1,000        | Rush             | 2,95               |
| 120 с          | 11,00        | 1,000        | Rush             | 2,55               |

### 3.6.2 Балансировка прыжка, дополнительного прыжка и подката

Прыжок оценивается через высоту и время нахождения в воздухе. При jumpForce 11,5 и эффективной гравитации около 29,43 теоретическая вершина достигается через 0,39 секунды. Полное возвращение на исходную высоту занимает около 0,78 секунды. За это время мир смещается на 3,9 единицы игрового пространства Unity (Unity units) при скорости 5, на 6,24 при скорости 8 и на 8,58 при скорости 11.

Unity units измеряется по формуле 3.3.

$$S = v \cdot t. \quad (3.3)$$

Эти значения показывают, что одна и та же дуга перекрывает разные горизонтальные расстояния по мере прогрессии. На поздней стадии игрок должен начинать прыжок дальше от препятствия, хотя вертикальная физика не меняется. Если визуальная зона предупреждения остается прежней, действие кажется менее отзывчивым. Поэтому препятствия нельзя располагать, исходя только из высоты спрайта.

Наземное препятствие имеет высоту 0,68, тогда как расчётный подъём центра превышает две единицы. Запас достаточен. Опасность связана не с вершиной, а с моментом начала и приземления: при позднем нажатии передняя часть коллайдера может коснуться объекта ещё до заметного подъёма.

Дополнительный прыжок расширяет вертикальное и временное окно. Поскольку его импульс составляет 94% от основного прыжка и равняется 10,81. Игрок может скорректировать ранний прыжок или задержаться в воздухе.

Подкат длится 0,22 секунды. При скорости 5 мир проходит за этот интервал 1,1 единицы, при скорости 8 – 1,76, при скорости 11 – 2,42. Препятствие Slide имеет ширину около 1,95. Следовательно, на начальной скорости одноразового таймера недостаточно для полного прохождения, если подкат запущен непосредственно у передней границы; игрок должен начать заранее или удерживать клавишу, повторно продлевая состояние. На скорости 11 базовой длительности уже достаточно примерно для ширины преграды, но окно реакции короче.

Высокое препятствие находится на высоте около 1,55 при толщине 0,28. Уменьшенный коллайдер 0,9×0,6 со смещением вниз должен помещаться под ним, тогда как обычная капсула сталкивается. Проверка должна проводиться с Gizmos коллайдеров, потому что визуальный спрайт может выходить за физическую форму и создавать ощущение несправедливого контакта.

В таблице 3.6 выведены замеры смещения мира при различных действиях

Таблица 3.6 – Контрольные значения смещения мира за время маневра

| Манёвр                            | Длительность, секунды | Смещение мира при скорости 5, Unity units | Смещение мира при скорости 8, Unity units | Смещение мира при скорости 11, Unity units |
|-----------------------------------|-----------------------|-------------------------------------------|-------------------------------------------|--------------------------------------------|
| Полный прыжок                     | 0,78 с                | 3,90                                      | 6,24                                      | 8,58                                       |
| Подъём до вершины                 | 0,39 с                | 1,95                                      | 3,12                                      | 4,29                                       |
| Подкат                            | 0,22 с                | 1,10                                      | 1,76                                      | 2,42                                       |
| Coyote time                       | 0,08 с                | 0,40                                      | 0,64                                      | 0,88                                       |
| Защита от заземления после прыжка | 0,12 с                | 0,60                                      | 0,96                                      | 1,32                                       |

### 3.7 Функциональное тестирование механик

В таблице 3.7 описаны проведённые тесты по функционалу проекта.

Таблица 3.7 – Проведённые тестирования

| ID | Сценарий                             | Предусловия                                     | Действия                                   | Ожидаемый результат                                             | Фактический результат               |
|----|--------------------------------------|-------------------------------------------------|--------------------------------------------|-----------------------------------------------------------------|-------------------------------------|
| 01 | Прыжок                               | Персонаж на земле                               | Нажать Space или стрелку вверх             | Однократный импульс вверх, IsGrounded = false, AirJumpsLeft = 1 | Соответствует ожидаемому результату |
| 02 | Доп. прыжок                          | Выполнен прыжок, персонаж в воздухе             | Нажать кнопку Q                            | Дополнительный однократный импульс вверх                        | Соответствует ожидаемому результату |
| 03 | Повторное использование доп. прыжка  | Выполнен прыжок, выполнен дополнительный прыжок | Нажать кнопку Q повторно                   | Ничего не происходит                                            | Соответствует ожидаемому результату |
| 04 | Столкновение с препятствием Slide    | Нет щита                                        | Коснуться препятствия Slide                | Мир и персонаж останавливаются, появляется окно поражения       | Соответствует ожидаемому результату |
| 05 | Подкат находясь в воздухе            | Выполнен прыжок, быть в воздухе                 | Нажать кнопку S или стрелку вниз в воздухе | Команда не меняет состояние и форму коллайдера                  | Соответствует ожидаемому результату |
| 06 | Перезапуск забега находясь в воздухе | Выполнен прыжок, быть в воздухе                 | Нажать кнопку R                            | Исходный коллайдер и физика восстановлены, показатели сброшены  | Соответствует ожидаемому результату |
| 07 | Перезапуск забега выполняя подкат    | Выполняется подкат,                             | Нажать кнопку R                            | Исходный коллайдер и физика восстановлены, показатели сброшены  | Соответствует ожидаемому результату |
| 08 | Столкновение с препятствием Jump     | Нет щита                                        | Коснуться препятствия Jump                 | Мир и персонаж останавливаются, появляется окно поражения       | Соответствует ожидаемому результату |
| 09 | Столкновение с препятствием Air      | Нет щита                                        | Прыгнуть, коснувшись препятствия Air       | Мир и персонаж останавливаются, появляется окно поражения       | Соответствует ожидаемому результату |

Продолжение таблицы 3.7

| ID | Сценарий                                  | Предусловия                         | Действия                         | Ожидаемый результат                                         | Фактический результат               |
|----|-------------------------------------------|-------------------------------------|----------------------------------|-------------------------------------------------------------|-------------------------------------|
| 10 | Преодоление препятствия Slide подкатом    | Нет щита, есть препятствие Slide    | Нажать кнопку S или стрелку вниз | Персонаж преодолел препятствие, игра продолжается           | Соответствует ожидаемому результату |
| 11 | Столкновение с препятствием со щитом      | Есть щит                            | Коснуться любого препятствия     | Препятствие деактивируется, щит исчезает, игра продолжается | Соответствует ожидаемому результату |
| 12 | Повторное нажатие кнопок прыжка в воздухе | Выполнен прыжок, персонаж в воздухе | Нажать Space или стрелку вверх   | Новые импульсы не возникают                                 | Соответствует ожидаемому результату |

### 3.8 Выводы по разделу

В третьей главе рассмотрена фактическая реализация и балансировка игрового процесса линейного 2D-автораннера RusRunner.

Процесс разработки обоснован как итерационно-инкрементный. Для игрового проекта такой подход предпочтительнее жёсткой последовательности, поскольку управляемость и сложность уточняются после запуска. Предложены правила организации Git-репозитория.

Подтверждено соответствие реализации компонентно-модульной архитектуре. GameContext хранит состояние сессии, RunnerController отвечает за физику персонажа, TrackGenerator – за бесконечную трассу и пулы, отдельные системы управляют счетом, прогрессией и временными эффектами, а UI читает публичное состояние через RunnerBootstrap.

Реализовано управление персонажем через прыжок, подкат и дополнительный прыжок.

Реализованы препятствия типа Jump, Slide и Air. Генератор использует четыре профиля сложности, которые последовательно активируются. После достижения максимального профиля плотность стабилизируется, но скорость

продолжает расти. Такой двухслойный принцип предотвращает бесконечное увеличение числа объектов и сохраняет давление на игрока.

Система результата рассчитывает дистанцию и очки по эффективной скорости и множителю, а рекорд хранится в PlayerPrefs. HUD показывает ключевые значения и активные состояния.

Был проведён анализ на наличие качественного кода и принципов SOLID.

Балансировка была дополнена расчётными моделями. Показано, что горизонтальная дистанция действия растёт вместе со скоростью, поэтому генератору требуется учитывать минимальное временное окно, а не только фиксированные координаты.

В разделе тестирования сформирована таблица проведённых сценариев тестирований механик проекта. Тестирование прошло без ошибок.

Подводя итог, реализованный игровой процесс соответствует цели. Порядок запуска, настройки и использования программного комплекса RusRunner приведен в руководстве пользователя в приложении А.

## 4 РАСЧЕТ ТЕХНИКО-ЭКОНОМИЧЕСКОЙ ЭФФЕКТИВНОСТИ РАЗРАБОТКИ И ВНЕДРЕНИЯ 2D-АВТОРАННЕРА RUSRUNNER

### 4.1 Обоснование необходимости расчета эффективности

Экономическое обоснование необходимо для оценки затрат на создание программного продукта RusRunner и для проверки целесообразности его дальнейшего использования как демонстрационного, учебного и повторно применимого игрового модуля. Проект представляет собой 2D-автораннер на Unity, реализованный на C# с разделением на игровой контекст, системы ядра, контроллер персонажа, генератор трассы, интерфейсные компоненты и слой представления.

В отличие от серверных информационных систем, RusRunner не требует постоянной аренды серверной инфраструктуры и не хранит пользовательские данные во внешней базе. Основная экономическая ценность формируется за счет повторного использования модульного ядра, ускорения разработки похожих игровых прототипов, снижения затрат на стартовый шаблон игры и возможности применять проект в учебном процессе по разработке игр.

Расчет выполняется для варианта внутренней разработки и демонстрационного внедрения в небольшой образовательной или проектной команде. Продажа игры на коммерческих площадках, рекламное продвижение и платная поддержка внешних пользователей не включаются в расчет, так как они относятся к отдельной бизнес-модели.

В таблице 4.1 описаны связи функций RusRunner с ожидаемым экономическим эффектом.

Таблица 4.1 – Связь функций RusRunner с ожидаемым экономическим эффектом

| Функциональный блок | Проблема без готового модуля                                                                           | Ожидаемый эффект                                                               |
|---------------------|--------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Модульное ядро      | Логика запуска, состояния забега, прогрессии и поражения разрабатывается заново для каждого прототипа. | Сокращается время подготовки новых 2D-runner проектов и демонстрационных сцен. |

| Функциональный блок          | Проблема без готового модуля                                                                             | Ожидаемый эффект                                                                                         |
|------------------------------|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| TrackGenerator и object pool | При ручном создании трассы растет трудоемкость настройки уровней и возможны просадки производительности. | Повторное использование сегментов, препятствий и бонусов снижает нагрузку и ускоряет настройку контента. |
| RunnerController             | Механики прыжка, скольжения, столкновений и восстановления после рестарта требуют отдельной отладки.     | Готовый контроллер уменьшает риск ошибок в базовой игровой механике.                                     |
| PowerUpSystem                | Бонусы часто реализуются разрозненно и усложняют балансировку.                                           | Единый контракт IPowerUp упрощает добавление временных эффектов.                                         |
| uGUI и HUD                   | Меню, настройки, HUD и экран поражения приходится проектировать отдельно.                                | Появляется готовый пользовательский сценарий: меню, забег, поражение, рестарт.                           |
| PlayerPrefs                  | Настройки и рекорды требуют отдельного механизма локального хранения.                                    | Локальное сохранение позволяет обойтись без сервера и базы данных для первой версии.                     |

## 4.2 Описание объекта оценки

Объектом оценки является программный комплекс RusRunner - 2D-игра в жанре автораннер, разработанная в Unity с использованием языка C#. В учебной литературе по Unity и C# подчеркивается, что объектно-ориентированные средства, классы, интерфейсы, свойства и методы позволяют разделять игровую логику на самостоятельные компоненты [10].

Клиентская часть проекта работает в среде Unity Runtime. Для интерфейса используются элементы uGUI, который в документации Unity описан как GameObject-based UI system для создания интерфейсов игр и приложений [28]. Локальные настройки и рекорд сохраняются через PlayerPrefs; документация Unity указывает, что PlayerPrefs хранит значения между игровыми сессиями, но не предназначен для чувствительных данных из-за отсутствия шифрования [25].

Графическая часть проекта использует ресурсы из Resources/Art и настройки URP 2D. Документация Unity по URP 2D описывает инструменты 2D-освещения и 2D Renderer, применимые для построения спрайтовой сцены [19].

В таблице 4.2 указана характеристика объекта оценки с принятым значением для расчёта.

Таблица 4.2 – Характеристика объекта оценки

| Параметр                    | Принятое значение для расчета                                                                                                   |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Тип продукта                | 2D-игра в жанре автораннер                                                                                                      |
| Название проекта            | RusRunner                                                                                                                       |
| Назначение                  | Демонстрация модульного игрового ядра, пользовательской навигации, генерации трассы, бонусов, столкновений и локальных настроек |
| Архитектура                 | Unity Client: RunnerBootstrap, GameContext, Systems, RunnerController, TrackGenerator, UI, InputSource, Presentation            |
| Основные пользователи       | Игрок, преподаватель или разработчик, использующий проект как учебный пример                                                    |
| Расчетный масштаб внедрения | Одна учебная или проектная команда, 1-2 разработчика, локальная демонстрационная эксплуатация                                   |
| Инфраструктура              | Unity Editor, C#, URP, uGUI, PlayerPrefs, Resources/Art, IDE                                                                    |
| Ограничения расчета         | Не учитываются маркетинг, публикация в магазинах приложений, серверная аналитика и коммерческая поддержка внешних игроков       |

### 4.3 Исходные данные для расчетов

Расчеты выполнены для 2026 года. В качестве расчетной нормы используется пятидневная рабочая неделя и годовая норма 1972 часа, что дает среднюю месячную норму 164,3 часа. Для страховых взносов принят общий тариф 30 процентов. Часть значений является расчётной оценкой для дипломного проекта и может уточняться при фактическом внедрении [8].

Средняя месячная норма рабочего времени рассчитывается по формуле 4.1:

$$F_{\text{мес}} = \frac{1972}{12} = 164,3\text{ч.} \quad (4.1)$$

В таблице 4.3 показаны исходные нормативные и расчётные показатели.

Таблица 4.3 – Исходные нормативные и расчётные показатели

| Показатель                       | Значение    | Источник или пояснение                                                     |
|----------------------------------|-------------|----------------------------------------------------------------------------|
| Расчетный год                    | 2026        | Используется для календарных и финансовых расчетов                         |
| Рабочие дни в году               | 247 дней    | Производственный календарь на 2026 год                                     |
| Норма рабочего времени           | 1972 ч/год  | 40-часовая рабочая неделя                                                  |
| Средняя месячная норма           | 164,3 ч/мес | 1972 / 12                                                                  |
| Тариф страховых взносов          | 30%         | Расчетное значение для оценки фонда оплаты труда                           |
| Накладные расходы                | 15%         | Организационные расходы, связь, сопровождение процесса разработки          |
| Резерв на непредвиденные расходы | 8%          | Риск изменения требований, исправления дефектов и дополнительной настройки |
| Расчетный срок разработки        | 3 месяца    | С учетом параллельной работы над кодом, сценами и документацией            |

#### 4.4 Методика расчета эффективности

Методика расчета строится на сравнении совокупных затрат на разработку и внедрение с ожидаемым годовым эффектом от повторного использования решения. Для игрового проекта эффект не сводится только к прямой выручке: важны снижение трудоемкости будущих прототипов, наличие готовой архитектуры и возможность использовать проект как учебный материал.

Общая трудоемкость разработки определяется суммированием трудоемкости отдельных видов работ:

$$T_{\text{общ}} = \sum T_i. \quad (4.2)$$

Фонд оплаты труда рассчитывается по формуле 4.3:

$$C_{\text{зн}} = \sum T_i \cdot S_i. \quad (4.3)$$

Страховые взносы и накладные расходы определяются как доля от фонда оплаты труда:

$$C_{св} = C_{зн} \cdot K_{св} \quad (4.4)$$

$$C_{накл} = C_{зн} \cdot K_{накл} \quad (4.5)$$

Полная стоимость разработки определяется по формуле 4.6:

$$C_{разр} = (C_{зн} + C_{св} + C_{накл} + C_{ам} + C_{инф} + C_{проч}) \cdot (1 + K_{рез}). \quad (4.6)$$

Годовой чистый экономический эффект определяется как разность между валовым эффектом и годовыми эксплуатационными расходами:

$$\mathcal{E}_{год} = \mathcal{E}_{вал} - C_{накл}. \quad (4.7)$$

Срок окупаемости и ROI рассчитываются по формулам 4.8–4.9:

$$T_{ок} = \frac{C_{разр}}{\mathcal{E}_{год}}, \quad (4.8)$$

$$ROI = \left( \frac{\mathcal{E}_{год} \cdot n - C_{разр}}{C_{разр}} \right) \cdot 100\%. \quad (4.9)$$

## 4.5 Затраты на разработку и внедрение

Затраты на разработку включают анализ требований, проектирование игровых сценариев, реализацию ядра, интеграцию Unity-сцены, подготовку UI, тестирование и документацию. Проект не требует покупки платных библиотек: используются Unity, C#, URP, uGUI, локальные ресурсы и собственная реализация игровых систем.

В таблице 4.4 расписана оценка трудоёмкости и затрат на оплату труда.

Таблица 4.4 – Оценка трудоёмкости и затрат на оплату труда

| Вид работ                                                           | Трудоемк., чел.-ч | Ставка, руб./ч | Стоимость, руб. |
|---------------------------------------------------------------------|-------------------|----------------|-----------------|
| Анализ требований, пользовательских сценариев и игровых ограничений | 48                | 800            | 38 400          |
| Проектирование механики автораннера, меню, настроек и HUD           | 64                | 850            | 54 400          |

Продолжение таблицы 4.4

| Вид работ                                                                        | Трудоемк., чел.-ч | Ставка, руб./ч | Стоимость, руб. |
|----------------------------------------------------------------------------------|-------------------|----------------|-----------------|
| Проектирование модульного ядра, GameContext, систем и интерфейсов                | 72                | 900            | 64 800          |
| Настройка Unity-сцены, URP 2D, uGUI и интеграция графических ресурсов            | 150               | 850            | 127 500         |
| Разработка RunnerController, обработки ввода, прыжка, скольжения и столкновений  | 110               | 850            | 93 500          |
| Разработка TrackGenerator, сегментов трассы, препятствий, бонусов и object pool  | 120               | 850            | 102 000         |
| Разработка систем прогрессии, счета, бонусов, способностей и сохранения настроек | 130               | 900            | 117 000         |
| Разработка главного меню, экрана настроек, HUD и сценариев Game Over             | 90                | 800            | 72 000          |
| Тестирование, балансировка сложности, оптимизация и исправление дефектов         | 100               | 650            | 65 000          |
| Подготовка документации, схем и демонстрационного сценария                       | 50                | 700            | 35 000          |

Общая трудоемкость работ составляет 934 чел.-ч. Фонд оплаты труда по таблице 4.4 составляет:

$$C_{зн} = 769600 \text{ руб.} \quad (4.10)$$

Страховые взносы при тарифе 30 процентов составляют:

$$C_{св} = 769600 \cdot 0,3 = 230880 \text{ руб.} \quad (4.11)$$

Накладные расходы приняты в размере 15 процентов от фонда оплаты труда:

$$C_{накл} = 115440 \text{ руб.} \quad (4.12)$$

В таблице 4.5 указана смета на разработку и внедрение проекта.

**Таблица 4.5 – Смета затрат на разработку и внедрение**

| Статья затрат              | Порядок расчета                                                 | Сумма, руб. |
|----------------------------|-----------------------------------------------------------------|-------------|
| Фонд оплаты труда          | По трудоемкости работ                                           | 769 600     |
| Страховые взносы           | 30% от фонда оплаты труда                                       | 230 880     |
| Накладные расходы          | 15% от фонда оплаты труда                                       | 115 440     |
| Амортизация оборудования   | $190\,000 / 36 \times 3$                                        | 15 833      |
| Интернет и связь           | $1\,200 \text{ руб./мес} \times 3 \text{ месяца}$               | 3 600       |
| Электроэнергия             | $0,12 \text{ кВт} \times 934 \text{ ч} \times 7,2 \text{ руб.}$ | 807         |
| Прочие расходы             | Файловое хранение, расходные материалы, оформление              | 15 000      |
| Резерв                     | 8% от суммы затрат                                              | 92 093      |
| Итого стоимость разработки | С учетом округления                                             | 1 243 300   |

С учетом округления стоимость разработки и начального внедрения принимается равной:

$$C_{\text{разр}} = 1243300 \text{ руб.} \quad (4.13)$$

#### **4.6 Расходы на эксплуатацию после внедрения**

После завершения разработки RusRunner не требует отдельного сервера, базы данных и круглосуточного администрирования. Основные эксплуатационные расходы связаны с сопровождением Unity-проекта, обновлением зависимостей, исправлением ошибок, хранением исходных материалов и периодической проверкой сборки.

В таблице 4.6 показаны результаты расчётов годовых эксплуатационных расходов.

**Таблица 4.6 – Годовые эксплуатационные расходы**

| Статья расходов                         | Порядок расчета                                                   | Сумма, руб./год |
|-----------------------------------------|-------------------------------------------------------------------|-----------------|
| Техническое сопровождение Unity-проекта | $8 \text{ ч/мес} \times 800 \text{ руб./ч} \times 12 \text{ мес}$ | 76 800          |

Продолжение таблицы 4.6

| Статья расходов                                  | Порядок расчета                 | Сумма, руб./год |
|--------------------------------------------------|---------------------------------|-----------------|
| Плановые обновления и мелкие доработки           | Экспертная оценка               | 30 000          |
| Плановые обновления и мелкие доработки           | Экспертная оценка               | 30 000          |
| Хранение резервных копий и исходных материалов   | Локальное и облачное хранение   | 10 000          |
| Электроэнергия для тестирования и демонстраций   | Оценка потребления рабочих мест | 2 000           |
| Проверка совместимости и демонстрационные сборки | Экспертная оценка               | 15 000          |
| Итого эксплуатационные расходы                   |                                 | 133 800         |

Годовые эксплуатационные расходы принимаются равными

$$C_{\text{экспл}} = 133800 \text{ руб./год} \quad (4.14)$$

#### 4.7 Экономия ресурсов от внедрения

Экономия ресурсов возникает за счет повторного использования архитектурных решений и готовых игровых механизмов. В проекте уже реализованы сценарии запуска забега, обработки ввода, остановки при столкновении, подбора бонусов, локального сохранения настроек и возврата объектов в пул. Unity Learn указывает, что object pooling снижает затраты на повторное создание и уничтожение GameObject и уменьшает нагрузку на сборщик мусора [24].

В таблице 4.7 расписаны расчёты годового экономического эффекта

Таблица 4.7 – Расчёт годового экономического эффекта

| Источник экономии                                          | Расчет             | Сумма, руб./год |
|------------------------------------------------------------|--------------------|-----------------|
| Повторное использование модульного ядра в новых прототипах | 180 ч × 850 руб./ч | 153 000         |
| Сокращение времени интеграции 2D-сцен и ресурсов           | 60 ч × 750 руб./ч  | 45 000          |

| Источник экономии                                            | Расчет                                      | Сумма, руб./год |
|--------------------------------------------------------------|---------------------------------------------|-----------------|
| Отказ от покупки коммерческого starter kit                   | Оценка стоимости готовых шаблонов и ассетов | 65 000          |
| Снижение трудоемкости тестирования базовых сценариев         | 72 ч × 650 руб./ч                           | 46 800          |
| Использование проекта как учебно-демонстрационного материала | 110 ч × 650 руб./ч                          | 71 500          |
| Валовой годовой эффект                                       |                                             | 381 300         |

Валовой годовой экономический эффект принимается равным

$$\mathcal{E}_{вал} = 381300 \text{ руб./год} \quad (4.15)$$

#### 4.8 Расчет финансовых показателей

Чистый годовой экономический эффект рассчитывается как разность между валовым эффектом и эксплуатационными расходами:

$$\mathcal{E}_{год} = 381300 - 133800 = 247500 \text{ руб./год} \quad (4.16)$$

В таблице 4.8 показаны исходные показатели для расчёта эффективности проекта. В таблице 4.9 указаны значения финансовые показатели эффективности.

Таблица 4.8 – Исходные показатели для расчета эффективности

| Показатель                           | Значение         |
|--------------------------------------|------------------|
| Стоимость разработки и внедрения     | 1 243 300 руб.   |
| Годовые эксплуатационные расходы     | 133 800 руб./год |
| Валовой годовой экономический эффект | 381 300 руб./год |
| Чистый годовой экономический эффект  | 247 500 руб./год |

Таблица 4.9 – Финансовые показатели проекта

| Показатель       | Значение  |
|------------------|-----------|
| Срок окупаемости | 5,02 года |

| Показатель    | Значение |
|---------------|----------|
| ROI за 3 года | -40,3%   |
| ROI за 5 лет  | -0,5%    |

Срок окупаемости составляет:

$$T_{ок} = \left( \frac{1243300}{247500} \right) = 5,02 \text{ года} \quad (4.17)$$

ROI за 3 года:

$$ROI_3 = \left( \frac{247500 \cdot 3 - 1243300}{1243300} \right) \cdot 100\% = -40,3\% \quad (4.18)$$

ROI за 5 лет:

$$ROI_5 = \left( \frac{247500 \cdot 5 - 1243300}{1243300} \right) \cdot 100\% = -0,5\% \quad (4.19)$$

#### 4.9 Анализ чувствительности, рисков и пути их минимизации

В таблице 4.10 расписан анализ чувствительности проекта. В таблице 4.11 описаны риски проекта и мероприятия по их снижению.

Таблица 4.10 – Анализ чувствительности проекта

| Сценарий       | Условия сценария                                                           | Чистый эффект, руб./год | Срок окупаемости, лет |
|----------------|----------------------------------------------------------------------------|-------------------------|-----------------------|
| Пессимистичный | Валовой эффект ниже на 20%, эксплуатационные расходы выше на 10%           | 157 860                 | 7,88                  |
| Базовый        | Показатели соответствуют расчету                                           | 247 500                 | 5,02                  |
| Оптимистичный  | Валовой эффект выше на 20%, эксплуатационные расходы соответствуют расчету | 323 760                 | 3,84                  |

Таблица 4.11 – Риски проекта и мероприятия по снижению

| Риск                                                       | Вероятность | Влияние | Мероприятия по снижению                                                                                            |
|------------------------------------------------------------|-------------|---------|--------------------------------------------------------------------------------------------------------------------|
| Недооценка трудоемкости игровой механики                   | Средняя     | Среднее | Разделение разработки на базовые сценарии, резерв 8%, раннее тестирование.                                         |
| Изменение требований к интерфейсу и балансу                | Средняя     | Среднее | Фиксация минимального набора экранов, отдельный список улучшений после демонстрационной версии.                    |
| Проблемы производительности при большом числе объектов     | Низкая      | Среднее | Использование object pool, ограничение числа активных сегментов, профилирование сцены.                             |
| Потеря локальных настроек или рекорда                      | Низкая      | Низкое  | Сохранение только некритичных данных, проверка ключей PlayerPrefs, резервное восстановление настроек по умолчанию. |
| Несовместимость после обновления Unity                     | Средняя     | Среднее | Фиксация версии проекта, контроль Packages и ProjectSettings, тестовая сборка после обновлений.                    |
| Недостаточная готовность проекта к коммерческой публикации | Средняя     | Среднее | Разделение учебной версии и коммерческого релиза, отдельная оценка маркетинга и платформ публикации.               |

#### 4.10 Выводы по разделу

Расчет показал, что стоимость разработки и демонстрационного внедрения RusRunner составляет 1 243 300 руб. Годовые эксплуатационные расходы невысоки по сравнению с серверными системами и составляют 133 800 руб./год, так как проект работает локально и использует PlayerPrefs для некритичных настроек.

Чистый годовой эффект оценивается в 247 500 руб./год, Срок окупаемости составляет около 5,02 года, ROI за 5 лет – 0,5%, Полученные показатели показывают, что проект экономически оправдан в первую очередь как повторно используемое учебно-проектное ядро и основа для дальнейших игровых прототипов.

## 5 БЕЗОПАСНОСТЬ И ЭКОЛОГИЧНОСТЬ РАЗРАБОТКИ

### 5.1 Анализ основных вредных и опасных факторов

При разработке игрового приложения основная часть времени приходится на программирование на C#, настройку Unity-сцены, работу с графическими ресурсами, тестирование игрового цикла и подготовку документации. Эти работы требуют длительного взаимодействия с монитором, клавиатурой и мышью.

В таблице 5.1 указаны основные вредные и опасные факторы при разработке RusRunner.

Таблица 5.1 – Основные вредные и опасные факторы при разработке RusRunner

| Группа факторов               | Источник возникновения                                           | Возможные последствия                                   | Актуальность для разработки       |
|-------------------------------|------------------------------------------------------------------|---------------------------------------------------------|-----------------------------------|
| Зрительное напряжение         | Длительная работа с Unity Editor, IDE, схемами и документами     | Утомление глаз, снижение концентрации, головная боль    | Высокая                           |
| Статическая нагрузка          | Длительная работа сидя, неправильная высота стола и кресла       | Боли в спине, шее, плечах и кистях                      | Высокая                           |
| Психофизиологическая нагрузка | Отладка кода, поиск ошибок, балансировка сложности               | Переутомление, рост числа ошибок                        | Средняя                           |
| Неблагоприятный микроклимат   | Недостаточная вентиляция, сухой воздух, перегрев помещения       | Дискомфорт, утомляемость                                | Средняя                           |
| Электрический ток             | Компьютер, монитор, зарядные устройства, удлинители              | Поражение электрическим током, повреждение оборудования | Низкая при исправном оборудовании |
| Пожарная опасность            | Перегрузка сетевых фильтров, поврежденные провода, блоки питания | Возгорание оборудования или помещения                   | Средняя                           |
| Информационная безопасность   | Локальные настройки, сохранение рекорда, файлы проекта           | Потеря данных, некорректная работа приложения           | Средняя                           |

## 5.2 Меры обеспечения безопасности

### 5.2.1 Организация рабочего места и режима труда

Рабочее место разработчика должно обеспечивать устойчивое положение корпуса, удобное расположение монитора, клавиатуры и мыши, а также отсутствие бликов на экране. Нормальная продолжительность рабочего времени в российском трудовом законодательстве не должна превышать 40 часов в неделю. Это означает, что необходимо планировать работу по этапам и избегать длительных непрерывных сессий без перерывов [7; 9].

В таблице 5.2 описаны рекомендуемые параметры рабочего места. В таблице 5.3 указаны мероприятия по снижению воздействия вредных факторов.

Таблица 5.2 – Рекомендуемые параметры рабочего места

| Параметр                    | Рекомендуемое значение                                                   | Назначение требования                                     |
|-----------------------------|--------------------------------------------------------------------------|-----------------------------------------------------------|
| Освещенность рабочего стола | 300-500 лк                                                               | Снижение зрительной нагрузки при работе с кодом и схемами |
| Положение монитора          | Экран прямо перед пользователем, верхняя граница примерно на уровне глаз | Снижение нагрузки на шею и глаза                          |
| Расстояние до экрана        | 60–70 см                                                                 | Комфортное чтение кода и интерфейса Unity                 |
| Положение клавиатуры        | Перед пользователем, с местом для опоры кистей                           | Снижение нагрузки на кисти и предплечья                   |
| Режим работы                | Короткие перерывы после длительных периодов концентрации                 | Профилактика переутомления и ошибок                       |

Таблица 5.3 – Мероприятия по снижению воздействия факторов

| Фактор                | Мероприятие                                             | Ожидаемый результат              |
|-----------------------|---------------------------------------------------------|----------------------------------|
| Зрительное напряжение | Настройка яркости, отсутствие бликов, перерывы для глаз | Снижение утомления глаз          |
| Статическая поза      | Регулируемое кресло, удобная высота стола, разминка     | Снижение нагрузки на спину и шею |

| Фактор                      | Мероприятие                                                     | Ожидаемый результат                               |
|-----------------------------|-----------------------------------------------------------------|---------------------------------------------------|
| Умственное переутомление    | Планирование задач, чередование программирования и тестирования | Сохранение концентрации                           |
| Неблагоприятный микроклимат | Проветривание, контроль температуры и влажности                 | Комфортные условия работы                         |
| Бытовые риски               | Укладка проводов, свободный проход к рабочему месту             | Снижение риска падений и повреждений оборудования |

### 5.2.2 Электробезопасность

Разработка выполняется на персональном компьютере или ноутбуке, поэтому необходимо использовать исправные кабели питания, сертифицированные блоки питания и сетевые фильтры. Запрещается эксплуатация поврежденных проводов, перегрузка розеток и подключение большого количества потребителей к одному удлинителю. При обнаружении запаха гари, перегрева блока питания или нестабильной работы оборудования эксплуатация должна быть прекращена.

### 5.2.3 Пожарная безопасность

Пожарная безопасность обеспечивается исправностью электропроводки, недопущением перегрузки сетевых фильтров, отсутствием легковоспламеняющихся материалов рядом с блоками питания и соблюдением порядка на рабочем месте. Для снижения риска возгорания оборудование выключается после завершения работы, а вентиляционные отверстия ноутбука или системного блока не перекрываются [6].

### 5.2.4 Информационная безопасность игрового приложения

RusRunner не обрабатывает персональные данные и не использует серверную авторизацию. Тем не менее информационная безопасность важна для корректной работы локальных настроек, рекорда и исходного проекта.

PlayerPrefs хранит данные без шифрования, поэтому через него нельзя сохранять пароли, токены и другие чувствительные значения [25].

В таблице 5.4 определены меры для информационной безопасности при разработке RusRunner.

Таблица 5.4 – Меры информационной безопасности RusRunner

| Угроза                               | Возможное проявление                                      | Реализуемая или рекомендуемая мера                                                       |
|--------------------------------------|-----------------------------------------------------------|------------------------------------------------------------------------------------------|
| Изменение локальных настроек         | Пользователь вручную меняет значения PlayerPrefs          | Хранить только некритичные настройки и рекорд, проверять диапазоны значений при загрузке |
| Потеря сохраненного рекорда          | Очистка локального хранилища или переустановка приложения | Не считать PlayerPrefs единственным источником важных данных                             |
| Повреждение проекта                  | Удаление сцен, ресурсов или ProjectSettings               | Хранение резервной копии проекта и фиксация версии Packages                              |
| Ошибки в игровых состояниях          | Неверный переход между меню, забегом и Game Over          | Централизация управления в RunnerBootstrap и GameContext                                 |
| Неконтролируемый рост объектов сцены | Создание новых препятствий без возврата в пул             | RecycleOffscreen и возврат объектов в Stack-пулы                                         |

### 5.3 Экологичность разработки

Экологичность проекта связана не с производственными выбросами, а с рациональным использованием вычислительных ресурсов, уменьшением бумажного документооборота и продлением срока службы оборудования. В проекте используется локальная архитектура без постоянного сервера, что снижает эксплуатационное энергопотребление по сравнению с сетевыми системами.

В таблице 5.5 определены мероприятия по экологичности разработки и эксплуатации при разработке RusRunner.

Таблица 5.5 – Мероприятия по экологичности разработки и эксплуатации

| Направление              | Мероприятие                                                                  | Ожидаемый результат                               |
|--------------------------|------------------------------------------------------------------------------|---------------------------------------------------|
| Бумажный документооборот | Хранение схем, кода и отчетов в электронном виде                             | Снижение расхода бумаги                           |
| Энергопотребление        | Использование режима сна, отключение неиспользуемого оборудования            | Снижение лишнего потребления электроэнергии       |
| Оптимизация игры         | Object pool, ограничение активных объектов, локальная генерация трассы       | Снижение лишней нагрузки на CPU и сборщик мусора  |
| Хранение ресурсов        | Удаление временных ассетов, контроль дубликатов изображений                  | Рациональное использование дискового пространства |
| Эксплуатация техники     | Очистка оборудования от пыли, обновление ПО                                  | Продление срока службы компьютера                 |
| Электронные отходы       | Передача неисправной техники и аккумуляторов специализированным организациям | Снижение риска загрязнения окружающей среды       |

## 5.4 Выводы по разделу

Разработка RusRunner относится к безопасным видам работ при условии правильной организации рабочего места и соблюдения режима труда. Основные факторы риска связаны с длительной работой за компьютером, зрительной нагрузкой, статической позой и использованием электрического оборудования.

Для программного продукта предусмотрены меры информационной безопасности, соответствующие его локальной архитектуре: через PlayerPrefs сохраняются только некритичные данные, управление состояниями сосредоточено в RunnerBootstrap и GameContext, а повторно используемые игровые объекты возвращаются в пул. Экологичность разработки обеспечивается электронным документооборотом, отсутствием постоянной серверной инфраструктуры и оптимизацией игрового цикла.

## ЗАКЛЮЧЕНИЕ

В ходе выполнения комплексной выпускной квалификационной работы был спроектирован, оптимизирован и реализован программный комплекс казуальной 2D-игры RusRunner на основе жанра линейного автораннера. Итоговая разработка объединила два взаимосвязанных направления: создание модульного и расширяемого игрового ядра, а также реализацию и балансировку конкретного игрового процесса. Такой подход позволил рассматривать архитектурные решения не изолированно, а через их влияние на управляемость, сложность, производительность и возможность многократного повторения игровой сессии.

На этапе анализа предметной области были определены основные особенности жанра: автоматическое продвижение, ограниченный набор действий, короткий цикл «старт – прохождение – поражение – рестарт», высокая зависимость результата от точности коллизий и времени реакции. Сравнение с существующими решениями показало значимость визуальной читаемости препятствий, быстрого возврата к новой попытке, постепенного роста сложности, выразительности движения и понятной причинно-следственной связи между действием игрока и поражением. На этой основе были сформулированы функциональные и нефункциональные требования к разрабатываемой игре.

Было спроектировано модульное ядро 2D-автораннера. Центральным координирующим компонентом стал RunnerBootstrap, управляющий жизненным циклом забега и взаимодействием между подсистемами. Изменяемые данные игровой сессии объединены в GameContext, что позволило использовать единый источник состояния для скорости, дистанции, счета, заземления, подката, неуязвимости и множителей. Логика прогрессии, начисления результата и временных эффектов вынесена в самостоятельные классы ProgressionSystem, ScoreSystem и PowerUpSystem. Работа со сценой и физикой сосредоточена в RunnerController и TrackGenerator, а пользовательский интерфейс получает данные через публичный фасад координатора.

Выбранный компонентно-модульный архитектурный стиль соответствует принципам Unity, но ограничивает смешение обязанностей MonoBehaviour-компонентов. Для описания структуры были использованы модели C4 и UML, определены связи между контекстом, контейнерами, компонентами и классами. В реализации применены принципы SOLID и шаблоны Facade, Strategy, Object Pool и объекты временных эффектов. Интерфейсы InputSource, IPowerUp и IAbility сформировали точки расширения, благодаря которым новые способы ввода, бонусы и способности могут добавляться без изменения основной последовательности игрового цикла.

Особое внимание было уделено оптимизации бесконечной трассы. TrackGenerator поддерживает списки активных сегментов, препятствий и бонусов и возвращает ушедшие за экран объекты в отдельные пулы. Повторное использование GameObject снижает необходимость постоянных операций создания и уничтожения. Дополнительную устойчивость обеспечивает выбранная модель движения: персонаж фиксируется по оси X, а игровой мир смещается влево. Это упрощает работу камеры, ограничивает диапазон координат и делает переработку объектов предсказуемой. Визуальные задачи отделены от ядра с помощью презентационных компонентов и параллакс-слоев.

Реализован полный базовый цикл линейного автораннера. Персонаж выполняет наземный прыжок, дополнительный прыжок и подкат. В игровом мире используются препятствия типов Jump, Slide и Air, требующие разных действий. Обработка столкновения учитывает активный щит и корректный подкат, после чего либо продолжает сессию, либо переводит игру в состояние поражения.

Система временных эффектов включает щит, удвоение очков и ускорение. Каждый бонус реализует общий контракт, применяется к GameContext и автоматически отменяется после завершения таймера. ScoreSystem рассчитывает дистанцию и очки с учетом эффективной скорости и множителя, а лучший результат сохраняется через PlayerPrefs. Пользовательский интерфейс содержит главное меню, настройки громкости и частоты кадров, игровой HUD,

отображение активных состояний и итоговую панель поражения. Перезапуск сцены обеспечивает полный сброс временных объектов, физики и интерфейса перед новой попыткой.

Балансировка игрового процесса выполнена на основе параметров фактической реализации. Начальная скорость установлена равной 5 Unity units в секунду и увеличивается на 0,05 в секунду. Нормализованная сложность возрастает на 0,015 в секунду и определяет последовательное включение профилей VillageEasy, ForestMid, OutpostHard и Rush. После достижения максимального профиля плотность препятствий стабилизируется, а дальнейшее давление формируется ростом скорости. Такой двухуровневый подход позволяет избежать бесконтрольного увеличения количества объектов и одновременно сохраняет усложнение длительной сессии.

Расчет параметров прыжка и подката подтвердил необходимость оценивать маневры не только по вертикальной геометрии, но и по смещению мира за время действия. При неизменной длительности прыжка горизонтальная дистанция относительно препятствия увеличивается вместе со скоростью, поэтому поздние этапы требуют более ранней реакции. Для подката были сопоставлены длительность 0,22 секунды, ширина препятствия и пройденное за это время расстояние. Полученные значения позволили обосновать принятые окна управления и требования к интервалам генерации.

Функциональное тестирование охватило двенадцать основных сценариев: обычный и дополнительный прыжки, запрет повторного дополнительного прыжка, подкат, столкновения с препятствиями трех типов, прохождение препятствия Slide, действие щита, запрет подката в воздухе и восстановление состояния при рестарте. Фактические результаты соответствовали ожидаемым. Тем самым подтверждена корректность работы ключевых механик и пользовательского интерфейса.

Технико-экономический расчет показал, что оценочная стоимость разработки и демонстрационного внедрения составляет 1 243 300 рублей, годовые эксплуатационные расходы – 133 800 рублей, а чистый годовой эффект

от повторного использования и учебного применения – 247 500 рублей. Расчетный срок окупаемости равен 5,02 года.

В разделе безопасности установлено, что основные риски разработки связаны с длительной работой за компьютером, зрительной и статической нагрузкой, электробезопасностью и пожарной безопасностью рабочего места. Информационная безопасность соответствует локальному характеру приложения: PlayerPrefs используется только для некритичных настроек и рекорда, а исходные материалы подлежат резервному копированию и хранению в системе контроля версий. Экологичность поддерживается электронным документооборотом, отсутствием постоянной серверной инфраструктуры и снижением вычислительной нагрузки за счет повторного использования объектов.

Таким образом, поставленная цель достигнута, а предусмотренные задачи выполнены. Создан работающий программный комплекс, в котором архитектура, оптимизация и геймплей образуют согласованную систему. Было обеспечено структурная основа, единое состояние, расширяемые контракты и оптимизированная генерация мира. Были созданы механики, препятствия, бонусы, интерфейс, и расчётная кривая сложность. Совокупный результат подтверждает, что компонентно-модульный подход подходит для разработки казуального 2D-автораннера и позволяет выполнять балансировку через изменение локализованных параметров без перестройки всего проекта.

Перспективы дальнейшего развития RusRunner связаны с переносом конфигураций трассы и баланса в полноценные ScriptableObject-ресурсы, переходом на Input Actions для клавиатуры, мобильных устройств и геймпада, расширением автоматизированных тестов и проведением профилирования в Unity Profiler на нескольких целевых устройствах. Также целесообразно увеличить набор препятствий и визуальных тем, добавить статистику сессий, звуковое оформление, более гибкую систему прогрессии.

Ссылка на репозиторий Github:

[https://github.com/Bigryb/Diplom\\_2D\\_Autorunner](https://github.com/Bigryb/Diplom_2D_Autorunner)

## СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

- 1 Бонд, Д. Г. Unity и C#. Геймдев от идеи до реализации / Д. Г. Бонд ; пер. с англ. А. Макаровой. – 2-е изд. – Санкт-Петербург : Питер, 2021. – 928 с.
- 2 Браун, С. The C4 model for visualising software architecture [Электронный ресурс]. – URL: <https://c4model.com/> (дата обращения: 13.06.2026).
- 3 Буч, Г. Язык UML : руководство пользователя / Г. Буч, Д. Рамбо, И. Якобсон ; пер. с англ. Н. А. Мухина. – 3-е изд., электрон. – Москва : Академия АйТи : ДМК Пресс, 2022. – 495 с.
- 4 Гамма, Э. Паттерны объектно-ориентированного проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Д. Влиссидес ; пер. с англ. – Санкт-Петербург : Питер, 2022. – 448 с.
- 5 Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин ; пер. с англ. – Санкт-Петербург : Питер, 2026. – 352 с.
- 6 Об утверждении Правил противопожарного режима в Российской Федерации : постановление Правительства Российской Федерации от 16.09.2020 № 1479 [Электронный ресурс]. – URL: <https://rostransnadzor.gov.ru/documents/categories/79/document/4735> (дата обращения: 15.06.2026).
- 7 Об утверждении санитарных правил и норм СанПиН 1.2.3685-21 : постановление Главного государственного санитарного врача Российской Федерации от 28.01.2021 № 2 [Электронный ресурс]. – URL: <http://publication.pravo.gov.ru/Document/View/0001202102030022> (дата обращения: 15.06.2026).
- 8 Производственный календарь на 2026 год [Электронный ресурс] // КонсультантПлюс. – URL: [https://www.consultant.ru/document/cons\\_doc\\_LAW\\_508943/298688c6ab4b49246df4c7a5676ccea149a84868/](https://www.consultant.ru/document/cons_doc_LAW_508943/298688c6ab4b49246df4c7a5676ccea149a84868/) (дата обращения: 15.06.2026).
- 9 Трудовой кодекс Российской Федерации : Федеральный закон от 30.12.2001 № 197-ФЗ [Электронный ресурс]. – URL:

<https://pravo.gov.ru/proxy/ips/?docbody=&nd=102074279> (дата обращения: 15.06.2026).

10 Ферроне, Х. Изучаем С# через разработку игр на Unity / Х. Ферроне ; пер. с англ. – 5-е изд. – Санкт-Петербург : Питер, 2022. – 400 с.

11 Чакон, С. Pro Git / С. Чакон, Б. Штрауб. – 2-е изд. [Электронный ресурс]. – URL: <https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell> (дата обращения: 14.06.2026).

12 Шелл, Д. Геймдизайн : как создать игру, в которую будут играть все / Д. Шелл ; пер. с англ. – Москва : Альпина Паблишер, 2019. – 640 с.

13 Halfbrick Studios. Jetpack Joyride Classic [Электронный ресурс]. – URL: <https://www.halfbrick.com/games/jetpack-joyride-classic> (дата обращения: 12.06.2026).

14 Nekki. Vector [Электронный ресурс]. – URL: <https://nekki.com/vector/> (дата обращения: 12.06.2026).

15 RobTop Games. Geometry Dash [Электронный ресурс]. – URL: [https://store.steampowered.com/app/322170/Geometry\\_Dash/](https://store.steampowered.com/app/322170/Geometry_Dash/) (дата обращения: 12.06.2026).

16 Snowman. Alto's Adventure : press kit [Электронный ресурс]. – URL: [https://www.builtbysnowman.com/press/sheet.php?p=altos\\_adventure](https://www.builtbysnowman.com/press/sheet.php?p=altos_adventure) (дата обращения: 12.06.2026).

17 SYBO Games. Subway Surfers : official homepage [Электронный ресурс]. – URL: <https://subwaysurfers.com/> (дата обращения: 12.06.2026).

18 Unity Technologies. 2D game development [Электронный ресурс]. – URL: <https://docs.unity3d.com/6000.0/Documentation/Manual/Unity2D.html> (дата обращения: 12.06.2026).

19 Unity Technologies. 2D Renderer Data overview : Universal Render Pipeline 17.0 [Электронный ресурс]. – URL: <https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@17.0/manual/2DRendererData-overview.html> (дата обращения: 13.06.2026).

20 Unity Technologies. Asset metadata [Электронный ресурс]. – URL: <https://docs.unity3d.com/6000.0/Documentation/Manual/AssetMetadata.html> (дата обращения: 14.06.2026).

21 Unity Technologies. Editor settings reference : Asset Serialization [Электронный ресурс]. – URL: <https://docs.unity3d.com/6000.0/Documentation/Manual/class-EditorManager.html> (дата обращения: 15.06.2026).

22 Unity Technologies. Input System 1.13.1 : installation guide [Электронный ресурс]. – URL: <https://docs.unity3d.com/Packages/com.unity.inputsystem@1.13/manual/Installation.html> (дата обращения: 13.06.2026).

23 Unity Technologies. Introduction to Rigidbody 2D [Электронный ресурс]. – URL: <https://docs.unity3d.com/6000.0/Documentation/Manual/2d-physics/rigidbody/introduction-to-rigidbody-2d.html> (дата обращения: 15.06.2026).

24 Unity Technologies. ObjectPool<T> [Электронный ресурс]. – URL: [https://docs.unity3d.com/6000.0/Documentation/ScriptReference/Pool.ObjectPool\\_1.html](https://docs.unity3d.com/6000.0/Documentation/ScriptReference/Pool.ObjectPool_1.html) (дата обращения: 12.06.2026).

25 Unity Technologies. PlayerPrefs [Электронный ресурс]. – URL: <https://docs.unity3d.com/6000.0/Documentation/ScriptReference/PlayerPrefs.html> (дата обращения: 13.06.2026).

26 Unity Technologies. Profiler window reference [Электронный ресурс]. – URL: <https://docs.unity3d.com/6000.0/Documentation/Manual/ProfilerWindow.html> (дата обращения: 13.06.2026).

27 Unity Technologies. ScriptableObject [Электронный ресурс]. – URL: <https://docs.unity3d.com/6000.0/Documentation/Manual/class-ScriptableObject.html> (дата обращения: 13.06.2026).

28 Unity Technologies. Unity UI (uGUI) 2.0.0 [Электронный ресурс]. – URL: <https://docs.unity3d.com/Packages/com.unity.ugui@2.0/manual/index.html> (дата обращения: 13.06.2026).

## **ПРИЛОЖЕНИЕ А**

### **Руководство пользователя программного комплекса RusRunner**

#### **А.1 Назначение и область применения**

RusRunner представляет собой однопользовательскую казуальную 2D-игру жанра линейного автораннера. Во время игровой сессии персонаж автоматически продвигается вперед за счет смещения игрового мира, а пользователь управляет прыжком, дополнительным прыжком и подкатом. Цель состоит в том, чтобы преодолеть максимально возможную дистанцию, избежать столкновений с препятствиями, собрать бонусы и улучшить личный рекорд.

Руководство предназначено для пользователя настольной версии игры, управляющего персонажем с помощью клавиатуры и выбирающего пункты меню мышью. Для прохождения не требуется регистрация, подключение к сети Интернет или создание учетной записи. Настройки и лучший результат сохраняются локально на устройстве.

Основной пользовательский сценарий включает запуск приложения, настройку громкости и частоты кадров, начало забега, выполнение маневров, получение временных усилений, просмотр результата после поражения и запуск новой попытки.

#### **А.2 Установка и запуск**

Перейти на репозиторий Github проекта по ссылке:

[https://github.com/Bigryb/Diplom\\_2D\\_Autorunner](https://github.com/Bigryb/Diplom_2D_Autorunner)

Скачайте из релиза один из архивов с игрой в соответствии со своей операционной системой:

1. RusRunner\_Win.zip.
2. RusRunner\_MacOS.tar.gz.
3. RusRunner\_Linux.tar.

Распакуйте архив. Если у вас MacOS, установить и запустить игру.

Если у вас Windows, запустить из распакованного архива исполняющий файл My project.exe

### А.3 Главное меню

Главное меню является начальной экранной формой приложения. Оно содержит название игры, краткую подсказку по управлению и три основные кнопки. Внешний вид главного меню приведён на рисунке А.1. Разъяснение элементов главного меню находится в таблице А.1.

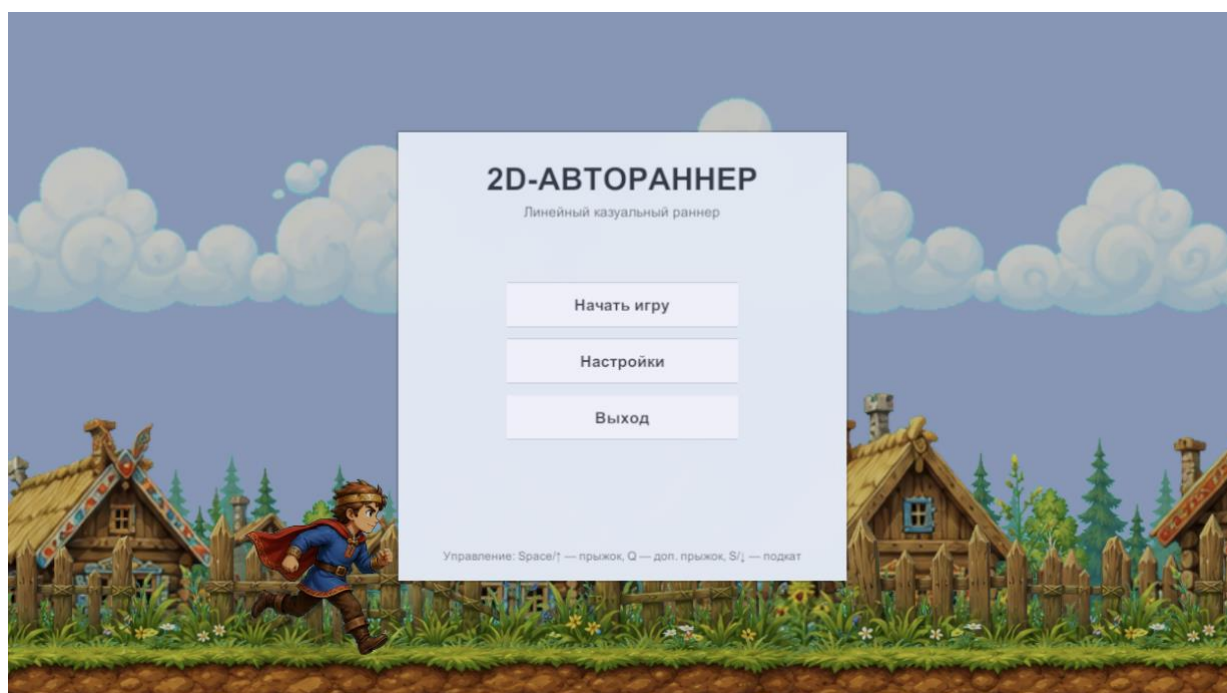


Рисунок А.1 – Главное меню RusRunner

Таблица А.1 – Назначение элементов главного меню

| Элемент                  | Назначение                                                                             |
|--------------------------|----------------------------------------------------------------------------------------|
| Начать игру              | Закрывает главное меню, сбрасывает показатели текущей сессии и запускает забег.        |
| Настройки                | Открывает экран изменения громкости музыки, громкости звуков и целевой частоты кадров. |
| Выход                    | Завершает работу приложения. При запуске в Unity Editor останавливает режим Play.      |
| Подсказка в нижней части | Напоминает клавиши прыжка, дополнительного прыжка и подката.                           |

#### А.4 Изменение настроек

Для перехода к параметрам необходимо нажать кнопку «Настройки». Экран настроек содержит два ползунка громкости и кнопки выбора частоты кадров. Внешний вид окна настроек приведён на рисунке А.2.

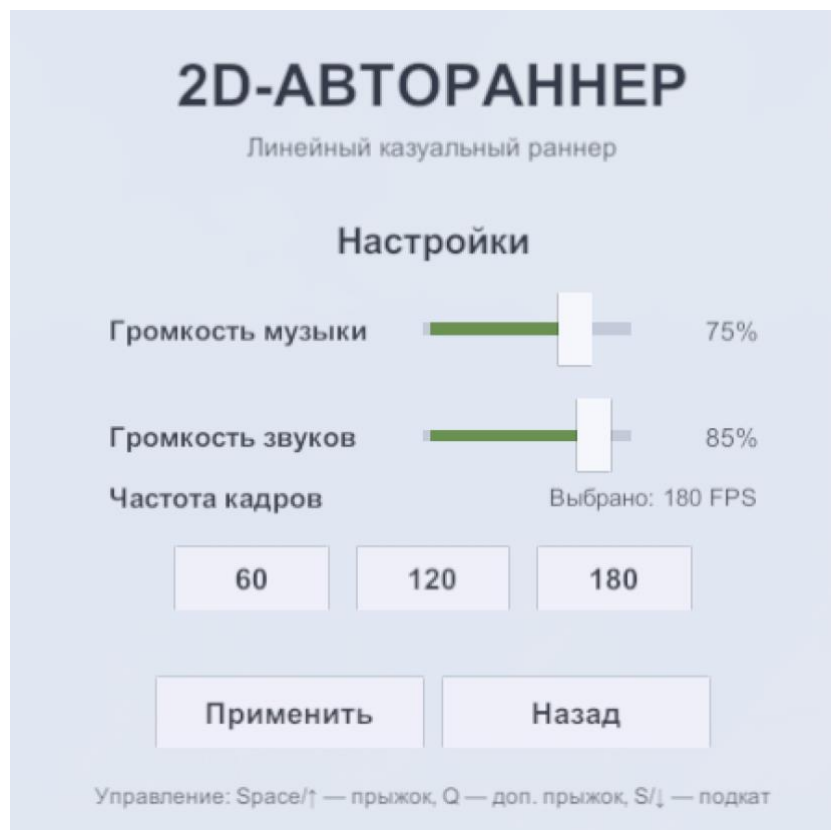


Рисунок А.2 – Экран настроек

Ползунок «Громкость музыки» задает значение от 0 до 100 %

Ползунок «Громкость звуков» – уровень звуковых эффектов.

Кнопками 60, 120 и 180 выбирается целевая частота кадров. Текущее выбранное значение отображается рядом с подписью «Частота кадров».

Кнопка «Применить» сохраняет параметры и возвращает пользователя в главное меню. Кнопка «Назад» закрывает экран без сохранения внесенных изменений. При следующем запуске приложения применяются последние сохраненные значения. На устройствах, не способных поддерживать выбранную частоту кадров, фактическое значение может быть ниже целевого.

## А.5 Начало и цель игрового забега

После нажатия кнопки «Начать игру» главное меню скрывается, персонаж переходит в состояние бега, а элементы трассы начинают смещаться справа налево. Пользователь не управляет горизонтальным перемещением: необходимо своевременно реагировать на препятствия и собирать бонусы. Пример игрового экрана приведен на рисунке А.3.

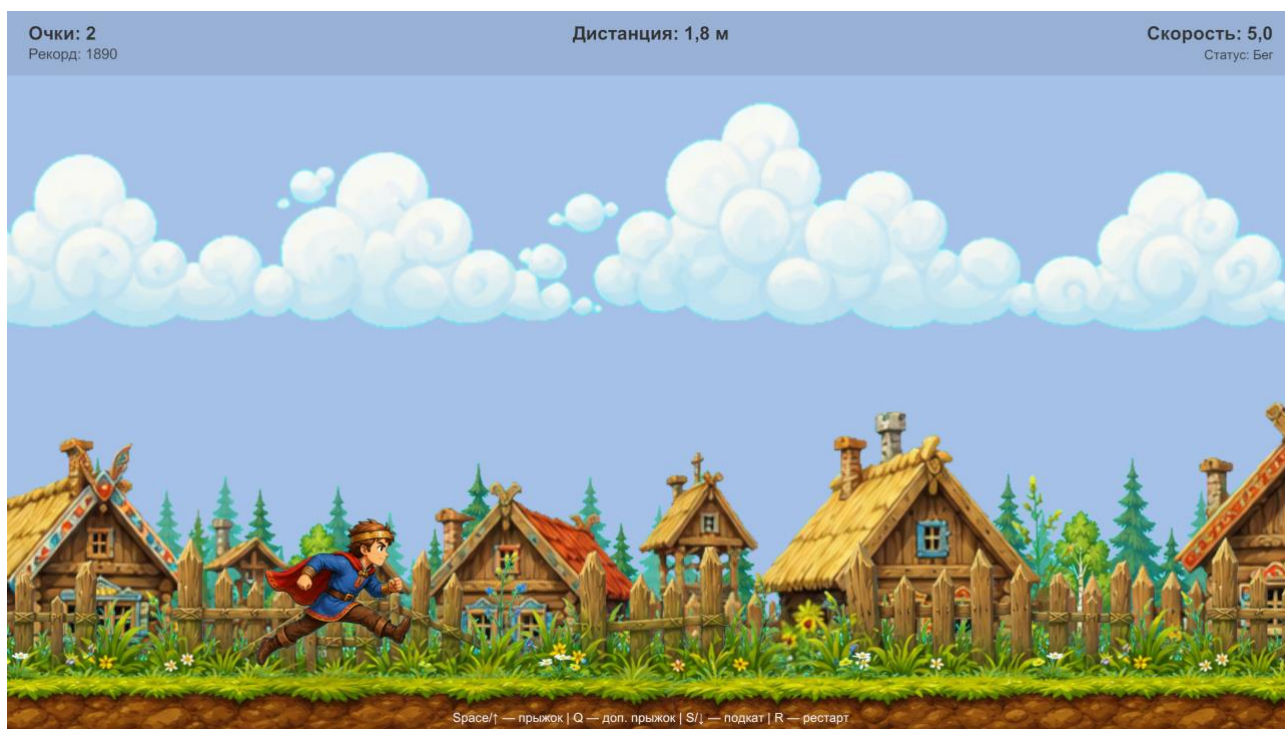


Рисунок А.3 – Игровой экран

В начале забега скорость невысока, затем она постепенно увеличивается. Одновременно меняются профили генерации трассы, поэтому препятствия появляются чаще и образуют более сложные последовательности. Игровая сессия продолжается до столкновения с опасным объектом, если в момент контакта не действует защитный бонус и не выполняется допустимый маневр.

## А.6 Управление персонажем

В таблице А.2 описаны все возможные действия для управления персонажем.

Таблица А.2 – Управление в настольной версии

| Клавиша или действие | Команда               | Условия и результат                                                                                  |
|----------------------|-----------------------|------------------------------------------------------------------------------------------------------|
| Space или ↑          | Прыжок                | Выполняется с поверхности. После прыжка персонаж получает возможность одного дополнительного прыжка. |
| Q                    | Дополнительный прыжок | Доступен один раз до следующего приземления.                                                         |
| S или ↓              | Подкат                | Выполняется только на земле. Для продолжительного подката клавишу следует удерживать.                |
| R                    | Повторный запуск      | Срабатывает после поражения и немедленно запускает новую попытку.                                    |

Обычный прыжок нельзя повторять, пока персонаж находится в воздухе. Для коррекции траектории можно использовать дополнительный прыжок, однако количество дополнительных прыжков ограничено одним. Подкат не запускается в воздухе и позволяет пройти под высокой преградой.

## А.7 Элементы игрового интерфейса

В таблице А.3 указаны разъяснение для всех элементов игрового интерфейса.

Таблица А.3 – Элементы HUD

| Элемент       | Описание                                                                                                                               |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Очки          | Текущий результат. Значение увеличивается по мере прохождения дистанции и быстрее растет при активном бонусе удвоения очков.           |
| Рекорд        | Лучший сохранённый результат предыдущих и текущей игровых сессий.                                                                      |
| Дистанция     | Условное расстояние, пройденное в текущем забеге, в метрах.                                                                            |
| Скорость      | Текущая скорость прокрутки игрового мира. При ускорении отображаемое значение возрастает.                                              |
| Статус        | Текущее состояние персонажа: бег, подкат или поражение; также отображаются щит, множитель x2 и число доступных дополнительных прыжков. |
| Нижняя строка | Постоянная краткая подсказка по клавишам управления.                                                                                   |

HUD отображается только после начала забега. Во время активного щита в строке состояния появляется отметка «щит», при удвоении очков –

обозначение «х2». После поражения скорость становится равной нулю, а статус изменяется на «поражение».

## **А.8 Препятствия**

На трассе встречаются три типа препятствий. Их внешний вид приведен на рисунках А.4–А.6, а требуемая реакция пользователя описана в таблице А.4.



Рисунок А.4 – Наземное препятствие



Рисунок А.5 – Препятствие для подката



Рисунок А.6 – Воздушное препятствие

Таблица А.4 – Способы преодоления препятствий

| Тип препятствия         | Действие                                                                                               |
|-------------------------|--------------------------------------------------------------------------------------------------------|
| Наземное препятствие    | Выполнить обычный прыжок заранее, при необходимости скорректировать траекторию дополнительным прыжком. |
| Препятствие для подката | Сделать подкат и пройти под препятствием, либо перепрыгнуть с помощью прыжка и дополнительного прыжка. |
| Воздушное препятствие   | Безопасно оставаться на земле, либо изменить траекторию прыжка дополнительным прыжком.                 |

Столкновение с препятствием завершает забег. Исключением является контакт во время действия щита.

## А.9 Бонусы

Бонусы размещаются над поверхностью трассы и подбираются при соприкосновении с персонажем. Их внешний вид показан на рисунках А.7–А.9. Каждый эффект действует ограниченное время и не требует отдельной клавиши активации. В таблице А.5 описаны эффекты и продолжительность бонусов.



Рисунок А.7 – Бонус «Щит»



Рисунок А.8 – Бонус «Удвоение очков»



Рисунок А.9 – Бонус «Ускорение»

Таблица А.5 – Назначение бонусов

| Бонус          | Продолжительность | Эффект                                                                                                                                     |
|----------------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Щит            | 5 секунд          | Временно включает неуязвимость. Контакт с препятствием не завершает забег, а препятствие деактивируется.                                   |
| Удвоение очков | 6 секунд          | Увеличивает множитель начисления очков до x2. Наличие эффекта отображается в строке состояния.                                             |
| Ускорение      | 4 секунды         | Увеличивает эффективную скорость примерно на 25 %. Одновременно быстрее растут дистанция и количество очков, но сокращается время реакции. |

Повторный подбор бонуса того же типа обновляет его таймер. После истечения времени параметры автоматически возвращаются к обычным значениям.

### А.10 Завершение забега и повторный запуск

После поражения движение персонажа и трассы останавливается, а поверх игрового экрана появляется итоговая панель. На ней отображаются очки, рекорд и пройденная дистанция. Внешний вид панели приведен на рисунке А.10.

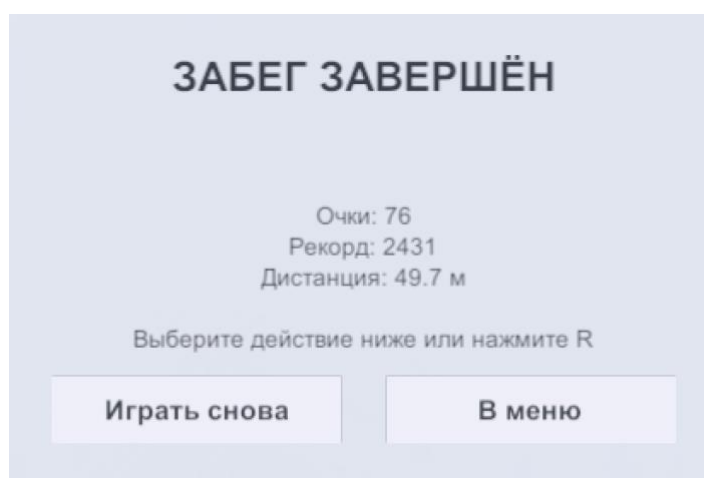


Рисунок А.10 – Итоговая панель после поражения

Кнопка «Играть снова» перезагружает игровую сцену и немедленно начинает новую попытку. Аналогичное действие выполняет клавиша R. Кнопка «В меню» перезагружает сцену и возвращает пользователя к главному меню. Перед новой попыткой сбрасываются очки, дистанция, скорость, активные бонусы, положение и форма коллайдера персонажа.

Если результат текущего забега превышает сохраненный рекорд, новое значение автоматически записывается в локальное хранилище и отображается в следующих сессиях.

### **A.11 Сохранение данных и завершение работы**

Приложение сохраняет локально громкость музыки, громкость звуков, выбранную частоту кадров и лучший результат. Текущая дистанция, текущие очки и временные бонусы между запусками не сохраняются. Удаление локальных данных приложения приводит к возврату настроек к значениям по умолчанию и сбросу рекорда.

Для штатного завершения работы следует вернуться в главное меню и нажать кнопку «Выход». Принудительное закрытие окна не повреждает проект, однако для сохранения измененных параметров на экране настроек предварительно должна быть нажата кнопка «Применить».

## ПРИЛОЖЕНИЕ Б

### Листинги кодов

#### Б.1 Листинг кода ограничения дополнительного прыжка

```
public bool TryAirJump(GameContext context)
{
    if (context.IsGrounded || context.AirJumpsLeft <= 0)
        return false;

    context.AirJumpsLeft--;
    JumpImpulse(jumpForce * 0.94f);
    return true;
}
```

#### Б.2 Листинг кода обновления таймеров активных бонусов

```
for (int i = _active.Count - 1; i >= 0; i--)
{
    ActivePowerUp item = _active[i];
    item.Remaining -= deltaTime;
    item.PowerUp.Tick(context, deltaTime);

    if (item.Remaining <= 0f)
    {
        item.PowerUp.Expire(context);
        _active.RemoveAt(i);
    }
}
```